Securing People and their Machines Against Major Faults (Full Version)

Ohad Eitan[1], Idit Keidar[1], and Ehud Shapiro[2]

[1] Technion – Israel Institute of Technology, Haifa, Israel
[2] London School of Economics and Political Science, UK and Weizmann Institute of Science, Israel

Abstract. We consider grassroots platforms—distributed systems of agents consisting of people identified by self-chosen public keys and their machines (smartphones)—and wish to make them secure against major faults: the loss of their private keys and/or their smartphones. As grassroots platforms have no global resource to rely on for recovery, our peer-based solution is based on: (*i*) a grassroots social graph in which agents establish and maintain friendships; (*ii*) identity custodians, designated by each person, and (*iii*) state custodians, which are grassroots platform-specific. Upon a person experiencing identity loss, and given a willing supermajority of their identity custodians, the person's friends replace the old public key with the new one across the graph and restore friendships, where all friends serve as state custodians for the social graph.

We specify the social graph and its secure version as guarded multiagent atomic transactions, and implement the secure social graph via communicating volitional agents, an asynchronous message-passing model one step closer to implementation. We prove the implementation maps runs with recoverable faults to correct runs of the specification.

We follow a similar path for grassroots coins and bonds, showing a common core as well as the platform-specific aspects of state recovery: a currency's single-writer log is recovered exactly, the recovered sovereign resuming without double-spending.

Keywords: Grassroots Protocols · Multiagent Transition Systems · Atomic Transactions · Major Faults · Key Recovery · Social Networks

1 Introduction

Grassroots platforms. The Internet today is dominated by global platforms: centralised—social networks, Internet commerce, 'sharing-economy'—with autocratic control [48,49], and decentralised—blockchains and cryptocurrencies [12, 13,30,45–47]—with plutocratic control [7]. Grassroots platforms [36–38,41] aim to offer an egalitarian alternative. Grassroots platforms can have multiple instances that emerge and operate independently of any global resource except the network, yet interoperate and coalesce once interconnected into ever-larger

instances, possibly a single global one. Key grassroots platforms include grassroots social networks [22,38], grassroots coins [21,36,39], and grassroots democratic federations [14,17,41].

Recovery from major faults. Grassroots platforms consist of people operating their personal machines (smartphones). A person's identity is a self-chosen cryptographic key, and a platform's state resides only on the personal machines of its participants. A participant may suffer a major fault: the loss of their private key (identity loss) and/or their machine state (state loss). As grassroots platforms have no central authority or global infrastructure to recover from, recovery must be peer-based; this is the subject of the paper.

Grassroots platforms have been specified via volitional multiagent atomic transactions [22], each agent being a person-machine pair and each atomic transaction guarded by the volitions of the participating persons. We use this foundation to specify recovery from major faults.

Upon choosing a key, each person designates a set of trusted identity custodians, together with a supermajority threshold; the custodians authorise replacement of the person's key should the key be lost or compromised. Recovery is then guarded by a willing supermajority of a person's identity custodians, upon which the friends of the person replace the old public key with the new one across the social graph, preserving their friendships. A fault that leaves the key intact—loss of machine state—needs no custodian authorisation: the person proves their identity by signing with the retained key, and the social graph is restored from the friends that still record it, with every friend serving as a state custodian of the social graph. These custodians are the person's real-world friends, known off-chain, so a recovering person can reach a subset of them out of band to initiate recovery.

The social graph is the foundational platform: it carries identities and recovers them. Every higher platform is built upon it, delegating identity recovery to the social graph and adding only the recovery of its own state, held by platform-specific state custodians; we give such a protocol for grassroots coins. Recovery has a common core: it is peer-based, carried by state custodians, delegates identity recovery to the social graph, is grassroots by construction, and is privacy-preserving. Here we show two instances of this common core: the social graph and grassroots coins.

Fault model. We assume that every agent's machine runs the same attested protocol code inside a trusted execution environment [32], with mutual remote attestation among agents' machines [9]. An attested machine executes the protocol as written; an attacker controlling the host can still (1) roll the enclave back to an earlier state [18,26], (2) manipulate its clock, or (3) run several instances of the same enclave [6]. Our protocol is robust to these attacks, since (1) we store nothing outside the enclave, so there is no stored state to roll back to; (2) no transaction reads a clock; (3) the protocol is robust to duplication and reordering, so if messages are partitioned between multiple enclaves, the code still behaves correctly (Lemma 11). Since the protocol overcomes state loss, it is also robust to message loss. An agent therefore fails only by losing its private key

and/or its machine state, not by deviating from the protocol or misreporting its state. As in [18], recovery is peer-based— it relies on the friends and custodians of a recovering agent reporting their state faithfully. Liveness requires that a supermajority of agents' identity custodians, and currency's state custodians, are functional. Every fault is eventually repaired, after a finite but unbounded delay, and a fault occurring during a recovery simply re-triggers it. The results we state at quiescence are over runs with finitely many faults. The only adversarial capability we admit is an attacker holding a person's private key, whose effect is confined to the compromised identity and ends once the key is replaced (Appendix D.16). Communication among correct agents is reliable, whereas agent failure may cause transient message loss to and from the failed agent. There is no bound on message latency, i.e., the system is asynchronous.

Secure grassroots coins. In grassroots coins [36,39], each agent is the sovereign of their own currency: they mint coins—a coin with a future maturity date is a bond—and every transaction in their coins requires their participation. The Grassroots Flash payment system [21] realises this by having the sovereign approve each payment in their coins, with the sovereign's personal blockchain serving as the authoritative ledger for their currency. This ledger is append-only and hence monotonic. If the sovereign loses this ledger, they may inadvertently approve a double-spend—approving payment of a coin that was already paid to someone else. Designated friends—state custodians—participate in every transaction and thereby maintain up-to-date copies of the sovereign's ledger. Since the ledger is monotonic, a single available state custodian suffices when updates are atomic; the asynchronous implementation instead couples finality to a supermajority of state custodians, recovering the log exactly (Appendices E and F).

Theft. An identity thief can transact as the person at all grassroots platforms—befriending, unfriending, paying and minting. As a practical urgent step, and prior to identity custodians willing key replacement, the victim may personally warn their friends and request them to withhold transactions with the stolen identity; they can abide, for a while, without violating the protocol. Upon recovery, the person can restore any damage to their social graph. Recovering from final fraudulent transactions in other people's coins is impossible. However, recovering from fraudulent transactions in one's own currency (i.e., not re-issuing them in the new currency) can be done with the approval of the old currency's custodians. (Section 7).

What we prove. We develop each platform at three levels of abstraction: an abstract specification as guarded multiagent atomic transactions; a secure specification, again as guarded transactions, adding recovery from major faults; and an implementation as communicating volitional agents (CVA), an asynchronous message-passing model one step closer to implementation. For the social graph we prove that the secure specification—in which each agent also records its 'friends-of-friends', for state recovery—is a fault-resilient implementation of the abstract one (Theorem 4), and that its CVA implementation realises the secure specification at quiescence (Theorem 6), and hence the social graph (Corollary 1): eventually every quiescent state maps correctly to a specification state. At qui-

escence every friendship still recorded by a friend of either party is preserved; a friendship whose only records, at the moment of an identity loss, are the two friends themselves, is lost. We further establish the soundness of every reachable implementation state, and that every level is grassroots.

We then follow a similar path for grassroots coins and bonds, showing a common core as well as the platform-specific aspects of state recovery: a totally-ordered, single-writer transaction log requires exact recovery, obtained by coupling finality to a supermajority of state custodians, so the recovered sovereign resumes without double-spending.

Paper outline. Section 2 recalls guarded multiagent atomic transactions, the formalism in which grassroots platforms and their recovery are specified, with formal details deferred to Appendix C. Section 3 sets out the recovery framework: identity recovery as the foundational service of the social graph, and platform-specific state recovery layered upon it. Section 4 specifies the social graph and the secure social graph, and proves that the secure social graph implements the social graph. Section 5 introduces communicating volitional agents: the specifications above are atomic multiagent transactions, which an implementation on networked smartphones cannot provide, so CVA refines them to an asynchronous message-passing model one step closer to implementation. Section 6 summarises the CVA implementation of the secure social graph, with the full development and proofs in Appendix D. Section 7 follows a similar path for grassroots coins, the recovery framework's second instance, summarising their specification and CVA implementation. Section 8 discusses related work. Appendix B surveys formal models of persons in concurrent systems. Appendices E and F develop secure grassroots coins, with their ordinary and secure specification as volitional multiagent atomic transactions and implementation as communicating volitional agents, with recoverable finality and exact recovery.

2 Grassroots Guarded Multiagent Atomic Transactions

This section recalls guarded multiagent atomic transactions [22], a high-level formalism for specifying grassroots platforms as systems of agents, each consisting of a person operating a machine (e.g., smartphone). A guarded atomic transaction specifies how the machine states of the participating agents change, and which of the persons (if any) participating in the transaction must be willing for the transaction to occur.

We assume a potentially infinite set of agents Π, considering only finite subsets $P \subset \Pi$. We use S^P to denote the set of all total functions from P to S, and for $c \in S^P$ use c_p for the member of c indexed by p.

Definition 1 (Machine State, Configuration, Transaction, Guarded Transaction). *Given an arbitrary set S of **machine states**, with a designated **initial state** $s0 \in S$, and agents $Q \subset \Pi$, a **machine configuration** over Q is a member of S^Q, and a **machine transaction** over **participants** Q is a pair $c \to c' \in (S^Q)^2$ such that $c \neq c'$. Given such a machine transaction t, a **guarded transaction** over t is a pair (t, Q') where $Q' \subseteq Q$ are its **guards**.*

Guarded atomic transactions can be carried out by their participants regardless of the states of non-participants. Participants include both active agents (whose state changes) and stationary agents (whose state is a precondition but does not change). When a transaction is "guarded by $\{p, q\}$," both must be willing; when it is "guarded by either p or q," there are two guarded transactions over the same machine transaction, $(t, \{p\})$ and $(t, \{q\})$, so that either person's volition suffices.

The operational semantics of guarded multiagent atomic transactions [22] maps them to volitional multiagent atomic transactions [40], recalled in Appendix C. A guarded transaction is enabled when each participant is in the precondition machine state and every guard wills it (Definition 9); a transaction whose guard is empty is unguarded, enabled whenever its machine precondition holds.

A set of multiagent atomic transactions in turn induces a multiagent transition system [35], a special case of "standard" transition systems. A family of multiagent transition systems, parametrised over the set of participating agents, defines a protocol. We are particularly interested in grassroots protocols [22,37,40], defined informally as follows. Runs of two disjoint sets can be interleaved into a run of their union. A protocol is **oblivious** if every interleaving of two correct runs of disjoint sets of agents is a correct run of the union, and **interactive** if some correct run of the union is not an interleaving of any two such independent runs, because it includes a step whose participants span both sets; it is **grassroots** if both [22], capturing formally the informal idea that grassroots platforms (specified by grassroots protocols) can have multiple instances that can form and operate independently, yet may coalesce into ever-larger instances, possibly (but not necessarily) into a single global platform.

3 Recovery of Identity and State after Major Faults

The social graph is the foundational platform: it carries identities and recovers them as well as its own state, and every higher platform is built upon it. A higher platform does not recover identities itself; it delegates identity recovery to the social graph, and adds only the recovery of its own platform-specific state.

Identity recovery. Upon choosing a key, each person designates a set of identity custodians together with a supermajority threshold, fixed at identity creation. Upon identity loss, the person chooses a fresh keypair off-chain, obtains a new machine if needed, and convinces a supermajority of their identity custodians to authorise the replacement; the friends of the person then replace the old public key with the new one across the social graph, preserving their friendships.

A higher platform recovers an identity by delegating to the social graph: its agents are social-graph identities, and once the social graph has replaced a lost key, the higher platform's recovery proceeds against the renamed agent.

State recovery. Recovery from state loss is carried by state custodians: peers that hold the state from which a recovering agent restores. State recovery has three components on every platform: (i) the state custodians and what each

	Social graph	**Grassroots coins**
Recoverable state	friendships, each co-owned by two friends	the sovereign's transaction log, a single-writer total order
State custodians	every friend	a designated subset of the sovereign's friends
Maintenance	atomic friendship formation, lazy dissemination of friend-of-friend records	atomic log update across a super-majority of custodians
Read-back	each friendship from its friend	from a supermajority of custodians, adopting the longest

Table 1. State recovery: a shared core filled differently by each platform.

holds; (*ii*) the maintenance discipline that keeps the custodians' holdings current; and (*iii*) the read-back by which a recovering agent reconstructs its lost state. What is platform-specific is the filling of the three—who the custodians are, what discipline maintains their copies, and how a recovering agent reads its state back. State recovery is privacy-preserving, in that the machine of a state custodian stores a friend's state without its person having access to it.

The approaches used in the social graph and grassroots coins are summarised in Table 1. The secure social graph maintains friends-of-friends lists. In the message-passing setting, direct friendships are updated via distributed transactions, whereas friend-of-friend lists are periodically disseminated in a lazy manner. At the abstract level, a single friend acting as state custodian is enough for recovering the agent's full friend list. In the message-passing setting, each friend's periodic checkpoint restores the friendship it co-owns, and friend-of-friend lists are recovered in the background using the normal mode of operation. Note that even without failures, lazy updates imply that the secure social graph is only eventually consistent. Namely, it converges to the abstract graph (specified using atomic transactions) in quiescence, but may temporarily diverge from it while updates are taking place. These convergence semantics are preserved under failure and recovery.

4 The Social Graph

The grassroots social graph is the backbone of a grassroots platform: an evolving symmetric friendship relation among agents, each an autonomous person-machine pair. We specify it as guarded multiagent atomic transactions (Section 2) at two levels of fault-tolerance. The social graph maintains friendships under fault-free operation. The secure social graph adds recovery from the loss, compromise, or corruption of a person's key or machine. We then show that the secure social graph implements the social graph: under fault-free operation, and under recovery that does not lose all records of a friendship, the secure social graph exhibits exactly the friendships of the social graph at quiescence, except when a fault erases a friendship's last surviving record.

4.1 The Social Graph

Each agent $p \in \Pi$ is a person-machine pair, identified by a self-chosen public key. No further attributes are associated with an agent at this level. The machine state of $p \in P$ is its friend set $F_p \subseteq P$, initialised to $\emptyset$.

Definition 2 (Social Graph Transactions [22]). *The* ***social graph transactions*** *are:*

1. ***Befriend***(p, q)*:* $F'_p := F_p \cup \{q\}$, $F'_q := F_q \cup \{p\}$, *provided* $q \notin F_p$. *Guarded by* $\{p, q\}$.
2. ***Unfriend***(p, q)*:* $F'_p := F_p \setminus \{q\}$, $F'_q := F_q \setminus \{p\}$, *provided* $q \in F_p$. *Guarded by either* p *or* q.

Befriending requires both persons to will the transaction; unfriending can be initiated by either. In Appendix H, we prove the following:

Lemma 1 (Friendship Mutuality). *In any run of the social graph,* $q \in F_p \iff p \in F_q$ *for all* $p, q \in P$.

Theorem 1 (The Social Graph is Grassroots). *The social graph is grassroots.*

4.2 The Secure Social Graph

The secure social graph recovers from two kinds of major fault. Identity loss is the loss of a person's private key, possibly together with the machine, triggering a Replace transition. State loss is the loss of machine state with the key retained, which is fixed by a Recover transition. Recovery is friendship-based, drawing only on the state held by the recovering agent's friends, with no global resource.

Two ingredients are added. First, each agent carries an intrinsic, immutable identity record: a finite nonempty set of identity custodians and a supermajority threshold, fixed at identity creation. A supermajority of the identity custodians authorises the replacement of a person's key. Second, each agent records not only its own friends but, for each friend, that friend's friend set—so that a recovering agent can read its friendships back from any one friend. Every friend thereby serves as a state custodian of the agent's friendships. Because the transactions are atomic, every friend holds the same, current record; one friend therefore suffices for recovery.

Agents. Each agent $p \in \Pi$ carries an intrinsic immutable identity record $IR_p = (K_p, \sigma_p)$, where $K_p \subset \Pi$ is a finite nonempty set of identity custodians and $\sigma_p \in (1/2, 1]$ a supermajority threshold. A supermajority of K_p is a subset $G \subseteq K_p$ with $|G| \geq \lceil \sigma_p \cdot |K_p| \rceil$. We assume each person befriends their identity custodians and keeps at least a supermajority of them as friends, so a supermajority of K_p lies within F_p and the guard of Replace can be chosen within its participants. Immutability means that no transaction adds or removes a custodian or changes the threshold; Replace(p, p') substitutes p' for p in the custodian sets naming p, as in every other record of p.

Machine state. The machine state of $p \in P$ is a pair (IR_p, N_p): the immutable identity record IR_p and the friend-of-friend map $N_p : P \rightharpoonup 2^P$, a partial map in which $N_p(q)$ is p's record of q's friend set. The friend set $F_p := \mathrm{dom}(N_p)$ is the set of agents p records as friends; initially N_p is the empty map, so $F_p = \emptyset$.

Definition 3 (Secure Social Graph Transactions). *The* ***secure social graph transactions*** *are:*

1. ***Befriend***(p, q)*: with participants* $\{p, q\} \cup F_p \cup F_q$*, provided* $q \notin F_p$*:*
   - $F'_p := F_p \cup \{q\}$ *and* $F'_q := F_q \cup \{p\}$*;*
   - $N'_p(q) := F'_q$ *and* $N'_q(p) := F'_p$*;*
   - *for each* $r \in F_p$*:* $N'_r(p) := F'_p$*;* *for each* $r \in F_q$*:* $N'_r(q) := F'_q$*.*

   Guarded by $\{p, q\}$*.*
2. ***Unfriend***(p, q)*: with participants* $\{p, q\} \cup F_p \cup F_q$*, provided* $q \in F_p$*:*
   - $F'_p := F_p \setminus \{q\}$ *and* $F'_q := F_q \setminus \{p\}$*;*
   - *for each* $r \in F'_p$*:* $N'_r(p) := F'_p$*;* *for each* $r \in F'_q$*:* $N'_r(q) := F'_q$*.*

   Guarded by either p *or* q*.*
3. ***Recover***(p, q)*: with participants* $\{p, q\} \cup N_q(p)$*, unguarded, provided* $p \in F_q$*:*
   - $F'_p := N_q(p)$*;*
   - $N'_p(r) := F_r$ *for each* $r \in N_q(p)$*.*
4. ***Replace***(p, p')*: with participants* $\{p, p'\} \cup F_p \cup \bigcup_{r \in F_p} F_r$*, provided* $F_{p'} = \emptyset$ *and* p' *is a fresh agent not yet befriended by anyone:*
   - $F'_{p'} := F_p$ *and* $N'_{p'} := N_p$*, and* $F'_p := \emptyset$*,* $N'_p :=$ *the empty map;*
   - *for each* $r \in F_p$*:* $F'_r := (F_r \setminus \{p\}) \cup \{p'\}$*, and* $N'_r(p') := N_r(p)$*;*
   - *for each* $f \in F_p$ *with* $p \in K_f$*:* $K'_f := (K_f \setminus \{p\}) \cup \{p'\}$*;*
   - *for each* $r \in F_p$ *and each* $s \in F_r$ *with* $p \in F_s$*:* $N'_s(r) := F'_r$*.*

   Guarded by a supermajority $G \subseteq K_p$ *of* p*'s identity custodians.*

Definition 4 (State-Loss Fault). *A* ***state-loss fault*** *at* p *sets* N_p *to the empty map, so* $F_p = \emptyset$*, leaving* IR_p *and the state of every other agent unchanged.*

The set of state-loss faults is disjoint from the secure social graph transactions (Definition 3): a fault erases state rather than transforming it under the protocol.

Befriend and Unfriend update the friend-of-friend map atomically: whenever p's friend set changes, every friend's record $N_r(p)$ is updated in the same transaction, so the map stays exact. Recover rebuilds the friend set from a single friend's record $N_q(p)$, which—being exact and unchanged during the loss—is p's pre-loss friend set. Replace models identity loss: a supermajority of p's identity custodians authorises substituting a fresh p' for p across the graph, p' adopting p's friendships and every friend rewriting p to p'. Setting $F'_p := \emptyset$ retires the abandoned identity abstractly; it is not a write to the old machine, whose disk may still hold p's data under the dead key.

In Appendix H, we prove the following:

Lemma 2 (Friend-of-Friend Exactness). *In any run of the secure social graph in which no state loss occurs, for every* $p \in P, q \in F_p$*:* $N_p(q) = F_q$*.*

Theorem 2 (The Secure Social Graph is Grassroots). *The secure social graph is grassroots.*

4.3 The Secure Social Graph Implements the Social Graph

We show that the secure social graph implements the social graph of Section 4.1 (Definition 2), in the sense of Definition 11. The one fault no friendship-based recovery can repair is a state loss in which two friends are the only ones recording their friendship, and both lose their state before either recovers.

Definition 5 (Friendship-Preserving Run). *A friendship is unrecoverable in a run of the secure social graph if a state loss erases its last surviving record, or an identity loss replaces one party's key while no third agent records the friendship; otherwise it is recoverable. A run is friendship-preserving if every Recover(p, q) restores all the recoverable friendships in p's pre-loss friend set, and every Replace(p, p') reaches every recoverable friend of p.*

Definition 6 (Projection). *Let σ map a secure-social-graph configuration c to the social-graph configuration $\sigma(c)$ with the same agents, the same volitional states, and machine state*

$$\sigma(c)_p := F_p \cup \{r \in P : p \in F_r\}$$

for each p (discarding IR_p and N_p), with each replaced agent q' relabelled to the original q it replaced.

The projected friend set of p is the set of agents p records as friends together with those that record p as a friend. This is symmetric by construction—$q \in \sigma(c)_p \iff q \in F_p \vee p \in F_q \iff p \in \sigma(c)_q$—so every projected configuration satisfies Friendship Mutuality (Lemma 1) and is a configuration the social graph reaches. An agent p that has lost its state, with $F_p = \emptyset$, projects to $\{r : p \in F_r\}$, the friendships still recorded at its friends; the projection thus reads a state-lost agent's friendships from the friends that survive, so a state loss and the Recover that follows it leave $\sigma(c)$ unchanged whenever some friend records the edge. When two friends that are each other's only record both lose their state, neither records the edge, so it is absent on both sides of $\sigma(c)$—symmetric, the consistent state the social graph reaches by Unfriend.

Theorem 3 (Implementation). *With the projection σ of Definition 6, the secure social graph is an implementation of the social graph in the sense of Definition 11, correct and complete in the sense of Definition 12.*

Theorem 4 (Fault-Resilience). *With the projection σ of Definition 6 and F the state-loss faults (Definition 4), the secure social graph is an F-resilient implementation of the social graph (Definition 13), restricted to friendship-preserving runs (Definition 5).*

With $F = \emptyset$ this is the correctness of Theorem 3; the theorem extends it to runs that perform state-loss faults and recover from them, every recoverable friendship being restored and the unrecoverable residue excluded by friendship-preservation.

5 Communicating Volitional Agents

The framework of Section 2 abstracts over how a guarded multiagent atomic transaction is carried out: a k-ary transaction simultaneously updates the machine states of all its participants, with no notion of how participating machines come to act in concert. An implementation on networked smartphones has no such simultaneity: every exchange between two agents is a separate asynchronous message-passing event, and a k-ary transaction must be realised by a protocol composed of such events.

Here we present **communicating volitional agents** (CVA) [22], an intermediary level between the abstract guarded-transaction specification and its implementation on networked smartphones. A CVA protocol's guarded machine transactions are drawn from four syntactic forms only: binary **discover** transactions by which an agent comes to know of another; binary **communicate** transactions that copy a message from one agent's outbox to another's inbox; and unary **platform** transactions. Thus, a CVA platform need only specify its platform transactions, changing an agent's platform-specific state. Moreover a CVA protocol is grassroots by construction (Theorem 5).

Local states. A CVA agent's local state packages five components: a set of known peers with whom the agent can communicate, an outbox of messages awaiting delivery, an inbox of messages received, and a platform state in which each platform specifies whatever data it requires (friend sets, groups, bonds, feeds, and so on). A message is a triple of sender, recipient, and cargo; outboxes and inboxes are sets of messages. The cargo space C and the platform state space A are parameters of the CVA platform.

The formal definitions—CVA local states and configurations, and the CVA protocol, comprising Discover, Communicate, and the platform transactions—are recalled in Appendix C, together with the Known-Peers and Outbox Containment lemmas and the following, proven in Appendix H:

Theorem 5 (CVA Grassroots). *Every CVA protocol is grassroots.*

6 CVA Implementation of the Secure Social Graph

We implement the secure social graph as communicating volitional agents (Section 5): the abstract Befriend, Unfriend, StateLoss, Recover, and Replace transactions of Section 4.2 are realised over CVA's asynchronous message discipline, with no global resource. This section summarises the construction and states its main results; the full development—platform state, transaction specifications, supporting invariants, and proofs—is given in Appendix D.

Construction. A friendship is encoded by an epoch counter whose parity gives its status: an odd epoch is active, an even epoch inactive. Befriend is an offer/accept handshake at a fresh odd epoch; Unfriend is unilateral, advancing the epoch to the next even value; and every change an agent makes to one of its friendships is disseminated to its friends as a stream update, so that friend-of-friend views converge.

On top of this friendship machinery the security layer adds each agent's intrinsic, immutable identity record—its custodian set and supermajority threshold—which is carried on Befriend and stored by each friend alongside the friendship.

One recovery mechanism. Both faults are repaired by a single mechanism, periodic re-broadcast and absorption, with identity loss recovery prepending one step. Each agent periodically re-broadcasts a snapshot of its friendships, carrying its identity record, to its friends, and a recipient absorbs it monotonically. A recovered agent that lost its machine state but kept its key is still recorded by its friends, so their checkpoints keep arriving and it rebuilds passively (Restore, Appendix D.15)—state recovery with no further machinery. An agent that lost its key recovers as a fresh identity p' that no friend yet records; the one extra step is to install p' at the friends, a custodian-authorised rename of p to p' (the Replace cascade below), after which p' is in the position of a state-lost agent and rebuilds by the same absorption. State loss is thus the special case in which the install is a no-op, the key being retained; identity loss is the case in which the install is custodian-authorised.

Replace: installing the recovered identity. Identity loss is repaired by the Replace cascade (Appendix D.16), the conceptual heart of the construction. It proceeds in three phases. Phase 1 is off-protocol: the person behind the lost agent p chooses a fresh agent p' and convinces a supermajority of p's custodians to authorise the replacement. Phase 2 is on-protocol vouching: each willing custodian sends p' a vouch carrying its own view of p's friends, bootstrapping p''s knowledge of whom to notify. Phase 3 is the cascade: once a supermajority has vouched, p' sends new_identity to every friend of p it has learned of; each recipient verifies the vouching set against the identity record it stored for p on befriend, renames p to p' preserving the epoch, and returns a rebind carrying its own view of p's friends—which may reveal friends no voucher named. The cascade thus reaches every friend of p recorded by a custodian or an already-reached friend. Verification is local: each friend checks the vouchers against the immutable identity record it received on befriend, with no global custodian registry.

Implementation. The implementation realises the secure social graph specification at the granularity of completed protocols at quiescence (Definition 20), and hence realises the social graph. A run recovering from major faults reaches such a quiescent configuration under eventual delivery, the periodic re-broadcast (Appendix D.14) eventually delivering every checkpoint. The abstract friend set $\widetilde{F}_p$ a configuration is read against (Definition 26) is the set of friendships of p as recorded in p's state.

The following are proven in Appendix H:

Theorem 6 (Secure Quiescent Correspondence). *Each completed transaction protocol of the secure social graph CVA implementation realises, under the mapping $F_p \mapsto \widetilde{F}_p$, the corresponding transaction of the abstract secure social graph (Section 4.2); the correspondence is at the granularity of completed protocols at quiescence, not a step-by-step bisimulation:*
1. A completed Befriend handshake in CVA realises the abstract Befriend(p, q).

2. *A completed Unfriend in CVA realises the abstract Unfriend(p, q).*
3. *A completed Replace cascade in CVA (Vouch, Announce new identity, Integrate new identity, Integrate rebind) realises the abstract Replace(p, p') on every friendship it reaches; on a friendship-preserving run (Definition 5) it reaches every recoverable friend. The one unrecoverable friendship—recorded, at the fault, only by the two friends, the identity of one then lost—is not recovered; the specification does not require its recovery (Definition 5).*
4. *A completed Restore in CVA, after a state loss at p, realises the abstract Recover(p, q): it sets $\widetilde{F}_p$ to the friends that still record p, which at quiescence is the full set of friendships recorded at p before the state loss.*

Corollary 1 (The CVA Implementation Securely Realises the Social Graph). *The secure social graph CVA implementation securely realises the social graph (Section 4.1), composing the Secure Quiescent Correspondence with the implementation of the social graph by the secure social graph (Theorem 3) and transitivity of implementation, on friendship-preserving runs at quiescence.*

Safety. Abstract-state correspondence is guaranteed only at quiescence (Theorem 6), so the simulation is a liveness property. Safety, by contrast, holds at every reachable state, and continues to hold under finitely many failures, which the periodic re-broadcast of checkpoints repairs (Lemma 17): we guarantee soundness at all times—a friendship (p, q) is reported only if both parties at some point wanted to befriend each other (Appendix D.17).

The unrecoverable fault. Recovery is exact except on the one information-theoretically unrecoverable condition (Definition 5; Lemma 18): a major fault erases the last surviving record of a friendship. This has two instances. Under state loss, both friends lose their state before recovery: no friend still records the edge, so nothing remains to recover from (the friendship-preserving exclusion of Section 4.3). Under identity loss, one friend's key is replaced and the edge was disseminated to no other agent, so the Replace cascade never reaches the surviving friend (Appendix D.16). The specification does not require the recovery of an unrecoverable friendship (Definition 5). The person behind the surviving friend may re-establish it out of band.

7 CVA Implementation of Secure Grassroots Coins

The second instance of the recovery framework (Section 3) is grassroots coins [21, 36,39]: each agent is the sovereign of its own currency, mints its own coins—a coin with a future maturity date is a bond—and authorises every transaction in them. Where the social graph's recoverable state is friendships, each co-owned by two friends and recovered lazily, a currency's recoverable state is the sovereign's transaction log: a totally ordered record under a single authoritative writer, in which every recorded payment matters, demanding exact recovery. We summarise the specification and its CVA implementation; the full development, with proofs, is in Appendix E (specification) and Appendix F (implementation).

Specification. Each sovereign p maintains an append-only transaction log L_p recording every transaction in p-coins—Mint, Pay, Redeem, Swap—following Grassroots Flash [21], where the sovereign approves each payment and its personal blockchain is the authoritative ledger. To recover from a fault, p designates a set of its friends as state custodians at the formation of its currency, fixed and immutable thereafter, each holding a copy of L_p. Because the transactions are atomic, every custodian's copy is updated in the same step as the sovereign's, so all agree and a single custodian suffices to recover the log, from which every p-coin holding is derived (Appendix E).

From atomicity to a supermajority. An asynchronous implementation has no such atomicity: a custodian's copy may lag the sovereign's. Recovering from a single lagging custodian could reinstate a log that omits an already-approved payment, and the recovered sovereign might then approve the same coin a second time—an inadvertent double-spend. We instead restore exactness by coupling finality to recoverability through a supermajority: a payment is final, and a payee may rely on it, only once a supermajority of the sovereign's state custodians hold its block.

To facilitate recovery, the sovereign proceeds through monotonically increasing epochs, with each log block carrying the sovereign's epoch at its creation. In addition, each custodian keeps the latest epoch number proposed by the sovereign, initially 0, and the epoch in which its copy of the log was registered. A recovering sovereign has no memory of the epochs its currency has used and learns it from its custodians. It tags every recovery message with a fresh nonce, which the custodians echo, so it never mistakes a reply to its previous incarnation for a reply to itself. Upon recovery, it first queries them for the epoch they have recorded and for their copies of the log, and waits for a supermajority of replies. It then sets its new epoch to exceed the largest epoch it heard, proposes the new epoch number to the custodians, and waits for a supermajority of acks, indicating that a supermajority of custodians have recorded the new epoch; if too many have already recorded a higher epoch, it proposes again above it. Next, the sovereign selects the maximum epoch in which the copies it received are registered, and adopts the longest copy registered in this epoch. Finally, the sovereign sends the adopted log to all custodians and resumes operation; once a supermajority have acknowledged it, the adopted log is final and the sovereign notifies the parties of its blocks.

Custodians, in turn, align their logs with the one adopted by the sovereign, which may involve discarding blocks from earlier epochs. Note that such abandoned blocks never reached a supermajority (and hence were not recovered in an earlier recovery), they are not final. Since a supermajority records the new epoch before it is used to create blocks and any two supermajorities intersect, every future recovery is guaranteed discover this epoch number and initiate a higher one. Thus, epochs in the log increase monotonically, and within the latest epoch, all copies are prefixes of one log.

Recovery by a double supermajority. The bottom line of recovery is this: a payment is final once a supermajority of the sovereign's custodians hold it,

and a recovery is final once a supermajority of custodians have recorded its new epoch and a supermajority have acknowledged the log it adopted. Since any two supermajorities intersect, every recovery reaches a custodian holding each final payment, and every later recovery discovers the new epoch and the log adopted under iost. This is the supermajority-intersection structure of All-to-All Flash [20], here in a simpler crash-only setting with a single log writer, so the quorums are plain intersecting read and write quorums and no consensus is invoked.

A sound sovereign. The coins of a currency are as sound as its sovereign's operating system. We assume it is sound: it may crash and lose its state, but it never deviates from the protocol. Under this assumption the currency is safe; coins whose sovereign's operating system is compromised are outside our scope.

Construction. The protocol realises each spec transaction as a sequence of unary CVA transactions: a holder requests a payment, the sovereign approves it—checking the coins are unspent in its log—appends a block and disseminates it to its custodians, each custodian appends the block in order and acknowledges, and the sovereign binds finality once a supermajority have acknowledged, notifying the payee, who may then rely on the payment. A holding is debited from the payer at request and credited to the payee at finality; the authoritative record is the sovereigns' logs, of which each agent's local holdings are a view.

Recovery. State custodians, like the identity record, are intrinsic: the set S_p and the threshold survive a fault, so a recovering sovereign knows from whom to collect before it has recovered any log. State loss is handled by adopting the longest ledger in the latest epoch, as described above. Under identity loss, the sovereign is first replaced by a fresh agent p' through the social graph (Section 6), which inherits the intrinsic custodian set and then collects from a supermajority exactly as under state loss; the social graph recovers the identity, the currency layer adds only the recovery of its own log. A holder that loses its local account recovers it from each friend sovereign, which answers from its log; no custodian is involved. Recovery presumes that a supermajority of a sovereign's state custodians do not concurrently lose state within a recovery window; with attestation and no persistent storage, this bounds the failures tolerated while a recovery completes.

Recovery restores the ledger, not the value. Identity recovery restores a sovereign's log, not the value of its currency, which rests on the integrity and solvency of its sovereign. A key lost without abuse leaves the currency intact: the recovered sovereign resumes it unchanged. A key held by an attacker who paid or minted fraudulently leaves valid, final transactions in the log—a faithfully recorded debt—to the extent of which the sovereign is insolvent. Finality is not revoked: the recovered log stands, and every payment a payee relied upon remains final. However, the sovereign may mitigate the loss in its own currency: any mitigation a supermajority of its state custodians approve is possible, and it is up to the sovereign to convince them. For example, the sovereign may reissue the currency from a final prefix of the log not containing fraudulent transactions.

The irrecoverable loss is in other currencies. Reissue is a remedy only over a sovereign's own currency. A key compromise does its principal damage elsewhere: the attacker transfers the victim's holdings in other sovereigns' coins to third parties, each payment final at its issuing sovereign, against whom the victim is an ordinary holder. These transfers cannot be undone—recovery restores the person's identity and the ledgers they issue, not wealth alienated in currencies they do not control.

Implementation and recovery results. The implementation realises the secure coins specification at quiescence, and recovers the log exactly across a fault.

Theorem 7 (Correspondence at Quiescence). *At a quiescent configuration (Definition 20) of any run of the CVA protocol in which every custodian has registered the current epoch, the abstract state read off the configuration is a reachable state of the secure coins specification (Appendix E): every custodian copy equals its sovereign's log, every log block is final, and each agent's holdings equal the holdings derived from the logs.*

The correspondence is at quiescence, not at every state, and necessarily so: between a payment's request and its finalisation a coin is debited from the payer and not yet credited to any payee, a configuration the atomic specification never exhibits.

Lemma 3 (Conservation of Money). *At every quiescent configuration of a run with no state-loss fault, the s-coins across all holdings are exactly those recorded as minted in L_s.*

Theorem 8 (Exact Recovery). *If a sovereign recovers—under state loss, or as p' under identity loss—by adopting the longest of the log copies in the maximal epoch collected from a supermajority $G \subseteq S_p$, then the recovered log contains every block of L_p that was final at any time before recovery.*

Corollary 2 (No Inadvertent Double-Spend). *After recovery, the sovereign never approves a payment of a coin already spent in a final payment.*

Liveness. In a correct run, a requested payment whose sovereign and a supermajority of its custodians stay live becomes final.

Theorem 9 (Payment Liveness). *In a correct run in which the sovereign s does not suffer a state loss after receiving a valid pay request and a supermajority of S_s registered in s's current epoch does not suffer a state loss, the requested payment eventually becomes final and its payee receives notification of finality.*

The platform is grassroots, like every CVA protocol (Theorem 5); finality soundness, log validity, and the prefix consistency of custodian copies, together with the proofs of the results above, are in Appendix F.6.

8 Related Work

Volitional multiagent atomic transactions. The formal framework for grassroots platforms used in this paper—agents consisting of people and machines, with machine transactions guarded by people's volitions—was introduced in [22]. That work decomposes each agent's state into a machine state and a volitional state, and introduces equivalence classes of machine transactions and their consumption upon satisfaction; a full survey of the prior formal tradition of modelling people in concurrent systems, and a discussion of how volitional multiagent atomic transactions depart from it, appears there (see also Appendix B). The contribution of the present work is to extend that framework to cover major faults: identity loss—the loss of a person's private key and/or machine—and state loss—the loss of machine state with the key retained.

Key recovery and social recovery of identity. Recovering a cryptographic identity after the loss or compromise of a private key is a long-standing problem. Classical solutions rely on threshold cryptography—Shamir's secret sharing [34] distributes a secret among custodians such that a threshold subset can reconstruct it. A distinct approach, which underlies the present work, does not share the secret at all: instead, the person chooses a fresh keypair off-chain and a designated set of friends collectively authorise the substitution of the new public key for the old one across the person's social context. Shahaf et al. [33] employ such a mechanism for identifier recovery via mutual sureties. Similar designs appear in social-recovery wallets in the cryptocurrency space, where designated guardians authorise a wallet's key change [8]. For example, Maram, Kelkar and Eyal [25] and Mouallem and Eyal [29] decide which of two parties presenting credentials is their true owner, assuming the credentials still exist. In contrast, we consider major faults where the key and device are gone and nothing can be presented, so recovery must be performed by the person's custodians rather than by the person. Here, we give a formal specification of friendship-based major-fault recovery as a guarded multiagent atomic transaction over the social graph, and extend the same approach to the recovery of higher-level state (the sovereign's transaction log in grassroots coins) via state custodians.

Comparison. Deployed systems in the same design space identify a person with a keypair and offer no protocol for recovery from its loss: in Secure Scuttlebutt [42] and Nostr [31] a lost key is a lost identity, and its social connections can be re-established only out of band. Threshold schemes [34] reconstruct the lost secret from shares; social-recovery wallets [8] replace the key but record the change on a blockchain, a global resource. Here the person assumes a fresh key, authorised by custodians, and the rename is carried by the social graph itself, with no global resource.

Acknowledgements

We are grateful to Sadegh Keshavarzi, Gregory Chockler, and Alexey Gotsman for fruitful discussions on trusted execution environments.

References

1. Abramsky, S., Jagadeesan, R., Malacaria, P.: Full abstraction for PCF. Information and Computation **163**(2), 409–470 (2000)
2. Alur, R., Henzinger, T.A., Kupferman, O.: Alternating-time temporal logic. Journal of the ACM **49**(5), 672–713 (2002)
3. Artikis, A., Sergot, M., Pitt, J.: Specifying norm-governed computational societies. ACM Transactions on Computational Logic **10**(1), 1–42 (2009)
4. Back, R.J., von Wright, J.: Refinement Calculus: A Systematic Introduction. Springer (1998)
5. Bolton, M.L., Bass, E.J., Siminiceanu, R.I.: Using formal verification to evaluate human-automation interaction: A review. IEEE Transactions on Systems, Man, and Cybernetics: Systems **43**(3), 488–503 (2013)
6. Briongos, S., Karame, G., Soriente, C., Wilde, A.: No forking way: Detecting cloning attacks on Intel SGX applications. In: 39th Annual Computer Security Applications Conference (ACSAC) (2023), https://www.acsac.org/2023/program/final/s207.html
7. Buterin, V.: Governance, part 2: Plutocracy is still bad (2018), https://vitalik.eth.limo/general/2018/03/28/plutocracy.html, available at https://vitalik.eth.limo/general/2018/03/28/plutocracy.html
8. Buterin, V.: Why we need wide adoption of social recovery wallets. https://vitalik.ca/general/2021/01/11/recovery.html (2021)
9. Coker, G., Guttman, J., Loscocco, P., Herzog, A., Millen, J., O'Hanlon, B., Ramsdell, J., Segall, A., Sheehy, J., Sniffen, B.: Principles of remote attestation. International Journal of Information Security **10**(2), 63–81 (2011). https://doi.org/10.1007/s10207-011-0124-7
10. Ellison, C.: Ceremony design and analysis. Tech. Rep. 2007/399, IACR (2007)
11. Esteva, M., Rodríguez-Aguilar, J.A., Sierra, C., Garcia, P., Arcos, J.L.: On the formal specification of electronic institutions. In: Agent-Mediated Electronic Commerce (AMEC). LNCS, vol. 1991, pp. 126–147. Springer (2001)
12. Ethereum: Decentralized autonomous organizations (DAOs) — ethereum.org (2021), https://ethereum.org/en/dao/
13. Faqir-Rhazoui, Y., Arroyo, J., Hassan, S.: A comparative analysis of the platforms for decentralized autonomous organizations in the ethereum blockchain. Journal of Internet Services and Applications **12**, 1–20 (2021). https://doi.org/10.1186/s13174-021-00139-6
14. Halpern, D., Procaccia, A.D., Shapiro, E., Talmon, N.: Federated assemblies. Proc AAAI 2025; arXiv preprint arXiv:2405.19129 (2024), https://arxiv.org/abs/2405.19129
15. Harel, D., Pnueli, A.: On the development of reactive systems. In: Logics and Models of Concurrent Systems. NATO ASI Series, vol. 13, pp. 477–498. Springer (1985)
16. Hoare, C.A.R.: Communicating Sequential Processes. Prentice-Hall (1985)
17. Keidar, I., Lewis-Pye, A., Shapiro, E.: Constitutional consensus. arXiv preprint arXiv:2505.19216 (2025), https://arxiv.org/abs/2505.19216
18. Keshavarzi, S., Chockler, G., Gotsman, A.: TEE is not a healer: Rollback-resistant reliable storage. In: Proceedings of the 39th International Symposium on Distributed Computing (DISC 2025). LIPIcs, vol. 356, pp. 39:1–39:18 (2025). https://doi.org/10.4230/LIPIcs.DISC.2025.39

19. Kupferman, O., Vardi, M.Y., Wolper, P.: Module checking. Information and Computation **164**(2), 322–344 (2001)
20. Lewis-Pye, A., Naor, O., Shapiro, E.: Flash: An asynchronous payment system with good-case linear communication complexity. arXiv preprint arXiv:2305.03567 (2023), https://arxiv.org/abs/2305.03567
21. Lewis-Pye, A., Naor, O., Shapiro, E.: Grassroots flash: A payment system for grassroots cryptocurrencies. arXiv preprint arXiv:2309.13191 (2023), https://arxiv.org/abs/2309.13191
22. Lewis-Pye, A., Shapiro, E.: Volition-guarded multiagent atomic transactions: Describing people and their machines. arXiv preprint arXiv:2604.25596 (2026), https://arxiv.org/abs/2604.25596
23. Lynch, N., Segala, R., Vaandrager, F.: Hybrid I/O automata. Information and Computation **185**(1), 105–157 (2003)
24. Lynch, N.A., Tuttle, M.R.: An introduction to input/output automata. Tech. Rep. MIT/LCS/TM-373, MIT Laboratory for Computer Science (1989)
25. Maram, D., Kelkar, M., Eyal, I.: Interactive multi-credential authentication. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS). pp. 408–422 (2024)
26. Matetic, S., Ahmed, M., Kostiainen, K., Dhar, A., Sommer, D., Gervais, A., Juels, A., Capkun, S.: ROTE: Rollback protection for trusted execution. In: Proceedings of the 26th USENIX Security Symposium (2017), https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/matetic
27. Milner, R.: A Calculus of Communicating Systems, LNCS, vol. 92. Springer (1980)
28. Milner, R.: The Space and Motion of Communicating Agents. Cambridge University Press (2009)
29. Mouallem, M., Eyal, I.: Asynchronous authentication. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS) (2024)
30. Nakamoto, S., Bitcoin, A.: A peer-to-peer electronic cash system. Bitcoin.–URL: https://bitcoin. org/bitcoin. pdf **4** (2008)
31. Nostr Protocol: NIP-01: Basic protocol flow description. https://github.com/nostr-protocol/nips/blob/master/01.md (2024), accessed 2026
32. Sabt, M., Achemlal, M., Bouabdallah, A.: Trusted execution environment: What it is, and what it is not. In: 2015 IEEE Trustcom/BigDataSE/ISPA. vol. 1, pp. 57–64. IEEE (2015)
33. Shahaf, G., Shapiro, E., Talmon, N.: Genuine personal identifiers and mutual sureties for sybil-resilient community growth. In: International Conference on Social Informatics. pp. 320–332. Springer, Springer, EU (2020). https://doi.org/10.1007/978-3-030-60975-7_24
34. Shamir, A.: How to share a secret. Communications of the ACM **22**(11), 612–613 (1979). https://doi.org/10.1145/359168.359176
35. Shapiro, E.: Multiagent transition systems: Protocol-stack mathematics for distributed computing. arXiv preprint arXiv:2112.13650 (2021), https://arxiv.org/abs/2112.13650
36. Shapiro, E.: Grassroots currencies: A foundation for a grassroots digital economy. arXiv preprint arXiv:2202.05619 (2022), https://arxiv.org/abs/2202.05619
37. Shapiro, E.: Grassroots distributed systems: Concept, examples, implementation and applications (brief announcement). In: 37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391). pp. 47:1, 47:7. LIPICS, Italy (2023), https://arxiv.org/abs/2301.04391

38. Shapiro, E.: Grassroots social networking: Serverless, permissionless protocols for twitter/linkedin/whatsapp. In: OASIS '23. Association for Computing Machinery (2023). https://doi.org/10.1145/3599696.3612898
39. Shapiro, E.: Grassroots bonds as a foundation for market liquidity. arXiv:2603.13671 (2026), https://arxiv.org/abs/2603.13671
40. Shapiro, E.: Grassroots platforms with atomic transactions: Social graphs, cryptocurrencies, and democratic federations. In: Proceedings of the 27th International Conference on Distributed Computing and Networking. pp. 71–81 (2026). https://doi.org/10.1145/3772290.3772309, arXiv preprint arXiv:2502.11299
41. Talmon, N., Shapiro, E.: Grassroots federation: Fair democratic governance at scale. arXiv preprint arXiv:2505.02208; also Proc. of AAMAS'26 (2025), https://arxiv.org/abs/2505.02208
42. Tarr, D., Lavoie, E., Meyer, A., Tschudin, C.: Secure scuttlebutt: An identity-centric protocol for subjective and decentralized applications. In: Proceedings of the 6th ACM conference on information-centric networking. pp. 1–11 (2019). https://doi.org/10.1145/3357150.3357396
43. Turing, A.M.: On computable numbers, with an application to the Entscheidungsproblem. Proceedings of the London Mathematical Society **s2-42**(1), 230–265 (1936)
44. Turing, A.M.: Systems of logic based on ordinals. Proceedings of the London Mathematical Society **s2-45**(1), 161–228 (1939)
45. Wang, S., Yuan, Y., Wang, X., Li, J., Qin, R., Wang, F.Y.: An overview of smart contract: architecture, applications, and future trends. In: 2018 IEEE Intelligent Vehicles Symposium (IV). pp. 108–113. IEEE (2018). https://doi.org/10.1109/ivs.2018.8500488
46. Wang, Z., Jin, H., Dai, W., Choo, K.K.R., Zou, D.: Ethereum smart contract security research: survey and future research opportunities. Frontiers of Computer Science **15**(2), 1–18 (2021)
47. Wood, G., et al.: Ethereum: A secure decentralised generalised transaction ledger. Ethereum project yellow paper **151**(2014), 1–32 (2014), https://ethereum.github.io/yellowpaper/paper.pdf
48. Zuboff, S.: The age of surveillance capitalism: The fight for a human future at the new frontier of power. Public Affairs, US (2019), https://www.hachettebookgroup.com/titles/shoshana-zuboff/the-age-of-surveillance-capitalism/9781610395694/
49. Zuboff, S.: Surveillance capitalism or democracy? the death match of institutional orders and the politics of knowledge in our information civilization. Organization Theory **3**(3), 26317877221129290 (2022). https://doi.org/10.1177/26317877221129290

A The GSG and SGSG Contracts

GSG, the grassroots social graph contract, says how two persons become friends and cease to be friends. It extends GKK, whose clauses give a person an identity, a key pair (GKK:1), and let persons come to know each other (GKK:2, GKK:3). A clause marked ■ is enforced by the platform, one marked ♦ is attested by the persons who act in it, and one marked ∘ is undertaken by a person.

GSG: Grassroots Social Graph

GSG extends GKK.

Alice and Bob are persons. Alice trusts Bob when she knows Bob personally and believes him to keep the undertakings he signs under grassroots contracts.

1. ♦ **Friendship.** Alice and Bob, who trust each other and are not friends, may jointly decide to become friends.
2. ■ **Ending a friendship.** Alice may end her friendship with Bob.
3. ○ **Loss of Trust.** Alice ends her friendship with Bob when she ceases to trust him.

Pragmas

Friends: the friends of Alice.
Friendship: Alice [Befriend]
Bob: Alice asks to be your friend.
Ending a friendship: Alice [Unfriend]

SGSG, the secure grassroots social graph contract, amends GSG with recovery from the two faults a person may suffer: a state loss, in which her machine loses some or all of its state and she keeps her private key, and an identity loss, in which she loses her private key. Each person fixes an identity record when she establishes her identity, and each friend keeps a record of her friends, told at once by friendship updates and again by checkpoints. A state loss is recovered from those records, held by her friends; an identity loss by her identity custodians vouching for her new identity, which each friend accepts against the identity record he holds.

SGSG: Secure Grassroots Social Graph

SGSG amends GSG.

Alice, Bob and Carol are persons. The identity record of Alice is her identity custodians, persons she names, and her threshold, a share of them greater than half. The records of Alice are her friends and, for each friend, the friends of that friend. Alice suffers a state loss when her machine loses some or all of its state and she keeps her private key, and an identity loss when she loses her private key.

1. ■ **Identity record.** Alice establishes her identity (GKK:1) together with her identity record; neither her identity custodians nor her threshold ever change, but for the replacement of a custodian's lost identity (SGSG:5).
2. ■ **Friendship updates.** When Alice and Bob become friends (GSG:1), Alice tells each of her friends of the new friendship and sends Bob her friends and her identity record; when Alice ends a friendship (GSG:2), she tells each of her friends. Each friend of Alice records what she tells.
3. ■ **Checkpoint.** Alice may send each of her friends a checkpoint of her records: her friends and the friends of each. Each friend of Alice keeps it.
4. ■ **State recovery.** After a state loss of Alice, her machine empties all of her records, and she records them again from the friendship updates and the checkpoints her friends send her.
5. ♦ **Identity replacement.** After an identity loss of Alice, Alice establishes a new identity (GKK:1) that has no friends and asks her identity custodians to vouch for it. Carol, an identity custodian of Alice, may vouch, once, that the new identity is Alice's. Bob, a friend of Alice, accepts the new identity in place of the lost one once at least the threshold of the identity custodians he records for Alice have vouched: the new identity becomes his friend in place of the lost one, and his identity custodian if he named Alice; Bob tells the new identity the friends of Alice he records, and tells his friends of the change.

Pragmas

Friends: the friends of Alice.
Identity custodians: the identity custodians of Alice.
Custodian of: the persons who named Alice their identity custodian.
Identity record: Alice [Establish] <identity custodians> <threshold>
Checkpoint: Alice [Checkpoint]
Identity replacement: Alice [Ask custodians] <lost identity>
Carol: Alice asks you to vouch that this is her new identity.

Clauses 1 to 3 of SGSG are what is kept: the identity record, fixed for good, and the records of friends, kept current by friendship updates and checkpoints. Clause 4 is the recovery from a state loss, with no act of anyone but the friends' updates and checkpoints. Clause 5 is the recovery from an identity loss: only the identity custodians vouch, and each friend checks the vouches against the identity record he received when the friendship began.

SGSG in the format of grassroots contracts

Below, SGSG is written as the other grassroots contracts are: each act has a name and says who may take it, and an act written with no 'may' is taken with

nobody deciding. Deleted text is ~~struck through~~ and inserted text is underlined. The changes, each with its reason:

1. SGSG extends GSG and does not amend it. Its clauses add acts and change no act of GKK or of GSG, and no grassroots contract amends another.
2. The persons are declared after the definitions and before the clauses, as in every grassroots contract.
3. Clause 1 is an act of its own, taken once, and not a part of GKK:1: a contract that extends another cannot add to an act of it.
4. In clause 2 the telling and the sending are acts nobody decides, taken when a friendship is entered or ended, as a steward's delivery is. They add nothing to GSG:1 and GSG:2.
5. Clause 4 does not say that the machine empties her records: the state loss has done so.
6. The new identity has a name, Alicia, since an identity is the person who acts; and Bob's acceptance of Alicia is an act nobody decides.

SGSG: Secure Grassroots Social Graph

SGSG ~~amends~~ extends GSG.

~~Alice, Bob and Carol are persons.~~ The identity record of Alice is her identity custodians, persons she names, and her threshold, a share of them greater than half. The records of Alice are her friends and, for each friend, the friends of that friend. Alice suffers a state loss when her machine loses some or all of its state and she keeps her private key, and an identity loss when she loses her private key.

Alice, Bob and Carol are persons, Alicia a new identity of Alice.

1. ■ ~~**Identity record.** Alice establishes her identity (GKK:1) together with her identity record; neither her identity custodians nor her threshold ever change, but for the replacement of a custodian's lost identity (SGSG:5).~~
   Identity record: Alice may fix her identity record, once. It then changes only by the replacement of a custodian's lost identity (SGSG:5).
2. ■ ~~**Friendship updates.** When Alice and Bob become friends (GSG:1), Alice tells each of her friends of the new friendship and sends Bob her friends and her identity record; when Alice ends a friendship (GSG:2), she tells each of her friends. Each friend of Alice records what she tells.~~
   Friendship updates. Tell: Alice tells each of her friends when she becomes a friend of Bob (GSG:1) and when she ends a friendship (GSG:2), and the friend records it. Send record: Alice sends Bob, when they become friends, her friends and her identity record, and Bob keeps them.
3. ■ ~~**Checkpoint.** Alice may send each of her friends a checkpoint of her records: her friends and the friends of each. Each friend of Alice keeps it.~~
   Checkpoint: Alice may send each of her friends a checkpoint of her records, and the friend keeps it.

SGSG, continued

4. ■ ~~**State recovery.** After a state loss of Alice, her machine empties all of her records, and she records them again from the friendship updates and the checkpoints her friends send her.~~
   State recovery. Restore: After a state loss of Alice, each friend of Alice sends her the friendship updates and the checkpoints he keeps, and Alice records them again.
5. ♦ ~~**Identity replacement.** After an identity loss of Alice, Alice establishes a new identity (GKK:1) that has no friends and asks her identity custodians to vouch for it. Carol, an identity custodian of Alice, may vouch, once, that the new identity is Alice's. Bob, a friend of Alice, accepts the new identity in place of the lost one once at least the threshold of the identity custodians he records for Alice have vouched: the new identity becomes his friend in place of the lost one, and his identity custodian if he named Alice; Bob tells the new identity the friends of Alice he records, and tells his friends of the change.~~
   Identity replacement. Replace: After an identity loss, Alice may establish a new identity, Alicia (GKK:1), and ask her identity custodians to vouch for it. Vouch: Carol, an identity custodian of Alice, may vouch, once, that Alicia is Alice. Accept: Bob, a friend of Alice, accepts Alicia in place of Alice once at least the threshold of the identity custodians he records for Alice have vouched; Alicia is then his friend in place of Alice, and his identity custodian if Alice was. Bob tells Alicia the friends of Alice he records, and tells his friends of the change.

Pragmas

Friends: the friends of Alice.
Identity custodians: the identity custodians of Alice.
Custodian of: the persons who named Alice their identity custodian.
Identity record: Alice ~~[Establish]~~ [Fix] <identity custodians> <threshold>
Checkpoint: Alice [Checkpoint]
Identity replacement: ~~Alice [Ask custodians]~~ Alicia [Replace] <lost identity>
Carol: ~~Alice asks you to vouch that this is her new identity.~~ Alicia asks you to vouch that she is Alice.

B Formal Models of Persons in Concurrent Systems

The formal methods tradition has a long lineage of modeling human agents as sources of nondeterminism alongside deterministic machines, but—to the best of our knowledge—without decomposing an agent into person and machine as components of its state.

Turing's choice machines. Before defining what we now call Turing machines (automatic machines, or a-machines), Turing [43] introduced choice machines (c-machines), "whose motion is only partially determined by the configuration"—at designated states, the machine "cannot go on until some arbitrary choice has been made by an external operator." The external operator is a person who freely chooses between alternatives; the sequence of choices determines which computation unfolds. Turing immediately set c-machines aside, showing that any c-machine computation can be enumerated by an a-machine. His 1939 oracle machines [44] extend this further: the oracle "cannot be a machine" and provides

answers the computation cannot derive internally. Both formalisms model the person as external to the machine, providing input at designated points.

Process algebras. Hoare's CSP [16] provides the cleanest process-algebraic encoding of human choice. External choice ($\Box$) offers the environment—potentially a person—a selection among initial events, while internal choice ($\sqcap$) is resolved by the system. However, the person remains outside the system boundary: CSP models what the person does, not what the person is willing to do. Milner's CCS [27] uses a single summation operator without formally separating internal from external nondeterminism at the syntactic level. In his later work on bigraphs [28], Milner makes the scope explicit: agents "can be artificial, as in computing systems... or they can be natural, e.g. communicating humans." Both CSP and CCS model agents uniformly—there is no formal distinction between a person and a machine within the same agent.

I/O automata and reactive systems. Lynch and Tuttle's I/O automata [24] partition actions into input (environment-controlled), output (automaton-controlled), and internal actions. The key property is input-enabling: an automaton cannot block input actions, so the environment—potentially a human operator—can act at any moment. The Hybrid I/O Automata extension [23] explicitly states that HIOAs are "intended to model all components of hybrid systems, including... humans." The person, however, is part of the environment, not a component of the automaton's state. Harel and Pnueli's reactive systems paradigm [15] draws the foundational dichotomy between transformational systems (batch, terminating) and reactive systems (ongoing interaction with environment). The system is deterministic; all nondeterminism is attributed to the environment. This paradigm was explicitly motivated by human-machine interaction, yet the formalism treats the human as environment rather than as a component of the system.

Angelic and demonic nondeterminism. The distinction between angelic and demonic nondeterminism, developed by Back and von Wright [4] in the refinement calculus, provides a semantic treatment relevant to the person/machine boundary. Demonic nondeterminism models adversarial environments (the worst-case choice is made); angelic nondeterminism models cooperative choices (the best-case choice is made). This duality is the closest precursor to the volition/obligation distinction in volitional multiagent atomic transactions [22]: a volitional transaction guarded by both parties (both must be willing) versus one guarded by either party (either can force it). Two distinctions separate the frameworks. First, the refinement calculus operates within a sequential program framework, not a multiagent transition system. Second, angelic nondeterminism is a point-of-choice semantics: a choice is resolved locally at each transition, with no residue carried forward. Volitions, in contrast, are persistent, inspectable state that accumulates across transitions; a guard condition reads an agent's record of willing over the history of the run, not a single local resolution. This shift—from choice as a point-semantic primitive to choice as state—is what lets volitions be shared, compared, and reasoned about within the transition system, rather than external to it.

Game structures and alternating-time temporal logic. Module checking [19] models open systems with the environment fully adversarial. Alternating-time temporal logic (ATL) [2] interprets formulas over concurrent game structures where multiple agents simultaneously choose actions. Game semantics [1] models computation as dialogue between Proponent (program, following a deterministic strategy) and Opponent (environment, making free moves). These frameworks treat agents as symmetric players but do not decompose a single agent into person and machine components.

Ceremony analysis and human-interactive verification. Ellison's ceremony analysis [10] extends security protocol analysis to include human participants as protocol nodes. Bolton's Enhanced Operator Function Model (EOFM) [5] translates hierarchical human task models into state machines for model checking, with the human as the sole source of nondeterminism. Both treat human nondeterminism as a source of error to be verified against, rather than as a source of legitimate volition to be formally recorded.

Normative multiagent systems and electronic institutions. Electronic institutions [11] model multiagent interaction as dialogical frameworks where human and software agents are treated uniformly as role-playing entities. Normative multiagent systems [3] use Event Calculus to specify societies where agents "may fail to, or even choose not to, conform to the specifications." These approaches model norms that constrain agents, but do not decompose an agent's state into machine and volitional components, nor do they formalise the distinction between transactions requiring all parties to be willing and those that are obligatory once initiated.

C Guarded Multiagent Atomic Transactions

This section recalls essential definitions of the theory of volitional multiagent atomic transactions [22], which provides a formal foundation for specifying grassroots platforms as systems of people operating machines. The full mathematical development—transition systems, runs, liveness, closure, grassroots protocols, and proofs—appears in [22]; here we recall only what is needed for the specifications that follow.

Each agent consists of a person and a machine (e.g., smartphone) operated by the person. A machine transaction specifies what machines do; a guard specifies which persons must be willing for the transaction to occur.

We assume a potentially infinite set of agents Π, considering only finite subsets $P \subset \Pi$. We use $\subset$ for the strict subset relation and $\subseteq$ when equality is also possible. We use S^P to denote the set of all total functions from P to S, and c_p for the member of $c \in S^P$ indexed by p.

Machine states, configurations, machine transactions, and guarded transactions are as in Definition 1 (Section 2), which also recalls the reading of guards and of the unguarded case.

Notation for transaction specifications. Protocol specifications in the sections that follow describe a machine transaction $c \to c'$ (Definition 1) by naming

its participants Q, giving a precondition on c introduced by "provided," and specifying the updates that produce c' from c. Where the machine state has a short name we use primed notation: "$F'_p := F_p \cup \{q\}$" means $c'_p = c_p \cup \{q\}$, reading F_p as c_p and F'_p as c'_p. Where the machine state is structured, as in CVA (Section 5), we use a procedural update: "add $\mathsf{message}(\ldots)$ to o_p," "set $a_p.\mathrm{FMap}[q] := \ldots$," each understood as modifying only the component named. In both styles, components outside the participants Q and components of participants' states not mentioned are unchanged.

Distinct machine transactions can represent "the same action" in different configurations, captured by an equivalence relation on machine transactions.

Definition 7 (Transaction Equivalence). *Given a set of machine transactions R, a **transaction equivalence** is an equivalence relation $\sim$ on R such that $t \sim t'$ implies t and t' have the same participants. We write $[t]$ for the equivalence class of t under $\sim$.*

Definition 8 (Agent State and Configuration). *Given agents P, states S with initial state $s0$, a set of machine transactions T each over its own participants $Q \subseteq P$ and S, and equivalence $\sim$ on T, an **agent state** is a pair $(V, m) \in \mathcal{A} = (2^{T/\sim} \times S)$ where V is its **volitional state** and $m \in S$ its **machine state**. The **initial agent state** is $(\emptyset, s0)$. An **agent configuration** c over P, S, T, and $\sim$ is a member $c \in \mathcal{A}^P$, in which case we write c^v_p for the volitional state and c^m_p for the machine state of agent p in c.*

Definition 9 (Enabled). *A guarded transaction (t, Q') with $t = d \to d'$ over participants Q is **enabled** in an agent configuration c if $c^m_p = d_p$ for every participant $p \in Q$ (the machine precondition) and $[t] \in c^v_q$ for every guard $q \in Q'$ (the volitional condition). An equivalence class is **enabled** when some representative is. A transaction with empty guard ($Q' = \emptyset$) is enabled whenever its machine precondition holds.*

Definition 10 (Volitional Multiagent Atomic Transaction). *Given agents P, states S, machine transactions T over P and S, and equivalence $\sim$ on T:*

1. *A **change-volition transaction of agent** $p \in P$ is a pair $c \to c'$ of agent configurations over $\{p\}$, S, T, and $\sim$ such that $c^v_p \neq {c'}^v_p \subseteq T/\sim$ and $c^m_p = {c'}^m_p$.*
2. *A **volitional machine transaction** induced by a guarded machine transaction (t, Q'), for some $t = (d \to d') \in T$ over $Q \subseteq P$ and $Q' \subseteq Q$, is a pair $c \to c'$ where $c \neq c'$ are agent configurations over P, S, T, and $\sim$ such that (t, Q') is enabled in c (Definition 9); ${c'}^m_p = d'_p$ for every $p \in Q$; $c^m_p = {c'}^m_p$ for every $p \in P \setminus Q$; and ${c'}^v_p = c^v_p \setminus \{[t]\}$ for every $p \in P$.*
3. *A **volitional multiagent atomic transaction** is a change-volition transaction or a volitional machine transaction.*

When a volitional machine transaction induced by (t, Q') is taken, the class $[t]$ is removed from every agent's volitional state. Volitions are thus discharged upon satisfaction—a person wills a class of transactions, and once any transaction in the class is taken, the will is fulfilled and the class is removed. A person

may independently change their volitional state via change-volition transactions, which may add or remove classes; beyond these, the framework removes a class from c_p^v only via its satisfaction.

Protocols, correctness, and the grassroots property. A **protocol** $\mathcal{F}$ is a family of multiagent transition systems, one for each finite $P \subset \Pi$, over a common local-states function assigning to each P a set of local states containing the initial state $s0$ [22]. A run of $\mathcal{F}(P)$ is **safe** if every two consecutive configurations form a transition of $\mathcal{F}(P)$, and **live** if no equivalence class is enabled (Definition 9) throughout some suffix with no member of the class taken in that suffix; it is **correct** if it is safe and live. In the general theory [22] the relation on transitions is a partial equivalence, leaving change-volition transactions (Definition 10) outside any class and hence under no liveness obligation; here every machine-transaction class is total and the obligation falls on these classes alone. Unguarded transactions carry the liveness obligation with no volitional prerequisite: once the machine precondition holds, some class representative must eventually be taken.

Runs of two disjoint groups can be interleaved into a run of their union. $\mathcal{F}$ is **oblivious** if every interleaving of two correct runs is a correct run of the union, and **interactive** if some correct run of the union is not an interleaving of two independent runs, because it includes a step whose participants span both groups; it is **grassroots** if both [22]. A set of guarded transactions induces a **transactions-based protocol**, and we use two obliviousness results from [22]. General criterion: such a protocol is oblivious if no cross-group transaction—one whose participants span both groups—is ever enabled in an interleaving of two correct runs. **Guarded Obliviousness**: a guarded cross-group transaction is never so enabled, since the guarding person, acting in one group, never wills it; guarding every cross-group transaction therefore suffices.

Implementations. We recall when one transition system implements another [22]. The notions below refer to runs and to stutter-removal, not to the equivalence $\sim$ directly (which enters only through "correct run"), so we write a transition system as the triple (C, c_0, T).

Definition 11 (Implementation). *Given transition systems $TS = (C, c_0, T)$, the **specification**, and $TS' = (C', c_0', T')$, an **implementation of** TS **by** TS' is a function $\sigma : C' \to C$ with $\sigma(c_0') = c_0$. For a computation $r' = c_1' \to c_2' \to \cdots$ of TS', $\sigma(r')$ is the computation $\sigma(c_1') \to \sigma(c_2') \to \cdots$ with every **stutter transition** $c \to c$ removed. The mapping need not preserve transitions: an implementation transition may map to a stutter, and several to a single specification transition.*

Definition 12 (Correct and Complete Implementation). *An implementation (TS', σ) of TS is **correct** if σ maps every correct run of TS' to a correct run of TS, and **complete** if every correct run of TS is $\sigma(r')$ for some correct run r' of TS'.*

Lemma 4 (Transitivity of Implementation). *If (TS', σ) is an implementation of TS and (TS'', τ) an implementation of TS', then $(TS'', \sigma \circ \tau)$ is an*

implementation of TS; it is correct if both are correct, and complete if both are complete.

Proof. $\sigma \circ \tau(c_0'') = \sigma(c_0') = c_0$, so it is an implementation. If both are correct, τ maps a correct run of TS'' to a correct run of TS', which σ maps to a correct run of TS, and stutter-removal composes, so $\sigma \circ \tau$ is correct. If both are complete, a correct run of TS is $\sigma(r')$ for a correct run r' of TS', itself $\tau(r'')$ for a correct run r'' of TS'', so $(\sigma \circ \tau)(r'')$ is that run of TS; thus $\sigma \circ \tau$ is complete.

Definition 13 (Safety Fault, Fault-Resilient Implementation). *Let (TS', σ) be an implementation of TS, with $TS' = (C', c_0', T')$. A **safety fault** is a set $F \subseteq C'^2 \setminus T'$ of **faulty transitions**, and a computation **performs** F if it includes a transition in F. The implementation is **F-resilient** if σ maps every live run $r' \subseteq T' \cup F$ of TS' to a correct run of TS.*

Faulty transitions belong to no transaction class, so they impose no liveness requirement; with $F = \emptyset$ the live runs $r' \subseteq T'$ are exactly the correct runs, so F-resilience extends correctness (Definition 12) to runs that perform and recover from faults in F.

Communicating volitional agents: formal definitions. We recall the formal definitions of communicating volitional agents (Section 5) [22].

Definition 14 (CVA Local States and Configurations). *Given a set C of **cargoes** and a **platform state space** A with initial state $a_0 \in A$, the set of **messages** over $P \subset \Pi$ is*

$$M(P) \;:=\; \{\mathsf{message}(s, r, c) \mid s \neq r \in P,\; c \in C\},$$

*with $\mathsf{message}(s, r, c)$ a message of **sender** s, **recipient** r, and **cargo** c. A **CVA local state** over P is a tuple*

$$(\mathit{known},\, o,\, i,\, a) \;\in\; S(P) \;:=\; 2^P \times 2^{M(P)} \times 2^{M(P)} \times A$$

*where $\mathit{known} \subseteq P$ is a set of **known peers**, $o \subseteq M(P)$ is an **outbox**, $i \subseteq M(P)$ is an **inbox**, and $a \in A$ is a **platform state**; the **initial local state** is $(\emptyset, \emptyset, \emptyset, a_0)$. A **CVA configuration** over P is a member $c \in S(P)^P$; for $p \in P$ we write $c_p = (\mathit{known}_p, o_p, i_p, a_p)$ for the local state of p in c. The **initial configuration** over P, denoted $c0(P)$, assigns the initial local state to every $p \in P$.*

The local-states function S satisfies $P \subseteq P' \implies S(P) \subseteq S(P')$, as required of a transactions-based protocol. A CVA protocol consists of two "built in" binary transactions—discover and communicate—and platform-specific unary transactions:

Definition 15 (CVA Protocol). *A CVA protocol consists of every binary transaction $c \to c'$ over $\{p, q\}$ for every $p \neq q \in \Pi$ such that $c' = c$ except that:*

1. ***Discover**: guarded by $\{p\}$, $known'_p := known_p \cup \{q\}$.*
2. ***Communicate**: unguarded, $i'_q := i_q \cup M$, $o'_p := o_p \setminus M$, provided $M = \{\mathsf{message}(p, q, c)\} \subseteq o_p$.*

In addition it has for each $p \in P$ guarded unary platform transactions:

3. ***Platform transactions**: unary transactions $c \to c'$ with participant $\{p\}$ and guard $\emptyset$ or $\{p\}$, with a precondition over the local state of p—it may inspect $known_p$, i_p, and a_p—each modifying a_p to a'_p and/or adding one or more $\mathsf{message}(s, r, c)$ to o_p resulting in o'_p, provided $s = p$, $r \in known_p$ and $c \in C$. A platform transaction does not remove messages from i_p.*

Inboxes and outboxes are sets (Definition 14). A reactive platform transaction reads its inbox in its precondition but does not consume from it; each such transaction is idempotent by its own precondition, which a monotone guard (an epoch, a frontier, or a domain-membership test) renders unsatisfiable once the transaction has fired, so the same retained message cannot enable it again. Inboxes are therefore monotone non-decreasing, and bounding them is an implementation concern below the CVA abstraction.

The following are proven in Appendix H:

Lemma 5 (Known-Peers Containment). *In any run of $\mathcal{F}(P)$, for every configuration c along the run and every $p \in P$: $known_p \subseteq P$.*

Lemma 6 (Outbox Containment). *In any run of $\mathcal{F}(P)$, for every configuration c along the run, every $p \in P$, and every $\mathsf{message}(s, r, \cdot) \in o_p$: $r \in P$.*

D CVA Implementation of the Secure Social Graph (Full Development)

We present the grassroots social graph as a CVA platform (Section 5). The Befriend and Unfriend transactions of the abstract spec (Section 4.1) are realised as unary platform transactions. The realisation uses an epoch encoding, an offer-/accept handshake, and stream dissemination, established directly over CVA's message discipline.

Communication uses a single primitive: a CVA message (*sender*, *recipient*, *cargo*). A broadcast to p's friends is realised as one such message per recipient in p's outbox. Every recipient is a friend in FMap_p (added on every befriend and never removed), so the recipient set is exactly $\mathrm{dom}(\mathrm{FMap}_p)$ and needs no separate state.

D.1 Platform State

The epoch of a friendship is an integer in $\mathbb{N}$ that encodes both a counter and a status by parity: an odd epoch denotes an active friendship, an even epoch (including 0) denotes an inactive one. We treat 0 as the distinguished value $\perp$

denoting "no connection has ever been made." The Friendship Order on $\mathbb{N}$ is the natural ordering, corresponding to the friendship lifecycle

$$\bot = 0 \ < \ 1_{active} \ < \ 2_{inactive} \ < \ 3_{active} \ < \ 4_{inactive} \ < \ \cdots$$

The platform state of $p \in P$ is a pair

$$a_p \ = \ (\mathrm{FMap}_p, \ \mathrm{FoFMap}_p),$$

where
- $\mathrm{FMap}_p : \Pi \rightharpoonup \mathbb{N} \times D$ is a partial map, each entry $\mathrm{FMap}_p[q] = (e, d)$ recording the epoch e of the friendship between p and q together with an opaque application-data field $d \in D$ (D a platform parameter with default $\bot \in D$; Section D.6);
- $\mathrm{FoFMap}_p : \Pi \rightharpoonup (\Pi \rightharpoonup \mathbb{N})$ is a partial nested map, each entry $\mathrm{FoFMap}_p[q][f] = e$ recording p's observed epoch of the friendship between q and f.

The initial platform state is $(\emptyset, \emptyset)$.

We write $\mathrm{epoch}_p(q) := \mathrm{FMap}_p[q].\mathit{epoch}$ when $q \in \mathrm{dom}(\mathrm{FMap}_p)$ and $\mathrm{epoch}_p(q) := 0$ otherwise. Similarly $\mathrm{epoch}_p(q, f) := \mathrm{FoFMap}_p[q][f]$ when defined and 0 otherwise. The recipient set of p is $\mathrm{Rec}_p := \mathrm{dom}(\mathrm{FMap}_p)$.

D.2 Messages

The cargo space C includes the following tagged messages, for $x \in \mathbb{N}$ and $f \in \Pi$:
- $\mathsf{friend_request}(x)$: friendship offer at epoch x (odd).
- $\mathsf{accept}(x)$: acceptance of a pending $\mathsf{friend_request}(x)$.
- $\mathsf{unfriend}(x)$: unilateral unfriend at epoch x (even).
- $\mathsf{stream_update}(f, x)$: stream update about third party f at epoch x.

D.3 Befriend

Befriending between p and q proceeds in three steps: p sends a $\mathsf{friend_request}$ at a fresh odd epoch, q responds with accept, and p integrates the acceptance. A simultaneous offer from q at the same epoch is resolved by name order on Π: the agent with the larger name accepts the incoming request, the agent with the smaller name ignores it.

Fix a strict total order $<$ on Π.

Definition 16 (Befriend Platform Transactions).

Offer friendship. *A unary transaction at p, guarded by $\{p\}$, for $q \in \mathit{known}_p$ such that $\mathrm{epoch}_p(q)$ is even:*
- *Let $x := \mathrm{epoch}_p(q) + 1$.*
- *Add $\mathsf{message}(p, q, \mathsf{friend_request}(x))$ to o_p.*

Accept offer. *A unary transaction at q, guarded by $\{q\}$, provided $\mathsf{message}(p, q, \mathsf{friend_request}(x)) \in i_q$, $\mathrm{epoch}_q(p)$ is even and $\mathrm{epoch}_q(p) < x$, and no $\mathsf{message}(q, p, \mathsf{friend_request}(x)) \in o_q$:*

– *Set* $\mathrm{FMap}_q[p] := x$.
– *Add* $\mathsf{message}(q, p, \mathsf{accept}(x))$ *to* o_q.
– *For each* $r \in \mathrm{Rec}_q$ *(after the update), add* $\mathsf{message}(q, r, \mathsf{stream_update}(p, x))$ *to* o_q.
– *For each* $f \in \mathrm{Rec}_q$ *with* $f \neq p$ *(after the update), add* $\mathsf{message}(q, p, \mathsf{stream_update}(f, \mathrm{epoch}_q(f)))$ *to* o_q *(the on-join snapshot).*

Resolve simultaneous offer. *A unary transaction at* q*, unguarded, provided* $\mathsf{message}(p, q, \mathsf{friend_request}(x)) \in i_q$, $\mathsf{message}(q, p, \mathsf{friend_request}(x)) \in o_q$, $\mathrm{epoch}_q(p)$ *is even and* $\mathrm{epoch}_q(p) < x$*, and* $p < q$*:*
– *Set* $\mathrm{FMap}_q[p] := x$.
– *Add* $\mathsf{message}(q, p, \mathsf{accept}(x))$ *to* o_q.
– *For each* $r \in \mathrm{Rec}_q$ *(after the update), add* $\mathsf{message}(q, r, \mathsf{stream_update}(p, x))$ *to* o_q.
– *For each* $f \in \mathrm{Rec}_q$ *with* $f \neq p$ *(after the update), add* $\mathsf{message}(q, p, \mathsf{stream_update}(f, \mathrm{epoch}_q(f)))$ *to* o_q *(the on-join snapshot).*

Integrate accept. *A unary transaction at* p*, unguarded, provided* $\mathsf{message}(q, p, \mathsf{accept}(x)) \in i_p$, $\mathrm{epoch}_p(q)$ *is even and* $\mathrm{epoch}_p(q) < x$*:*
– *Set* $\mathrm{FMap}_p[q] := x$.
– *For each* $r \in \mathrm{Rec}_p$ *(after the update), add* $\mathsf{message}(p, r, \mathsf{stream_update}(q, x))$ *to* o_p.
– *For each* $f \in \mathrm{Rec}_p$ *with* $f \neq q$ *(after the update), add* $\mathsf{message}(p, q, \mathsf{stream_update}(f, \mathrm{epoch}_p(f)))$ *to* o_p *(the on-join snapshot).*

Offer friendship and Accept offer are volitional, guarded by the willing agent's person. Resolve simultaneous offer fires only at the agent with the larger name; the symmetric scenario with $q < p$ is handled by p's own instance of this transaction firing on q's incoming request. Integrate accept is reactive at the offerer: once the offer has been willed, consuming the acceptance requires no further volition. In each of the three activating transactions, the activating agent both broadcasts the new edge to its existing friends (the $\mathsf{stream_update}(\cdot, x)$ to each $r \in \mathrm{Rec}$) and sends the new friend a snapshot of its existing friendships (the $\mathsf{stream_update}(f, \mathrm{epoch}(f))$ to the new friend); the snapshot is discussed in Section D.5.

D.4 Unfriend

Unfriending is unilateral: either party may initiate. The initiator increments its FMap entry from an odd epoch to the next even epoch, sends $\mathsf{unfriend}$ to the counterpart, and emits a stream update. The counterpart integrates on receipt, discarding messages from non-friends.

Definition 17 (Unfriend Platform Transactions).

End friendship. *A unary transaction at* p*, guarded by* $\{p\}$*, for* $q \in \mathrm{Rec}_p$ *with* $\mathrm{epoch}_p(q)$ *odd:*
– $\mathrm{FMap}_p[q] := \mathrm{epoch}_p(q) + 1$. *Let* $x := \mathrm{FMap}_p[q]$ *(even).*

– *Add* $\mathsf{message}(p, q, \mathsf{unfriend}(x))$ *to* o_p.
– *For each* $r \in \mathrm{Rec}_p$, *add* $\mathsf{message}(p, r, \mathsf{stream_update}(q, x))$ *to* o_p.

Integrate unfriend. *A unary transaction at* q, *unguarded, provided* $\mathsf{message}(p, q, \mathsf{unfriend}(x)) \in i_q$, $p \in \mathrm{dom}(\mathrm{FMap}_q)$, *and* $\mathrm{epoch}_q(p) < x$*:*
– *Set* $\mathrm{FMap}_q[p] := x$.
– *For each* $r \in \mathrm{Rec}_q$, *add* $\mathsf{message}(q, r, \mathsf{stream_update}(p, x))$ *to* o_q.

The precondition $p \in \mathrm{dom}(\mathrm{FMap}_q)$ on Integrate unfriend ensures that a spurious unfriend from a non-friend has no effect. The precondition $\mathrm{epoch}_q(p) < x$ rejects stale and duplicate unfriend messages.

D.5 Stream Dissemination

Every Befriend and Unfriend transaction that updates FMap_p emits a stream_update to each $r \in \mathrm{Rec}_p$, as specified in the effects above. A recipient integrates the update subject to the Friendship Order.

In addition, when a friendship with a new friend is activated — by Accept offer, Resolve simultaneous offer, or Integrate accept — the activating agent sends that new friend a one-shot snapshot of its current friendships, one $\mathsf{stream_update}(f, \mathrm{epoch}(f))$ per existing friend f, as specified in the effects above. The snapshot catches the new friend up on friendships formed before it joined, so that an observer view never depends on the order in which friendships were established; without it, a friend that joined after a third-party edge was formed would never learn that edge, as the edge emits no further updates unless it changes again.

Definition 18 (Integrate stream update). *A unary transaction at* r, *unguarded, provided* $\mathsf{message}(q, r, \mathsf{stream_update}(f, x)) \in i_r$, $q \in \mathrm{dom}(\mathrm{FMap}_r)$, *and* $\mathrm{epoch}_r(q, f) < x$*:*
– $\mathrm{FoFMap}_r[q][f] := x$.
– $\mathrm{FoFMap}_r[f][q] := x$.

The second assignment preserves FoFMap Symmetry (Invariant 10): r's observed epoch of (q, f) and of (f, q) are equal at every reachable state. The Friendship Order guard ensures monotone integration: no stream update can drive an observed epoch backward. Per-edge updates and snapshot entries are integrated by the same transaction, so the snapshot adds no new integration rule.

D.6 Application Data

The application-data field is an optional extension; the social graph operates unchanged without it, and a platform needing no per-friend data omits D. Each direct-friend record carries an opaque application-data field $d \in D$, where D is a platform parameter with default $\perp \in D$. The social graph transports and stores d but never inspects it, disseminates it, or lets it influence epoch or status. The Befriend transactions of Section D.3, written "Set $\mathrm{FMap}_p[q] := x$", set the

epoch component to x and the data component to $\perp$; the Unfriend transactions of Section D.4 and every other transaction modify only the epoch and preserve the data component. Two purely-local operations expose d to the layered application:

Definition 19 (Application-Data Operations).

Get data. *A read-only query at p, for $q \in \mathrm{dom}(\mathrm{FMap}_p)$: return $\mathrm{FMap}_p[q].data$ to the application; no state change, no message.*

Set data. *A unary transaction at p, guarded by $\{p\}$, for $q \in \mathrm{dom}(\mathrm{FMap}_p)$ and a value $d \in D$ supplied by the application: set $\mathrm{FMap}_p[q].data := d$.*

Neither generates a protocol message nor affects epoch or status. The semantics of D are defined by the layered application and are outside the scope of this paper.

D.7 Properties

We state the CVA-level invariants of the social graph. The invariants below are stated over CVA's transactions and message discipline; their proofs are in Appendix H. These are the properties of the fault-free base graph; the security layer (Appendix D.17) inherits the invariants and adds the recovery results, re-establishing only what faults affect.

Definition 20 (Quiescence). *A configuration is quiescent if (1) no volitional transaction is enabled and (2) there are no new (yet unprocessed) messages in inboxes and outboxes.*

Note that periodic rebroadcasts of checkpoints continue to occur in quiescent states, but they repeatedly send the same message while new messages are sent.

Importantly, at a quiescent configuration, every message that has been sent has been delivered and processed at least once and no agent holds a willed transaction class that has not been consumed. The abstract social graph (Section 4.1), whose transactions are instantaneous, is quiescent exactly when no agent holds an unconsumed volition; its quiescent friend-set assignments are, by Friendship Mutuality (Lemma 1), precisely the mutual ones, and every mutual assignment over P is reached by a run that befriends the corresponding pairs.

Invariants

Lemma 7 (Knowledge Monotonicity). *In any run of the social graph CVA implementation, once $q \in \mathrm{dom}(\mathrm{FMap}_p)$ or $f \in \mathrm{dom}(\mathrm{FoFMap}_p[q])$ becomes true, it remains true at all later states.*

Lemma 8 (Friendship Monotonicity). *In any run of the social graph CVA implementation, $\mathrm{epoch}_p(q)$ and $\mathrm{epoch}_p(q, f)$ are monotonically non-decreasing in the natural order on $\mathbb{N}$.*

Lemma 9 (Message Bounds). *For every message that has been sent but not yet fully processed by its recipient:*
1. if friend_request(x) *is in transit from B to A, then* $\mathrm{epoch}_B(A) \geq x - 1$*;*
2. if accept(x) *is in transit from B to A, then* $\mathrm{epoch}_B(A) \geq x$*;*
3. if unfriend(x) *is in transit from B to A, then* $\mathrm{epoch}_B(A) \geq x$*;*
4. if stream_update(f, x) *is in transit from B, then* $\mathrm{epoch}_B(f) \geq x$*.*

Lemma 10 (Observer Bound). *For any observer A and any pair (p, q), at all times* $\mathrm{epoch}_A(p, q) \leq \max(\mathrm{epoch}_p(q), \mathrm{epoch}_q(p))$*.*

Invariant 10 (FoFMap Symmetry) *For every $p, q, f \in \Pi$ at every reachable state of the social graph CVA implementation,* $\mathrm{epoch}_p(q, f) = \mathrm{epoch}_p(f, q)$*.*

The following lemma establishes robustness to message reordering and duplication.

Lemma 11 (Monotone Integration). *Let m be a message from p to q concerning the friendship between p and r (possibly $r = q$) with epoch $m.e$, and let e be the epoch q holds for that friendship. If $m.e \leq e$, m is not consumed and does not change q's state.*

Definition 21 (Active Friend Set). *The active friend set of p is* $\widetilde{F}_p := \{q \in \mathrm{dom}(\mathrm{FMap}_p) : \mathrm{epoch}_p(q)$ *is odd*$\}$*.*

Safety Properties We state the intent-based safety guarantees of the social graph CVA implementation, which hold at every reachable state—a configuration on some run from the initial configuration—not only at quiescence. They are phrased in terms of an agent wanting to befriend another, the CVA counterpart of the volitional act of offering or accepting friendship.

We denote by Person_p the person behind agent p.

Definition 22 (Wanting to Befriend). *An agent p wants to befriend an agent q at time t if there is a time $t' \leq t$ at which Person_p wills either Offer friendship(q') for an agent q' s.t. $\mathrm{Person}_q = \mathrm{Person}_{q'}$, or the acceptance of a friend request from such an agent q' (the volition guarding Accept offer or Resolve simultaneous offer with q'), and p does not will End friendship(q') for an agent q' of Person_q at any time in $(t', t]$.*

Note that a major fault changes an agent's identity but not the person behind it, so wanting is a relation between the two persons rather than between the identities they currently hold.

Lemma 12 (Post-Accept State). *Suppose p sent* accept(x) *to q at time t_a (via Accept offer or Resolve simultaneous offer), and neither p nor q willed End friendship on the other during $(t_a, t]$, for some $t \geq t_a$. Then* $\mathrm{epoch}_q(p) \in \{x - 1, x\}$ *at t.*

Lemma 13 (Post-Unfriend State). *Suppose* $\mathrm{epoch}_p(q) = \mathrm{epoch}_q(p) = x$ *with* x *odd at time* t_0*, at least one of* p, q *willed End friendship on the other at some* $t_1 \geq t_0$*, and neither willed Offer friendship on the other during* $[t_0, t]$*. Then* $\mathrm{epoch}_p(q), \mathrm{epoch}_q(p) \in \{x, x+1\}$ *for all* $t \geq t_1$*.*

Lemma 14 (Observed-Epoch Provenance). *If* $\mathrm{epoch}_p(q, f) = x \geq 1$ *at time* t_1*, then there is a time* $t_2 \leq t_1$ *at which* $\mathrm{epoch}_q(f) = x$ *or* $\mathrm{epoch}_f(q) = x$*.*

Theorem 11 (Friend List Soundness). *If* $q \in \widetilde{F}_p$ *at time* t_3*, then there are times* $t_1, t_2 \leq t_3$ *such that* p *wants to befriend* q *at* t_1 *and* q *wants to befriend* p *at* t_2*.*

Theorem 12 (Friend-of-Friend Soundness). *If* $\mathrm{epoch}_p(q, r)$ *is odd at time* t_3*, then there are times* $t_1, t_2 \leq t_3$ *such that* q *wants to befriend* r *at* t_1 *and* r *wants to befriend* q *at* t_2*.*

Theorem 13 (Addressability). *If* $q \in \widetilde{F}_p$ *at time* t*, then* $q \in \mathit{known}_p$ *and* $p \in \mathit{known}_q$ *at* t*.*

Liveness Properties The liveness guarantees hold under the eventual-delivery property of correct runs: a message placed in an outbox is eventually delivered to its recipient and integrated, since the communicate transaction carrying it is unguarded and remains enabled until it fires (Section 2).

Lemma 15 (Mutual Friendship Liveness). *If* p *wants to befriend* q *at every time* $t' \geq t$ *and* q *wants to befriend* p *at every time* $t' \geq t$*, then there is a time* $t^* > t$ *such that, from* t^* *onward,* $\mathrm{epoch}_p(q) = \mathrm{epoch}_q(p)$ *and this common value is odd.*

Theorem 14 (Friendship Establishment). *If* p *wants to befriend* q *at every time* $t' \geq t$ *and* q *wants to befriend* p *at every time* $t' \geq t$*, then there is a time* $t^* > t$ *such that, from* t^* *onward,* $q \in \widetilde{F}_p$ *and* $p \in \widetilde{F}_q$*.*

Theorem 15 (Friend-of-Friend Visibility). *If* p *wants to befriend* q *at every time* $t' \geq t_1$*,* q *wants to befriend* p *at every time* $t' \geq t_1$*,* q *wants to befriend* r *at every time* $t' \geq t_2$*, and* r *wants to befriend* q *at every time* $t' \geq t_2$*, then there is a time* $t^* > \max(t_1, t_2)$ *such that, from* t^* *onward,* $\mathrm{epoch}_p(q, r)$ *is odd, so* (q, r) *appears in the friend-of-friend view at* p*.*

Theorem 16 (Unfriend Propagation). *If from some time* t *onward* p *does not want to befriend* q*, or from* t *onward* q *does not want to befriend* p*, then there is a time* $t^* \geq t$ *such that, from* t^* *onward:*

1. $q \notin \widetilde{F}_p$ *and* $p \notin \widetilde{F}_q$*; and*
2. *for every* r *that is a sustained mutual friend of* p *or of* q*,* $\mathrm{epoch}_r(p, q)$ *is even, so* (p, q) *does not appear in the friend-of-friend view at* r*.*

D.8 The Security Layer: Identity Records and Recovery

We present the secure social graph as a CVA platform extending the social graph CVA implementation with identity-record transport on Befriend, storage of friends' identity records in FMap entries, and the recovery protocols Replace and Restore that implement the abstract Replace and Recover transactions (Section 4.2). Replace and Restore are given in Sections D.16 and D.15.

D.9 Platform State Extension

An identity record of $p \in \Pi$ is the pair $IR_p = (K_p, \sigma_p)$ of p's custodian set $K_p \subset \Pi$ and supermajority threshold $\sigma_p \in (1/2, 1]$, intrinsic and immutable (Section 4.2). Being intrinsic, IR_p is accessible at p as a read-only datum throughout; in particular, it survives a state-loss fault, in which only platform state is reset.

The platform state of Secure GSG-CVA extends GSG-CVA's by widening each FMap entry with the counterpart's identity record:

$$\mathrm{FMap}_p : \Pi \rightharpoonup (\mathbb{N} \times D \times 2^{\Pi} \times (1/2, 1])$$

with each entry $\mathrm{FMap}_p[q] = (\mathit{epoch}, \mathit{data}, K, \sigma)$ recording p's epoch of the friendship with q and p's view of q's identity record as received on befriend. The data component is inherited from GSG-CVA (Section D.6), opaque to the security layer, and preserved by all secure transactions. For brevity, $\mathrm{FMap}_p[q].\mathit{epoch}$, $\mathrm{FMap}_p[q].K$, $\mathrm{FMap}_p[q].\sigma$ denote the components of the record;

$$IR_p[q] := (\mathrm{FMap}_p[q].K, \mathrm{FMap}_p[q].\sigma).$$

The shorthand $\mathrm{epoch}_p(q)$ refers to $\mathrm{FMap}_p[q].\mathit{epoch}$, with convention $\mathrm{epoch}_p(q) := 0$ when $q \notin \mathrm{dom}(\mathrm{FMap}_p)$. FoFMap_p is unchanged from GSG-CVA. A write to an FMap entry shown as a triple $(\mathit{epoch}, K, \sigma)$ sets those three components and leaves the data component unchanged—$\perp$ for a newly created entry; the full entry is the 4-tuple of the state definition above.

D.10 Cargo Extension

The GSG-CVA cargo is extended so that friendship-offer messages transport the sender's identity record, for $R = (K, \sigma)$ an identity record:

- $\mathsf{friend_request}(x, R)$: friendship offer at epoch x carrying the offerer's identity record.
- $\mathsf{accept}(x, R)$: acceptance of a pending $\mathsf{friend_request}(x, \cdot)$, carrying the accepter's identity record.

The $\mathsf{unfriend}$ and $\mathsf{stream_update}$ cargos are unchanged. Additional cargo for Replace is introduced in Section D.16.

D.11 Befriend Extension

The Befriend transactions of Section D.3 are augmented to transport and store identity records. The structure is unchanged; only the cargo and the effect on FMap entries are extended.

Definition 23 (Befriend Platform Transactions, Secure Extension).

Offer friendship. *As in Section D.3, sending* $\mathsf{message}(p, q, \mathsf{friend_request}(x, IR_p))$ *in place of* $\mathsf{message}(p, q, \mathsf{friend_request}(x))$.

Accept offer. *As in Section D.3, on receipt of* $\mathsf{message}(p, q, \mathsf{friend_request}(x, R))$, *setting* $\mathrm{FMap}_q[p] := (x, K, \sigma)$ *where* $R = (K, \sigma)$, *and sending* $\mathsf{accept}(x, IR_q)$.

Resolve simultaneous offer. *As in Section D.3, with cargo and effect extended analogously.*

Integrate accept. *As in Section D.3, on receipt of* $\mathsf{message}(q, p, \mathsf{accept}(x, R))$, *setting* $\mathrm{FMap}_p[q] := (x, K, \sigma)$ *where* $R = (K, \sigma)$.

D.12 Unfriend, Inherited

The Unfriend transactions are inherited from Section D.4 unchanged, with the convention that End friendship and Integrate unfriend preserve the K and σ fields of the FMap entry while modifying only the epoch.

D.13 Stream Dissemination, Inherited

Inherited from Section D.5 unchanged. The $\mathsf{stream_update}$ cargo does not carry identity records; FoFMap entries do not record them.

D.14 Periodic Re-broadcast

The fault model (Section 4.2) allows a state loss to erase an agent's machine state, including its outbox, so a friendship change in transit can be lost and a counterpart left stale. To recover without a global resource, each agent periodically re-broadcasts a snapshot of its own friendships to its friends, and a recipient absorbs it monotonically, catching up on whatever it missed. This single mechanism serves both purposes of state recovery: it repairs an agent that lost buffered messages, and it rebuilds an agent that lost its whole machine state, which recovers passively by absorbing the checkpoints its friends continue to send (Section D.15).

The cargo is extended with $\mathsf{checkpoint}(R, L)$: a re-broadcast carrying the sender's identity record $R = IR$ and a snapshot $L = \{(f, e)\}$ of the sender's friendships, each a (friend, epoch) pair. The identity record lets a recipient that has lost its record of the sender re-establish it, the mechanism on which state-loss Restore rests (Section D.15).

Definition 24 (Periodic Re-broadcast Transactions).

Re-broadcast. *A unary transaction at p, unguarded, provided* $\mathrm{Rec}_p \neq \emptyset$*, with* $L := \{(f, \mathrm{epoch}_p(f)) : f \in \mathrm{Rec}_p\}$*:*
– *For each* $r \in \mathrm{Rec}_p$*, add* $\mathsf{message}(p, r, \mathsf{checkpoint}(IR_p, L))$ *to* o_p*.*

Integrate checkpoint. *A unary transaction at r, unguarded, provided* $\mathsf{message}(p, r, \mathsf{checkpoint}(R, L)) \in i_r$ *and the state change below is non-trivial:*
– *If* $p \notin \mathrm{dom}(\mathrm{FMap}_r)$*, writing* $(K, \sigma) := R$*, set* $\mathrm{FMap}_r[p] := (0, \bot, K, \sigma)$*, a skeleton entry at the inactive epoch* 0 *carrying the identity record of the sender; if instead* $p \in \mathrm{dom}(\mathrm{FMap}_r)$ *and* $\mathrm{FMap}_r[p]$ *is a stub (identity-record fields* $\bot$*), writing* $(K, \sigma) := R$*, set those fields to* K, σ*, promoting the stub to a full entry and preserving its epoch.*
– *If* $(r, e) \in L$ *for some* e *with* $\mathrm{epoch}_r(p) < e$*: set* $\mathrm{FMap}_r[p].epoch := e$*, preserving the* K *and* σ *fields of the entry.*
– *For each* $(f, e) \in L$ *with* $f \neq r$ *and* $\mathrm{epoch}_r(p, f) < e$*: set* $\mathrm{FoFMap}_r[p][f] := e$ *and* $\mathrm{FoFMap}_r[f][p] := e$*.*

Re-broadcast is unguarded: it is mechanical, like the advance of local time, and requires no volition. As long as there are potential receivers, it is always enabled. Thus, every agent re-broadcasts infinitely often, the property on which passive state recovery rests (Section D.15).

Integrate checkpoint heals two kinds of staleness under the epoch order. The pair $(r, e) \in L$ carries the epoch p holds for the direct p–r friendship; absorbing it when it exceeds $\mathrm{epoch}_r(p)$ repairs a direct edge on which r fell behind, for instance after r lost an accept or unfriend from p. The direct heal advances only the epoch, preserving the identity-record fields, exactly as Unfriend does (Section D.12). The remaining pairs carry the other friendships of p and are absorbed exactly as stream_update messages, refreshing $\mathrm{FoFMap}_r[p]$ and preserving FoFMap Symmetry. When the sender p is one r does not yet record, Integrate checkpoint first installs a skeleton entry for p at the inactive epoch 0 carrying the sender's identity record from R, the mechanism on which state-loss Restore rests (Section D.15); it is inert in fault-free operation, where every checkpoint sender is already a recorded friend. If instead r holds only a stub for p—installed by a Replace rebind (Section D.16) whose acknowledgement to r was lost—the identity record carried by any later checkpoint from p promotes the stub to a full entry, so periodic re-broadcast completes a friend recovered only as a stub. The epoch order guards ensure no absorption drives an epoch backward. A checkpoint never establishes an active friendship the recipient did not consent to: $\mathrm{epoch}_p(r)$ is odd only after r itself offered or accepted at that epoch, so absorbing it restores a friendship r has already willed.

D.15 Restore Protocol

We present the CVA implementation of the abstract Recover transaction (Section 4.2): the recovery by which an agent p, after a state loss that reset its machine state, rebuilds FMap_p and FoFMap_p from its friends. Recovery is entirely

passive: it requires no custodian authorisation, no dedicated recovery message, and no action by p beyond resuming under its retained key. It rests on the periodic re-broadcast already specified (Section D.14).

Fault model. A state loss at p (the abstract state-loss fault, Definition 4) resets its machine state—$known_p$, o_p, i_p, FMap_p, FoFMap_p—to the initial empty values. Only the intrinsic identity record $IR_p = (K_p, \sigma_p)$ survives, as it is an immutable attribute of the agent rather than machine state. The retained key lets p resume under the same identity; no custodian action is required, in contrast with the key-replacement Replace of Section D.16.

Recovery. After the state loss p resumes with empty FMap_p and FoFMap_p. Each friend q of p still records p—state loss touches only p's own state—so q continues to re-broadcast its periodic $\mathsf{checkpoint}(IR_q, L_q)$ to p, with $p \in \text{Rec}_q$ and $(p, \text{epoch}_q(p)) \in L_q$. On the first such checkpoint p receives from q, Integrate checkpoint (Section D.14) finds $q \notin \text{dom}(\text{FMap}_p)$ and installs the skeleton $\text{FMap}_p[q] := (0, \bot, K_q, \sigma_q)$ from the carried identity record $IR_q = (K_q, \sigma_q)$; the direct-heal clause then absorbs $(p, \text{epoch}_q(p)) \in L_q$, advancing $\text{epoch}_p(q)$ to $\text{epoch}_q(p)$ and preserving the identity-record fields, and the observer-heal clause rebuilds $\text{FoFMap}_p[q]$ from the remaining pairs of L_q. Restore adds no platform transaction and no cargo of its own: recovery is carried entirely by Integrate checkpoint.

Coverage. Every friend q that still records p re-broadcasts to p infinitely often (Section D.14), so under reliable delivery p receives a checkpoint from each; at quiescence FMap_p holds $(q, \text{epoch}_q(p))$ with q's identity record, and FoFMap_p holds q's friendships, for every such q. Recovery is therefore complete and needs no custodian: p recovers every friendship still recorded at the friend's side, with no residue. This is the asymmetry with Replace, whose new identity is recorded by no friend and so cannot be recovered passively, requiring the active cascade of Section D.16. A checkpoint carries an odd epoch for the q–p edge only when q records that edge active, which q did only after p's own offer or acceptance (Message Bounds); so every active friendship p recovers reflects one p itself willed before the fault, and Friend List Soundness is preserved.

D.16 Replace Protocol

We present the CVA implementation of the abstract Replace transaction (Section 4.2): the protocol by which an agent p whose person has suffered identity loss is replaced by a fresh agent p' across the social graph, authorised by a supermajority of p's custodians.

Protocol outline. Replace proceeds in three phases. Phase 1 is off-chain: the person behind p chooses a new keypair, obtains a new machine if needed, and convinces a supermajority of p's custodians to authorise the replacement. Phase 2 is on-protocol vouching: each willing custodian $c \in K_p$ sends a vouch to p' carrying its own view of p's friends (to bootstrap p''s knowledge of whom to notify). Phase 3 is the cascade: p', once a supermajority has vouched, sends $\mathsf{new_identity}$ to every friend of p it has learned about; each recipient q verifies

the vouches against its stored $IR_q[p]$, renames $p \to p'$ in FMap_q and FoFMap_q (preserving the epoch), and returns a rebind to p'; p' integrates the rebinds to rebuild $\mathrm{FMap}_{p'}$ and $\mathrm{FoFMap}_{p'}$. Each rebind also carries the rebinding friend's own view of p's friends, which may reveal friends of p that no voucher named; p' notifies these in turn, so the cascade reaches every friend of p recorded by a custodian or an already-reached friend.

Phase 1. Outside the protocol. The person behind p selects p' (a fresh agent, not yet befriended by anyone) together with $IR_{p'} = (K_{p'}, \sigma_{p'})$, and convinces custodians off-chain.

Messages The cargo space C is extended with:
- $\mathsf{vouch}(p, p', L)$: custodian authorisation for the replacement of p by p', carrying a list L of (f, e) pairs drawn from the custodian's FoFMap$[p]$, for bootstrapping p''s knowledge of p's friends.
- $\mathsf{new_identity}(p, p', V, R')$: announcement that p is being replaced by p'; $V \subseteq \Pi$ is the set of custodians that have vouched, $R' = IR_{p'}$ is p''s identity record.
- $\mathsf{rebind}(e, R, L, L_p)$: acknowledgement sent by a friend q to p', carrying q's epoch e with p (now renamed as with p'), q's identity record R, a list L of (f, e') pairs drawn from q's FMap snapshot (excluding p'), and a list L_p of (w, e'') pairs drawn from q's view of the replaced identity's friends, $\mathrm{FoFMap}_q[p']$ (after the rename), for propagating the recovery to friends of p that no voucher named.

Platform Transactions

Definition 25 (Replace Platform Transactions).

Vouch. *A unary transaction at $c \in \Pi$, guarded by $\{c\}$, for a person-chosen $p' \in \Pi \setminus \{c\}$ and a $p \in \mathrm{dom}(\mathrm{FMap}_c)$:*
- *Let $L_c := \{(f, e) : f \in \mathrm{dom}(\mathrm{FoFMap}_c[p]),\ f \neq p,\ e = \mathrm{FoFMap}_c[p][f]\}$.*
- *Add $\mathsf{message}(c, p', \mathsf{vouch}(p, p', L_c))$ to o_c.*

Announce new identity. *A unary transaction at p', guarded by $\{p'\}$, for $p \in \Pi$ with $p \neq p'$, provided there exists $V \subseteq K_p$ with $|V| \geq \lceil \sigma_p \cdot |K_p| \rceil$ and, for every $c \in V$, some $\mathsf{message}(c, p', \mathsf{vouch}(p, p', L_c)) \in i_{p'}$ (the L_c's fixed by these messages). Let the known friends of p be*

$$N \;:=\; V \;\cup\; \bigcup_{c \in V} \{f : (f, e) \in L_c\} \;\cup\; \{w : \mathrm{FMap}_{p'}[w] \text{ is a stub entry}\},$$

where a stub entry is one whose identity-record fields are $\perp$ (installed by Integrate rebind below). For each $w \in N \setminus \{p, p'\}$ with $w \in known_{p'}$, $\mathrm{FMap}_{p'}[w]$ not a full entry, and no $\mathsf{message}(p', w, \mathsf{new_identity}(p, p', V, IR_{p'})) \in o_{p'}$:
- *Add $\mathsf{message}(p', w, \mathsf{new_identity}(p, p', V, IR_{p'}))$ to $o_{p'}$.*

The values K_p and σ_p are known to the person behind p' out-of-band, the same person being behind p and p'. The vouch messages are not consumed, so V and the L_c's are recomputable at each firing; N grows only as Integrate rebind installs stubs for friends of p revealed by rebinds. Because N is monotone and the

agent set is finite, the transaction notifies each reachable friend of p and reaches a fixpoint after finitely many rounds; a duplicate new_identity *is harmless, as Integrate new identity drops it once the recipient has renamed p.*

Integrate new identity. *A unary transaction at q, unguarded, provided* $\mathsf{message}(\cdot, q, \mathsf{new_identity}(p, p', V, R')) \in i_q$, $p \in \mathrm{dom}(\mathrm{FMap}_q)$, *and, writing* $(K, \sigma) := IR_q[p]$, $V \subseteq K$ *and* $|V| \geq \lceil \sigma \cdot |K| \rceil$*:*

- *Let* $e_q := \mathrm{epoch}_q(p)$, *and* $(K', \sigma') := R'$.
- *Remove the entry* $\mathrm{FMap}_q[p]$*; set* $\mathrm{FMap}_q[p'] := (e_q, K', \sigma')$.
- *For every* $y \in \mathrm{dom}(\mathrm{FoFMap}_q)$ *with* $y \neq p$*: if* $\mathrm{FoFMap}_q[y][p]$ *is defined, move its value to* $\mathrm{FoFMap}_q[y][p']$ *and delete* $\mathrm{FoFMap}_q[y][p]$.
- *If* $\mathrm{FoFMap}_q[p]$ *is defined, move the entire inner map to* $\mathrm{FoFMap}_q[p']$ *and delete* $\mathrm{FoFMap}_q[p]$.
- *For every* $f \in \mathrm{dom}(\mathrm{FMap}_q)$ *with* $p \in \mathrm{FMap}_q[f].K$*: set* $\mathrm{FMap}_q[f].K := (\mathrm{FMap}_q[f].K \setminus \{p\}) \cup \{p'\}$.
- *Let* $L_q := \{(f, e) : f \in \mathrm{dom}(\mathrm{FMap}_q),\ f \notin \{p, p', q\},\ e = \mathrm{epoch}_q(f)\}$ *and* $L_q^p := \{(w, e'') : w \in \mathrm{dom}(\mathrm{FoFMap}_q[p']),\ w \notin \{p', q\},\ e'' = \mathrm{FoFMap}_q[p'][w]\}$ *(both computed after the rename). Add* $\mathsf{message}(q, p', \mathsf{rebind}(e_q, IR_q, L_q, L_q^p))$ *to* o_q.

Integrate rebind. *A unary transaction at p', unguarded, provided* $\mathsf{message}(q, p', \mathsf{rebind}(e, R, L, L_p)) \in i_{p'}$ *with* $q \neq p'$*, and the state change below is non-trivial:*

- *Let* $(K, \sigma) := R$. *If* $\mathrm{epoch}_{p'}(q) < e$, *or* $\mathrm{FMap}_{p'}[q]$ *is a stub entry, set* $\mathrm{FMap}_{p'}[q] := (\max(\mathrm{epoch}_{p'}(q), e), K, \sigma)$ *(promoting a stub to a full record).*
- *For each* $(f, e') \in L$*: if* $\mathrm{epoch}_{p'}(q, f) < e'$, *set* $\mathrm{FoFMap}_{p'}[q][f] := e'$ *and* $\mathrm{FoFMap}_{p'}[f][q] := e'$ *(preserving FoFMap Symmetry).*
- *For each* $(w, e'') \in L_p$ *with* $w \notin \{p, p', q\}$ *and* $w \notin \mathrm{dom}(\mathrm{FMap}_{p'})$*: install the stub entry* $\mathrm{FMap}_{p'}[w] := (e'', \bot, \bot)$*, marking w as a friend of p to be notified by Announce new identity.*

Verification locality. Integrate new identity verifies the vouching set V against q's own stored $IR_q[p] = (K_p, \sigma_p)$, which was received from p upon initial befriend and is immutable within p's identity. No global custodian registry is consulted; each $r \in F_p$ verifies independently and consistently against the same identity record. On the same evidence the transaction also replaces p by p' in every custodian set q stores, so that a later Replace vouched for by p' passes this guard.

Changing custodians. Although the custodian set of an identity is fixed, a person may replace their custodians by invoking an identity change. In particular, should a custodian die, defect, or become unavailable, the person replaces their identity, choosing a fresh custodian set and threshold for the new one.

Post-replacement isolation. After q executes Integrate new identity, $p \notin \mathrm{dom}(\mathrm{FMap}_q)$. Every reactive transaction at q whose precondition references an incoming message from p additionally requires $p \in \mathrm{dom}(\mathrm{FMap}_q)$ and therefore fails. Messages from the compromised old key are silently ignored.

Inactive friendships. Integrate new identity makes no case split on the parity of $\text{epoch}_q(p)$; both active and inactive friendships are renamed from p to p', carrying the pre-rename epoch over. An inactive friendship between q and p becomes an inactive friendship between q and p' at the same epoch, without re-establishing the friendship. Should the person behind q later wish to re-friend, a fresh Befriend with p' is initiated via the normal protocol (Section D.3), starting a new active epoch.

Chained identity. A friend may miss one or more replacements of a given agent. If p is replaced in succession, $p \to p' \to p''$, and a friend q processes $\mathsf{new_identity}(p', p'', \cdot, \cdot)$ without having processed $\mathsf{new_identity}(p, p', \cdot, \cdot)$, then $p' \notin \text{dom}(\text{FMap}_q)$, the precondition of Integrate new identity fails, and the message is dropped: q has no record of p' to rename. Two recovery paths are available, the choice left to the person behind q. (i) q verifies the chain of $\mathsf{new_identity}$ announcements back to the most recent identity it recognises, checking each step against the identity record carried for the previous step — the first step against the immutable $IR_q[p]$ it stored on befriend, each subsequent step against the identity record announced by the step before it — and then applies the cumulative rename $p \to p''$, preserving the epoch. (ii) The person behind q learns out of band, by the same means used to designate custodians, that p'' succeeds p, and wills the cumulative rename $p \to p''$: the retained $\mathsf{new_identity}$ announcement in i_q supplies p'''s identity record, the epoch is preserved, and q's rebind promotes the stub p'' holds for q. Both paths preserve the epoch. If q recognises no identity in the chain and cannot reach the person, it is a stranded friend in the sense of the residue below.

Coverage. The cascade recovers every friendship between p and a friend w that was recorded, before the fault, at some friend of p connected to the seed (Theorem 18). The one edge it cannot reach is a w–p friendship recorded nowhere but at w and p themselves, p's $\mathsf{stream_update}$ having reached no other agent before the fault. This is the sole unrecoverable residue (Section 6, "The unrecoverable fault"; Definition 5).

D.17 Properties of the Security Layer

We state the additional CVA-level invariants introduced by the security layer and the two main theorems: implementation of the abstract secure social graph spec, and post-quiescence convergence of the Replace cascade. The base invariants (Knowledge Monotonicity, Friendship Monotonicity, Message Bounds, Observer Bound, FoFMap Symmetry; Section D.7) and the safety properties proved from them carry over. Their proofs enumerate the base writers; each writer the security layer adds either adopts an epoch its sender already held, re-files an entry under another identity of the same person without changing its epoch, or installs a skeleton at the inactive epoch 0 that no odd-epoch claim sees.

Invariants

Lemma 16 (Half Friendship Mutuality at CVA). *Consider a run of the secure social graph CVA implementation that uses only the Befriend, Unfriend, and Replace transactions (no Periodic Re-broadcast and no Restore) in which, whenever an agent is replaced, its cascade reaches every friend connected to its vouched seed through the disseminated graph (the hypothesis of Theorem 18). If* $\mathrm{epoch}_p(q) = x$ *with* x *odd at time* t*, then at* t *at least one of:*

1. $\mathrm{epoch}_q(p) \in \{x, x+1\}$*;*
2. *a* friend_request*,* accept*,* unfriend*, or* rebind *message of epoch* $\geq x$ *between* p *and* q *is in transit;*
3. q *has been replaced—*q *ran Replace to a new identity* q'*—and* p *has not yet integrated the rename, so a* $\mathsf{new_identity}(q, q', \cdot)$ *addressed to* p *is in transit or in* i_p*.*

Base safety and the invariants carry over as above; base liveness carries over except at the unrecoverable residue.

Corollary 3 (Secure Friendship Establishment). *On a friendship-preserving run (Definition 5), if* p *and* q *each want to befriend the other from some time on, then from some time on* $q \in \widetilde{F}_p$ *and* $p \in \widetilde{F}_q$*, unless a Replace of* p *or* q *drops the* p*–*q *edge as the unrecoverable residue.*

Proof. By Friendship Establishment (Theorem 14) the base handshake completes and both record the edge active. Unfriend fires only on an End-friendship volition, excluded by persistent wanting; Restore reinstalls the edge from the friends' checkpoints; Replace renames it, reaching the counterpart on a friendship-preserving run (Theorem 18) unless the edge is the residue. Hence it persists or recovers, the residue excepted.

Implementation

Definition 26 (Abstract Friend Set at CVA). *At a secure social graph CVA state, relabel each replaced identity to the original it replaced. The* abstract friend set *of a live agent* p *is*

$$\widetilde{F}_p \;:=\; \{\, q \in \mathrm{dom}(\mathrm{FMap}_p) : \mathrm{epoch}_p(q) \text{ is odd} \,\} :$$

the set of friendships of p *as recorded in* p*'s state.*

Every quiescent state of a friendship-preserving run is mapped correctly to a state of the secure social graph (Section 4.2), except for the unrecoverable residue (Remark below).

Two results are shown, paralleling the abstract level (Theorems 3 and 4): the Correspondence maps each completed protocol to its abstract transaction at quiescence on friendship-preserving runs, the unrecoverable residue excluded; the Fault-Resilience then shows that a state-loss fault and its Restore net to a stutter at quiescence, extending the mapping to runs that contain faults.

Theorem 6 (Secure Quiescent Correspondence). *Each completed transaction protocol of the secure social graph CVA implementation realises, under the mapping $F_p \mapsto \widetilde{F}_p$, the corresponding transaction of the abstract secure social graph (Section 4.2); the correspondence is at the granularity of completed protocols at quiescence, not a step-by-step bisimulation:*

1. *A completed Befriend handshake in CVA realises the abstract Befriend(p, q).*
2. *A completed Unfriend in CVA realises the abstract Unfriend(p, q).*
3. *A completed Replace cascade in CVA (Vouch, Announce new identity, Integrate new identity, Integrate rebind) realises the abstract Replace(p, p') on every friendship it reaches; on a friendship-preserving run (Definition 5) it reaches every recoverable friend. The one unrecoverable friendship—recorded, at the fault, only by the two friends, the identity of one then lost—is not recovered; the specification does not require its recovery (Definition 5).*
4. *A completed Restore in CVA, after a state loss at p, realises the abstract Recover(p, q): it sets $\widetilde{F}_p$ to the friends that still record p, which at quiescence is the full set of friendships recorded at p before the state loss.*

Corollary 1 (The CVA Implementation Securely Realises the Social Graph). *The secure social graph CVA implementation securely realises the social graph (Section 4.1), composing the Secure Quiescent Correspondence with the implementation of the social graph by the secure social graph (Theorem 3) and transitivity of implementation, on friendship-preserving runs at quiescence.*

Theorem 17 (CVA Fault-Resilience). *Let F'' be the CVA state-loss faults: the resets of known$_p$, o_p, i_p, FMap_p, and FoFMap_p to empty (Section D.15), disjoint from the CVA transactions, and let $\sigma'' := (F_p \mapsto \widetilde{F}_p)$. Restricted to friendship-preserving runs, the secure social graph CVA implementation is, at quiescence, an F''-resilient implementation of the abstract secure social graph (Definition 13): a state-loss fault and the Restore that follows it return $\widetilde{F}_p$ to its pre-fault value, so at quiescence they are a stutter and σ'' maps every live run over $T'' \cup F''$ to a correct run of the abstract secure social graph.*

As σ maps correct runs of the abstract secure social graph to correct runs of the social graph (Theorem 3), the composite $\sigma \circ \sigma''$ is F''-resilient over the social graph at quiescence (Lemma 4).

The construction is modular: the base social graph CVA implementation (Appendix D) is extended by the security layer, leaving the base transactions unchanged.

Convergence

Lemma 17 (Repair after Faults). *Consider a friendship-preserving run with finitely many faults, the last at time t_F. Then there is a time $t^* \geq t_F$ from which onward no staleness caused by the faults remains:* $\mathrm{epoch}_p(q)$ *agrees with* $\mathrm{epoch}_q(p)$ *except while a volitional transaction between p and q is in progress,*

and $\mathrm{epoch}_p(q, r)$ *equals* $\mathrm{epoch}_q(r)$ *for every observer* p*. From* t^* *the run is indistinguishable from one of the same volitions in which no message was lost, so Friendship Establishment (Theorem 14), Friend-of-Friend Visibility (Theorem 15) and Unfriend Propagation (Theorem 16) hold with* t^* *in place of the initial time.*

Definition 27 (Quiescence before Replace). *A run is quiescent at* p *before Replace if, at the moment Announce new identity fires at* p'*, every* stream_update *message* p *has ever sent has been delivered and integrated by its recipient—in particular, every recipient* $c \in K_p$ *has integrated every* $\mathsf{stream_update}(w, \cdot)$ *ever emitted by* p*.*

Theorem 18 (Replace Convergence). *Suppose the run is quiescent at* p *before Replace (Definition 27) and every* $c \in K_p$ *vouches (so* $V = K_p$*). Then the cascade reaches every friend of* p*: each* $w \in F_p$ *receives* new_identity*, renames* $p \to p'$*, and rebinds. After the cascade terminates,* $\widetilde{F}_{p'} = F_p$ *and for every* $r \in F_p$ *the friendship between* r *and* p *has been renamed to one between* r *and* p'*, realising the abstract Replace exactly.*

Remark (the unrecoverable residue). Under the quiescence hypothesis the cascade reaches all of F_p and recovery is exact; the identity-loss residue without it is the unrecoverable fault of Section 6 (Definition 5).

Lemma 18 (Recovery Requires a Record). *Let the friendship between* p *and* w *be unrecoverable at time* t *(Definition 5). Then in every continuation in which neither person wills a fresh Offer friendship or acceptance toward the other, the two never again record each other.*

E Secure Grassroots Coins

We extend the grassroots coins specification [39] with the sovereign's transaction log and state custodians, specified as guarded multiagent atomic transactions.

In grassroots coins [36,39], each agent issues their own currency: the issuer of a coin—its sovereign—is the natural authority on transactions in that currency. The Grassroots Flash payment system [21] realises this by having the sovereign approve each payment in their coins, with the sovereign's personal blockchain serving as the authoritative ledger. Here, we abstract this as a transaction log L_p maintained by each sovereign p, recording every transaction in p-coins. Since L_p is append-only, it is monotonic. To enable recovery if p suffers a fault, p designates some of its friends as state custodians at the formation of its currency, who hold copies of L_p.

E.1 Specification with Guarded Transactions

Each agent $p \in P$ has a machine state extending the secure social graph (Section 4.2) with:

- $S_p \subseteq F_p$: the state custodians of p—a finite set of p's friends, fixed at the formation of p's currency and immutable thereafter.
- B_p: the multiset of coins currently held by p.
- L_p: the transaction log of p-coins—an append-only sequence recording every transaction in p-coins.

A sovereign forms its currency once, before any transaction in its coins, fixing the custodian set S_p; until then S_p is undefined. Initially $B_p = \emptyset$ and $L_p = \varepsilon$ (the empty sequence) for all p, and on formation $L_p^r = \varepsilon$ for each $r \in S_p$. Each state custodian $r \in S_p$ holds a copy L_p^r of L_p as part of r's machine state. The custodian set is intrinsic and immutable. Replace(p, p') (Definition 3) extends to the coins state: $S'_{p'} := S_p$, $B'_{p'} := B_p$, $L'_{p'} := L_p$, and each custodian copy L_p^r is relabelled $L_{p'}^r$.

Definition 28 (Valid Log, Derived Holdings). *A sequence of transaction records is valid if, executed in order from the initial state, each record's transaction is enabled in the state produced by the records before it. The holdings derived from a valid log are the coin holdings of the state it produces. In particular, a coin is held by an agent in a valid log if the derived holdings assign it to that agent, and unspent by a prospective payer if the derived holdings still assign it to that payer.*

The transactions—Mint, Pay, Redeem, and Swap—correspond to the grassroots coins transactions [39], extended with log updates at the sovereign and its state custodians.

Definition 29 (Secure Coins Transactions). *The* ***secure coins transactions*** *are:*

1. ***Form currency***(p, S)*: where $S \subseteq F_p$ and p has not yet formed its currency. $S'_p := S$; for each $r \in S$: $L_p^r := \varepsilon$. Participants $\{p\} \cup S$. Guarded by p.*
2. ***Mint***(p, k, t)*: $B'_p := B_p \cup \mathfrak{c}_{p,t}^k$, $L'_p := L_p \cdot [mint(k, t)]$. For each $r \in S_p$: $L_p^{r\prime} := L'_p$. Participants $\{p\} \cup S_p$. Guarded by p.*
3. ***Pay***(q, r, x)*: where $x \subseteq B_q$ is a set of s-coins for some sovereign s. $B'_q := B_q \setminus x$, $B'_r := B_r \cup x$, $L'_s := L_s \cdot [pay(q, r, x)]$. For each $u \in S_s$: $L_s^{u\prime} := L'_s$. Participants $\{q, r, s\} \cup S_s$. Guarded by q.*
4. ***Redeem***(q, s)*: where $\mathfrak{c}_s \in B_q$ and $\mathfrak{c}_{r,t} \in B_s$. $B'_q := (B_q \setminus \{\mathfrak{c}_s\}) \cup \{\mathfrak{c}_{r,t}\}$, $B'_s := (B_s \setminus \{\mathfrak{c}_{r,t}\}) \cup \{\mathfrak{c}_s\}$, $L'_s := L_s \cdot [redeem(q, \mathfrak{c}_s, \mathfrak{c}_{r,t})]$. For each $u \in S_s$: $L_s^{u\prime} := L'_s$. Participants $\{q, s\} \cup S_s$. Guarded by q.*
5. ***Swap***(p, q, x, y)*: where $x \subseteq B_p$, $y \subseteq B_q$. $B'_p := (B_p \setminus x) \cup y$, $B'_q := (B_q \setminus y) \cup x$. For each sovereign s whose coins appear in $x \cup y$: $L'_s := L_s \cdot [swap(p, q, x_s, y_s)]$, and for each $u \in S_s$: $L_s^{u\prime} := L'_s$. Participants $\{p, q\}$ together with each such sovereign s and its custodians S_s. Guarded by $\{p, q\}$.*

In each transaction in p-coins, the sovereign p is a participant: p's log grows, and all state custodians' copies are updated atomically. Since every transaction in p-coins is enabled only when the coins it moves are held by the agent it debits, the log L_p is valid by construction, and the holdings of p-coins by any agent are the holdings derived from L_p (Definition 28). Pay and Redeem are guarded

by the payer/redeemer alone—the sovereign's person need not consent—but the sovereign's machine must be live for the transaction to execute. If the sovereign is unresponsive, transactions in their currency are blocked; holders who lose trust may treat the sovereign's coins as bad debt [39].

E.2 Properties

Safety.

Lemma 19 (Log Consistency). *In any run of the secure coins protocol, $L_p^r = L_p$ for every state custodian $r \in S_p$.*

Lemma 20 (Conservation of Money). *In any run of the secure coins protocol, the p-coins across all agents' holdings are exactly the p-coins recorded as minted in L_p.*

Recovery.

Recovery of the social graph proceeds via Replace (Definition 3). For the sovereign's transaction log, the person behind the recovered agent contacts any available state custodian to obtain a copy. Since log copies are maintained in sync (Lemma 19), a single available state custodian suffices.

Theorem 19 (Coin Recovery). *Let p be an agent with $K_p \neq \emptyset$ and $S_p \neq \emptyset$. If* Replace(p, p') *is taken and some $r \in S_p$ is correct, then $F_{p'} = F_p$, $S_{p'} = S_p$, and $L_p^r = L_p$.*

By Lemma 20, the recovered log L_p^r determines the correct holdings of all p-coins, enabling the recovered agent p' to resume as sovereign.

Grassroots.

Theorem 20. *The secure grassroots coins protocol is grassroots.*

Recovery without key loss.

If a sovereign p suffers a state loss—losing machine state but retaining their key—coin recovery is a special case of Recover (Definition 3). The person authenticates to any state custodian $r \in S_p$ using their retained key and obtains $L_p^r = L_p$ (Lemma 19). The recovered log determines the correct holdings of all p-coins (Lemma 20), and the sovereign resumes operation. For a trader q who suffers a state loss and loses their local coin cache, the person behind q contacts each friend sovereign s after Recover and asks whether q holds any s-coins; each sovereign can answer from their log L_s. No custodian authorization is required in either case.

F Secure Grassroots Coins: CVA Implementation

We implement the secure coins specification (Section E) as a CVA platform (Section 5), extending the secure social graph CVA implementation (Section D.8). At the specification the state custodians are updated in the same atomic step as the sovereign, so each holds the complete log and a single available custodian suffices for exact recovery. An asynchronous implementation has no such atomicity: a custodian's copy of the log may lag the sovereign's. Recovering from a single lagging custodian could reinstate a log that omits an already-approved payment, and the recovered sovereign might then approve the same coin a second time—an inadvertent double-spend. We instead restore exactness by coupling finality to recoverability through a supermajority of custodians: a payment is final, and a payee may rely on it, only once a supermajority of the sovereign's state custodians hold it, and recovery collects copies from a supermajority. Any two supermajorities of the same set intersect, so every final payment is recovered. A recovery is itself final by a double supermajority: a supermajority recording its new epoch, then a supermajority acknowledging the log it adopted.

Scope: a sound sovereign. The coins of a currency are as sound as its sovereign's operating system. We assume that operating system is sound: it may crash and lose state, but it never deviates from the protocol. The results below are stated for such currencies; a currency whose sovereign's operating system is compromised is outside our scope, since no protocol among its custodians can protect its holders from their own sovereign.

This is the supermajority-intersection structure of the All-to-All Flash payment system [20], here in two simpler forms. Our fault model is crash-only (Section 4.2), so the quorums are plain intersecting read and write quorums—a write quorum and a read quorum of a set of size n intersect once each exceeds $n/2$—with none of the slack a Byzantine setting requires. And the sovereign is the sole writer of its log, so the task is the durability of one writer's log across its custodians, not agreement among competing writers; no consensus is invoked.

Every CVA protocol is grassroots by construction (Theorem 5); the platform below is therefore grassroots without a separate argument.

F.1 Platform State

The platform state of p extends the secure social graph CVA state (Section D.9) with the components of a sovereign, a holder, and a custodian:

- $e_p \in \mathbb{N}$: the current epoch number at agent p, initially 0.
- B_p: the multiset of coins held by p, initially empty. This is a conservative local account, debited when p requests a payment and credited when a payment to p becomes final; the authoritative record of holdings is the sovereigns' logs, from which holdings are derived (Definition 28), and B_p is p's local view of its own entry in them.
- L_p: the log of p's currency, a sequence of blocks; blocks are associated with monotonically increasing epoch numbers; the block at position $n \in \mathbb{N}$ in epoch

e is $\mathsf{block}(e, n, \tau)$ with τ a transaction record, one of $\mathsf{mint}(k, t)$, $\mathsf{pay}(q, r, x)$, $\mathsf{redeem}(q, c, c')$ with c the bond redeemed and c' the coin returned, or $\mathsf{swap}(q, r, x, y)$. Initially empty.
- $\ell_p : S_p \to \mathbb{N} \times \mathbb{N}$, the ack frontier: $\ell_p(c)$ is the maximal pair e, n (pairs ordered lexicographically) associated with a block that custodian c has acknowledged holding, initially $0, 0$.
- for each sovereign s with $p \in S_s$, a copy $\widehat{L}_p[s]$ of L_s together with the epoch $\widehat{e}_p[s] \in \mathbb{N}$ of s's currency that p has registered and the epoch $\widehat{\pi}_p[s] \geq \widehat{e}_p[s]$ that p has promised to a recovering s (Recovery below), created empty and at epochs 0 when p is designated a custodian of s (Become custodian below). In the absence of failures, $\widehat{L}_p[s]$ is a prefix of L_s and $\widehat{e}_p[s] = \widehat{\pi}_p[s] = e_s$.

The state-custodian record of p's currency is the set $S_p \subseteq F_p$, fixed at currency formation and immutable thereafter (Section E), together with a state-custodian threshold $\sigma^S \in (1/2, 1]$, taken global to the platform; making σ^S specific to each sovereign changes nothing below. A supermajority of S_p is a subset $G \subseteq S_p$ with $|G| \geq \lceil \sigma^S \cdot |S_p| \rceil$. Since $\sigma^S > 1/2$, any two supermajorities of S_p intersect.

The custodians and the holders that transact in a currency are friends of its sovereign—custodians by $S_p \subseteq F_p$, holders by befriending the sovereign before transacting—so every message below travels between friends, whose channels the secure social graph layer maintains (Section D.8); the CVA known-peer precondition on emission (Section 5) is thereby met. A payer communicates only with the sovereign and the sovereign only with its custodians; a payer need not be a friend of the custodians.

F.2 Messages

The cargo space C is extended with, for a transaction record τ, epoch $e \in \mathbb{N}$, position $n \in \mathbb{N}$, and sovereign s:
- $\mathsf{become_custodian}$: sovereign s designating the recipient as a state custodian of s-coins.
- $\mathsf{request}(\tau)$: a holder's request that sovereign s approve the transaction τ in s-coins.
- $\mathsf{block}(e, n, \tau)$: the sovereign disseminating its n-th log block in epoch e to a custodian.
- $\mathsf{ack}(e, n)$: a custodian registered in epoch e attesting that it holds the prefix of the sovereign's log through position n in epoch e.
- $\mathsf{final}(e, n, \tau)$: the sovereign notifying a party to τ that block n in epoch e, which records τ, is final.

F.3 Transactions

The protocol realises each spec transaction (Definition 29) as a sequence of unary CVA platform transactions: a holder requests, the sovereign approves and appends a block and disseminates it, each custodian appends and acknowledges, and the sovereign binds finality once a supermajority have acknowledged. A

sovereign first forms its currency, fixing its custodian set and notifying each custodian, which registers an empty log copy. We give the protocol for Pay; Mint, Redeem, and Swap differ only in the transaction record carried and in which holdings are updated on finality.

Definition 30 (Secure Coins CVA Transactions).

Form currency. *A unary transaction at sovereign p, guarded by $\{p\}$, for a chosen $S \subseteq F_p$, provided S_p is undefined (the currency is not yet formed):*
– *Set $S_p := S$ and $\ell_p(c) := (0,0)$ for each $c \in S$.*
– *For each $c \in S$, add $\mathsf{message}(p, c, \mathsf{become_custodian})$ to o_p.*

Become custodian. *A unary transaction at c, unguarded, provided $\mathsf{message}(s, c, \mathsf{become_custodian}) \in i_c$ and $\widehat{L}_c[s]$ is undefined:*
– *Set $\widehat{L}_c[s] := \varepsilon$ and $\widehat{e}_c[s] := \widehat{\pi}_c[s] := 0$, registering c as a state custodian of s.*

Request pay. *A unary transaction at a holder q, guarded by $\{q\}$, for a recipient r, a set $x \subseteq B_q$ of s-coins of some sovereign $s \in known_q$:*
– *Remove x from B_q.*
– *Add $\mathsf{message}(q, s, \mathsf{request}(\mathsf{pay}(q, r, x)))$ to o_q.*

Approve. *A unary transaction at sovereign s, unguarded, provided $\mathsf{message}(q, s, \mathsf{request}(\mathsf{pay}(q, r, x))) \in i_s$ and x is unspent in L_s (Definition 28)—each coin in x is, in the holdings derived from L_s, currently held by q:*
– *Let $n := |L_s| + 1$; append $\mathsf{block}(e_s, n, \mathsf{pay}(q, r, x))$ to L_s.*
– *For each $c \in S_s$, add $\mathsf{message}(s, c, \mathsf{block}(e_s, n, \mathsf{pay}(q, r, x)))$ to o_s.*
A request that is not unspent in L_s is dropped without appending.

Append and acknowledge. *A unary transaction at custodian $c \in S_s$, unguarded, provided $\mathsf{message}(s, c, \mathsf{block}(e, n, \tau)) \in i_c$, $e = \widehat{e}_c[s]$, and $n = |\widehat{L}_c[s]| + 1$ (the next block in order):*
– *Append $\mathsf{block}(e, n, \tau)$ to $\widehat{L}_c[s]$.*
– *Add $\mathsf{message}(c, s, \mathsf{ack}(e, n))$ to o_c.*
A block whose epoch exceeds $\widehat{e}_c[s]$, or whose position is not next in order, is retained in i_c until c registers that epoch or the predecessors arrive; a block of an earlier epoch, or one already held, is stale and dropped.

Bind finality. *A unary transaction at sovereign s, unguarded, provided $\mathsf{message}(c, s, \mathsf{ack}(e_s, n)) \in i_s$ for a custodian $c \in S_s$ with $\ell_s(c) < (e_s, n)$:*
– *Set $\ell_s(c) := (e_s, n)$.*
– *For each position m in epoch e_s that has become final at this step—$|\{c' \in S_s : \ell_s(c') \geq (e_s, m)\}| \geq \lceil \sigma^S |S_s| \rceil$ for the first time—with $\mathsf{block}(e_s, m, \tau)$ in L_s, add $\mathsf{message}(s, a, \mathsf{final}(e_s, m, \tau))$ to o_s for each party a of τ: the payer q and payee r of a $\mathsf{pay}(q, r, x)$; both parties q, r of a $\mathsf{swap}(q, r, x, y)$; the redeemer q and s for a $\mathsf{redeem}(q, \cdot, \cdot)$; and s for a $\mathsf{mint}(k, t)$.*

Accept payment. *A unary transaction at recipient r, unguarded, provided $\mathsf{message}(s, r, \mathsf{final}(e, n, \mathsf{pay}(q, r, x))) \in i_r$ with r the payee:*
– *Add x to B_r.*

An $\mathsf{ack}(e, n)$ attests a prefix through position n in epoch e, so it counts toward the finality of every position $m \leq n$ of that epoch; a custodian that

has acknowledged any $n \geq m$ in epoch e is among the holders of block (e, m). Transactions in old epochs are already finalised in the recovery process before the epoch begins, and therefore only blocks in the current epoch are considered here. Bind finality therefore finalises, in one step, every position up to the new frontier that the new acknowledgment carries past the threshold. Mint, Redeem, and Swap follow the same four-transaction protocol: Mint is requested by the sovereign's own person and adds the minted coins to B_s on approval, Redeem and Swap carry their own records and update the relevant holdings on finality; in each the sovereign appends a block, disseminates it, and binds finality at a supermajority of acknowledgments exactly as for Pay.

F.4 Finality

Definition 31 (Finality). *A block at position (e, n) of L_s is **final** when a supermajority of s's state custodians hold the prefix of L_s through (e, n). A payment is **final** when its block is final.*

A payee relies on a payment only once it is final—only upon receiving final, which the sovereign sends only after a supermajority have acknowledged. Finality is the line between safe-to-rely and not: a payment the sovereign has approved but a supermajority has not yet acknowledged is not final, may be dropped on recovery, and is safe to drop precisely because no payee was told it was final.

A payment commits when it is requested: Request pay debits the payer's account at once, and a payment that is never approved (the request is dropped) or never reaches a supermajority is not refunded. This loses the payer nothing of value. A payment in s-coins can fail to finalise only if the sovereign s or a supermajority of its custodians is permanently unavailable, or if s loses its state before a supermajority hold the payment's block, in which case recovery drops it; in the first case no s-coin can ever finalise again, so every s-coin is worthless, and the coins stranded in the failed payment are worth no less than the rest, while in the second the payer loses the coins of a payment that was never final and that no payee relied on. An unresponsive sovereign renders its currency bad debt regardless [39]; the conservative account merely reflects this.

F.5 Recovery

The state-custodian record of a currency—the set S_p together with the threshold σ^S—is fixed at currency formation and is intrinsic in the same sense as the identity record (Section D.9): a read-only datum that survives a fault in which platform state is reset. A recovering sovereign therefore knows from whom to collect and how many copies it needs before it has recovered any log—knowledge that cannot be made to depend on the log itself, which is what is being recovered.

The exchanges in the procedures below are conducted out-of-band by the recovering person and are not modelled as platform cargo; only their outcome matters to the properties—the promises, registered epochs and log copies of a

supermajority for a sovereign, or the answer the sovereign reads from its log for a holder.

State loss. A sovereign p that suffers a state loss and retains its key resumes with empty L_p and recovers in three exchanges with its custodians. At the start of its recovery p draws a fresh nonce, one no earlier incarnation of p has drawn, and tags every message of the three exchanges with it; custodians echo it, and p counts only replies bearing it, so a reply still in transit to a previous incarnation is never mistaken for one to p. It first queries them and waits for replies from a supermajority $G \subseteq S_p$, each custodian c answering with its promised epoch $\widehat{\pi}_c[p]$, its registered epoch $\widehat{e}_c[p]$, and its copy $\widehat{L}_c[p]$. It then reserves the epoch $e' := 1 + \max_{c \in G} \widehat{\pi}_c[p]$: a custodian with $\widehat{\pi}_c[p] < e'$ sets $\widehat{\pi}_c[p] := e'$ and acknowledges, one with $\widehat{\pi}_c[p] \geq e'$ replies with $\widehat{\pi}_c[p]$ instead, and p waits for acknowledgements from a supermajority, reserving again above the highest reply if those replies leave it short. Finally p adopts as L_p the longest copy among those in G registered in the maximal epoch $\max_{c \in G} \widehat{e}_c[p]$, sends it with e' to every custodian, sets $e_p := e'$, and resumes approving, its next block being $\mathsf{block}(e', |L_p| + 1, \cdot)$; a custodian with $\widehat{\pi}_c[p] = e'$ and $\widehat{e}_c[p] < e'$ sets $\widehat{L}_c[p] := L_p$ and $\widehat{e}_c[p] := e'$, which may discard blocks of earlier epochs that never reached a supermajority and so were never final, and acknowledges with $\mathsf{ack}(e', |L_p|)$. Once a supermajority have so acknowledged, the adopted log is final, and p sends $\mathsf{final}(e, n, \tau)$ to the parties of each of its blocks $\mathsf{block}(e, n, \tau)$; a party already notified receives a message identical to one it holds, which its inbox, a set, absorbs. Since a supermajority promised e' and any two supermajorities intersect, every later recovery hears a promise of at least e' and reserves above it: an epoch is reserved, and a log installed under it, by one recovery only, so all copies registered in an epoch are prefixes of one log.

Identity loss. A sovereign that loses its key is replaced by a fresh agent p' through the secure social graph (Section D.16). The state-custodian record (S_p, σ^S) is known out-of-band to the person behind p'—the same person behind p and p', exactly as the new identity record is in Replace Phase 1 (Section D.16)—so p' adopts $S_{p'} = S_p$; the Replace cascade itself carries no log or custodian record. Once the custodians have processed the rename and address p', p' proceeds exactly as under state loss.

Holder recovery. A holder q that suffers a state loss and loses B_q recovers it from the sovereigns: for each friend sovereign s, the person behind q asks s which s-coins q holds, which s answers from L_s. No custodian is involved.

Changing custodians. The custodian set is fixed for the life of a currency; to change custodians, a sovereign issues a new currency with a fresh custodian set and migrates holdings to it, which the existing Mint and Swap transactions already express.

Custodian availability and delivery. The properties below assume that, for each currency, a supermajority of its state custodians remains correct and retains its log copy, and that delivery between a sovereign and its custodians is reliable. Under this assumption a custodian copy changes only by the protocol—

appending, or aligning at a recovery, which discards no final block, and no disseminated block is lost, so Prefix Consistency and Exact Recovery hold as stated. This is the standard quorum assumption: were a supermajority to fail, the currency would be permanently unable to finalise or recover and its coins would be worthless regardless. It is also where the coins layer differs from the secure social graph, which tolerates message loss and the loss of a counterpart copy through periodic re-broadcast (Section D.14); a custodian re-synchronisation that repairs a regressed or lagging copy, and so lifts the reliable-delivery assumption, is left to future work.

F.6 Properties

Lemma 21 (Prefix Consistency). *In any run, for every sovereign s and custodian $c \in S_s$ registered in the current epoch, $\widehat{L}_c[s]$ is a prefix of L_s, and within an epoch both L_s and $\widehat{L}_c[s]$ only grow.*

Lemma 22 (Log Validity). *In any run, every sovereign's log L_s is valid (Definition 28).*

Lemma 3 (Conservation of Money). *At every quiescent configuration of a run with no state-loss fault, the s-coins across all holdings are exactly those recorded as minted in L_s.*

Lemma 23 (Finality Soundness). *If a payee receives* $\mathsf{final}(e, n, \mathsf{pay}(q, r, x))$ *from s, then* $\mathsf{block}(e, n, \mathsf{pay}(q, r, x))$ *is in L_s, and was approved by s against an unspent x.*

Correspondence with the specification. The CVA protocol implements the secure coins specification (Section E) in the sense that its quiescent states are exactly the specification's states. We read an abstract state off a CVA configuration by the map α that takes the holdings B_p of each agent, the log L_p of each sovereign, and, for each custodian, its copy $\widehat{L}_p[s]$.

Theorem 7 (Correspondence at Quiescence). *At a quiescent configuration (Definition 20) of any run of the CVA protocol in which every custodian has registered the current epoch, the abstract state read off the configuration is a reachable state of the secure coins specification (Appendix E): every custodian copy equals its sovereign's log, every log block is final, and each agent's holdings equal the holdings derived from the logs.*

The correspondence is at quiescence, not at every state, and necessarily so: between Request pay and the finalisation of that payment a coin is debited from the payer and not yet credited to any payee nor recorded in any final block, a configuration the atomic specification—in which a payment moves a coin and records it in one step—never exhibits. The in-flight coin is below the abstraction; quiescence is exactly the condition under which no payment is in flight.

Theorem 21 (Coins Fault-Resilience). *Let F be the coins state-loss faults: the resets of a sovereign's L_p, ℓ_p and custodian copies, a custodian's $\widehat{L}_p[\cdot]$, or a holder's B_p to empty (Section F.5), disjoint from the coins transactions. Under the custodian-availability and reliable-delivery assumption (Section F.5), the secure coins CVA implementation is, at quiescence, an F-resilient implementation of the secure coins specification (Definition 13) under the map α: a state-loss fault and the recovery that follows it leave the derived holdings and every final block unchanged, so at quiescence they are a stutter and α maps every live run over the coins transactions and F to a correct run of the specification.*

Corollary 2 (No Inadvertent Double-Spend). *After recovery, the sovereign never approves a payment of a coin already spent in a final payment.*

Theorem 9 (Payment Liveness). *In a correct run in which the sovereign s does not suffer a state loss after receiving a valid pay request and a supermajority of S_s registered in s's current epoch does not suffer a state loss, the requested payment eventually becomes final and its payee receives notification of finality.*

Theorem 22 (Grassroots). *The secure coins and bonds CVA platform is grassroots.*

G Proofs for Grassroots Coins and Bonds

Lemma 19 (Log Consistency). *In any run of the secure coins protocol, $L_p^r = L_p$ for every state custodian $r \in S_p$.*

Proof. Initially $L_p^r = L_p = \varepsilon$ for every $r \in S_p$. Every transaction involving p-coins atomically appends the same entry to L_p and to L_p^r for all $r \in S_p$. No transaction modifies L_p or L_p^r independently.

Lemma 20 (Conservation of Money). *In any run of the secure coins protocol, the p-coins across all agents' holdings are exactly the p-coins recorded as minted in L_p.*

Proof. Mint adds p-coins to B_p and records the mint in L_p. Pay, Redeem, and Swap transfer coins between agents without creating or destroying them, and record the transfer in the relevant sovereign's log. No transaction creates coins without a corresponding log entry or destroys coins without recording the transfer.

Theorem 19 (Coin Recovery). *Let p be an agent with $K_p \neq \emptyset$ and $S_p \neq \emptyset$. If* Replace(p, p') *is taken and some $r \in S_p$ is correct, then $F_{p'} = F_p$, $S_{p'} = S_p$, and $L_p^r = L_p$.*

Proof. $F_{p'} = F_p$ follows from Replace (Definition 3). $S_{p'} = S_p$ follows from the inheritance of the custodian set under Replace. $L_p^r = L_p$ follows from Lemma 19.

Theorem 20. *The secure grassroots coins protocol is grassroots.*

Proof. Obliviousness: Let $P, P' \subset \Pi$ be disjoint and nonempty. By the obliviousness argument of Theorem 2, no social graph transaction with participants spanning both P and P' is ever enabled in any interleaving. For the coin transactions: Form currency(p, S) is guarded by p, with participants $\{p\} \cup S \subseteq \{p\} \cup F_p$, which lie in a single group in any interleaving of disjoint runs; Mint is unary. Pay(q, r, x) in s-coins requires $x \subseteq B_q$, where x consists of s-coins; in an interleaving, an agent in P never acquires coins issued by agents in P' (as this would require a prior Swap spanning both groups), so no Pay or Redeem with participants spanning both groups is enabled. Swap(p, q, x, y) with $p \in P$, $q \in P'$ requires $x \subseteq B_p$ and $y \subseteq B_q$; by the same argument, p holds no P'-coins and q holds no P-coins, so no swap with nonempty exchange spanning both groups is enabled.

Interactivity: Let $p \in P$, $q \in P'$. A run in which p and q befriend and then execute a voluntary swap—p mints p-coins, q mints q-coins, and they exchange—changes the local states of both p and q, producing a configuration not reachable by either group independently. Hence the protocol is interactive, and by the interactivity criterion of [22], grassroots.

Lemma 21 (Prefix Consistency). *In any run, for every sovereign s and custodian $c \in S_s$ registered in the current epoch, $\widehat{L}_c[s]$ is a prefix of L_s, and within an epoch both L_s and $\widehat{L}_c[s]$ only grow.*

Proof. By induction on the run. Both are initially empty, and a custodian registering the current epoch aligns its copy to a prefix of L_s. Approve appends one block, in epoch e_s, to the end of L_s and changes no copy. Append and acknowledge appends to $\widehat{L}_c[s]$ only the block at position $n = |\widehat{L}_c[s]| + 1$ in the registered epoch $\widehat{e}_c[s] = e_s$, and that block is the one Approve placed at position n of L_s and disseminated; so the appended block matches L_s at that position, extending the prefix by one. Within an epoch no transaction shortens or rewrites either sequence. Hence each remains a prefix of L_s, and both are non-decreasing.

Lemma 22 (Log Validity). *In any run, every sovereign's log L_s is valid (Definition 28).*

Proof. By induction on the length of L_s. The empty log is valid. A block is appended to L_s only by Approve, whose precondition requires the recorded transaction's coins to be unspent in L_s—held, in the holdings derived from the current L_s, by the agent the transaction debits. By the induction hypothesis L_s is valid, so its derived holdings are those of the specification state its records produce; the recorded transaction is therefore enabled in that state, and the extended log is valid. Mint, Redeem, and Swap append only via the same Approve discipline, each with its own holding check, so the same argument applies.

Lemma 3 (Conservation of Money). *At every quiescent configuration of a run with no state-loss fault, the s-coins across all holdings are exactly those recorded as minted in L_s.*

Proof. An s-coin enters a holding for the first time only through Mint, which adds the minted coins to B_s upon Approve and records $\mathsf{mint}(k, t)$ in L_s; no other transaction creates a coin, and none destroys one. Thereafter a coin leaves a holding only at the requesting step of Pay, Redeem, or Swap, which debits it from the requester, and enters one only at the finality step of the same transaction, Accept payment or its counterpart, which credits the same coin to the counterparty. Hence at every configuration the s-coins across all holdings are those recorded as minted in L_s, less the coins of requests that have been debited but not yet credited.

Let c be a quiescent configuration of a run with no state-loss fault. We show no such request exists at c. By the argument of Correspondence at Quiescence (Theorem 7), which relies only on Prefix Consistency and Log Validity, every request has been delivered and processed by Approve, every approved block has been appended by every custodian and is final, and every final has been accepted. It remains to rule out a request that Approve dropped. In a run with no state loss, a holder's B_q is credited only by the finality of a block of L_s and debited only by q's own requests, so B_q agrees with the holdings derived from L_s except for q's pending requests, whose coins are absent from B_q; the coins of a request are therefore unspent in L_s when Approve processes it, and it is approved. Every debit at c thus has a matching credit, and the holdings of s-coins are exactly those minted in L_s.

Lemma 23 (Finality Soundness). *If a payee receives* $\mathsf{final}(e, n, \mathsf{pay}(q, r, x))$ *from s, then* $\mathsf{block}(e, n, \mathsf{pay}(q, r, x))$ *is in L_s, and was approved by s against an unspent x.*

Proof. s emits $\mathsf{final}(e, n, \mathsf{pay}(q, r, x))$ only from Bind finality, with $e = e_s$, and only when $\mathsf{block}(e, n, \mathsf{pay}(q, r, x))$ is already in L_s; that block was placed by Approve, whose precondition requires x unspent in L_s at the time of approval.

Theorem 7 (Correspondence at Quiescence). *At a quiescent configuration (Definition 20) of any run of the CVA protocol in which every custodian has registered the current epoch, the abstract state read off the configuration is a reachable state of the secure coins specification (Appendix E): every custodian copy equals its sovereign's log, every log block is final, and each agent's holdings equal the holdings derived from the logs.*

Proof. At quiescence no reactive transaction is enabled, so every emitted message has been delivered and processed; otherwise the corresponding communicate, or the reactive transaction consuming it, would be enabled. Hence: no request is in flight or pending, so no payment is mid-approval; every $\mathsf{block}(e, n, \tau)$ approved has reached every custodian and been appended (every custodian having registered the current epoch, Append and acknowledge is enabled while an undelivered or unappended block remains), so $\widehat{L}_c[s] = L_s$ for every $c \in S_s$, whence by Prefix Consistency (Lemma 21) every block of L_s is held by all of S_s and is therefore final; and every $\mathsf{final}(e, n, \cdot)$ has been accepted (Accept payment is enabled while an unprocessed final remains), so every finalised credit has been

applied to the payee's B_r and every committed debit was applied at request. Thus B_p equals the holdings derived from the logs. By Log Validity (Lemma 22) each log is a valid sequence of transactions, hence a run of the specification that produces exactly these holdings; so α is the state reached by that run and is a reachable specification state.

Theorem 8 (Exact Recovery). *If a sovereign recovers—under state loss, or as p' under identity loss—by adopting the longest of the log copies in the maximal epoch collected from a supermajority $G \subseteq S_p$, then the recovered log contains every block of L_p that was final at any time before recovery.*

Proof. By induction on the recoveries of p's currency. Let $\mathsf{block}(e_0, n, \cdot)$ be final before the recovery, witnessed by a supermajority $G' \subseteq S_p$ registered in e_0 whose copies held the prefix through n at the finalising step (Definition 31). A copy changes thereafter only by appending, or by aligning at a later registration to the log adopted by that recovery, which contains the block by the induction hypothesis; so every $c \in G'$ still holds the block at recovery time. As $|G| \geq \lceil \sigma^S |S_p| \rceil$ and $|G'| \geq \lceil \sigma^S |S_p| \rceil$ with $\sigma^S > 1/2$, the two supermajorities intersect; pick $c \in G \cap G'$, whose collected copy contains the block. Reserved epochs are distinct and increase along the run: a recovery reserves e' only once a supermajority have promised it, promises never decrease, and every later recovery queries a supermajority that intersects the promising one, hears a promise of at least e', and reserves above it. Hence every copy registered in an epoch was installed by the one recovery that reserved it and extended only by blocks that recovery's sovereign approved in that epoch. Let e be the maximal epoch among the collected copies. Every copy registered in e is thus a prefix of one log, the log adopted at the reservation of e extended by the blocks of e; this log contains the block, either because c is registered in e, or because $e > \widehat{e}_c[p] \geq e_0$, so e was reserved by a recovery later than the one that reserved e_0, hence after the fault that ended the sovereign's stint in e_0 and so after the block became final, and that recovery's adopted log contains the block by the induction hypothesis. The longest copy in e therefore contains the block.

Theorem 21 (Coins Fault-Resilience). *Let F be the coins state-loss faults: the resets of a sovereign's L_p, ℓ_p and custodian copies, a custodian's $\widehat{L}_p[\cdot]$, or a holder's B_p to empty (Section F.5), disjoint from the coins transactions. Under the custodian-availability and reliable-delivery assumption (Section F.5), the secure coins CVA implementation is, at quiescence, an F-resilient implementation of the secure coins specification (Definition 13) under the map α: a state-loss fault and the recovery that follows it leave the derived holdings and every final block unchanged, so at quiescence they are a stutter and α maps every live run over the coins transactions and F to a correct run of the specification.*

Proof. The coins transactions realise the spec transactions at quiescence under α, where each quiescent log is valid (Lemma 22) and the derived holdings are the specification's. A state-loss fault at a sovereign p empties L_p; under the custodian-availability assumption (Section F.5) a supermajority retains its copies, so by

Exact Recovery (Theorem 8) the recovered log contains every final block, and the holdings derived from it agree with the pre-fault holdings on every committed coin. A custodian or holder loss is repaired likewise from the sovereign's log, which is authoritative for holdings (Definition 28). The fault and its recovery thus leave α unchanged at quiescence—a stutter—and a non-final payment that recovery may drop was relied upon by no payee (Section F.4). Hence the quiescent image of a live run over the coins transactions and F uses only spec transactions and is a correct run of the specification; that is, α is F-resilient.

Corollary 2 (No Inadvertent Double-Spend). *After recovery, the sovereign never approves a payment of a coin already spent in a final payment.*

Proof. By Exact Recovery (Theorem 8) the recovered log contains every final block, so the holdings derived from the recovered L_p (Definition 28) record every coin spent in a final payment as spent. Approve appends a payment of x only when x is unspent in L_p (Definition 30); a coin spent in a final payment is thus never re-approved. A payment that was not final may be absent from the recovered log, but no payee relied on it (Section F.4), so no relied-upon payment is reversed.

Theorem 9 (Payment Liveness). *In a correct run in which the sovereign s does not suffer a state loss after receiving a valid pay request and a supermajority of S_s registered in s's current epoch does not suffer a state loss, the requested payment eventually becomes final and its payee receives notification of finality.*

Proof. By eventual delivery in a correct run (Section 2) the $\mathsf{request}$ reaches s, which executes Approve, appends $\mathsf{block}(e_s, n, \mathsf{pay}(q, r, x))$, and disseminates it to every $c \in S_s$. Fix a custodian c in the live supermajority, registered in e_s. By eventual delivery c receives every block at positions $1, \ldots, n$; Append and acknowledge fires in order as each predecessor arrives, so the frontier $|\widehat{L}_c[s]|$ reaches n and c sends $\mathsf{ack}(e_s, m)$ for m up to n. Each such ack reaches s, which sets $\ell_s(c)$ accordingly. Once every c in the live supermajority has done so, $|\{c : \ell_s(c) \geq (e_s, n)\}|$ meets the threshold, block (e_s, n) is final, and Bind finality emits $\mathsf{final}(e_s, n, \mathsf{pay}(q, r, x))$ to q and r, delivered by eventual delivery.

Theorem 22 (Grassroots). *The secure coins and bonds CVA platform is grassroots.*

Proof. The platform transactions of Definition 30 are unary, each at a single agent, with guard $\{q\}$ or $\{p\}$ (Request pay, Form currency, and the requesting steps of Mint, Redeem, Swap) or empty (the reactive Approve, Append and acknowledge, Bind finality, Accept payment, Become custodian), and each modifies only its own agent's platform state and emits messages to known peers. They are therefore CVA platform transactions in the sense of Definition 15, and by CVA Grassroots (Theorem 5) the platform is grassroots.

H Proofs

Lemma 1 (Friendship Mutuality). *In any run of the social graph,* $q \in F_p \iff p \in F_q$ *for all* $p, q \in P$.

Proof. By induction on run length. Initially $F_p = \emptyset$ for all p, so the biconditional holds vacuously. Befriend(p, q) adds q to F_p and p to F_q symmetrically; Unfriend(p, q) removes them symmetrically; unrelated pairs are unchanged.

Theorem 1 (The Social Graph is Grassroots). *The social graph is grassroots.*

Proof. Obliviousness. Both transactions have nonempty guards: Befriend by $\{p, q\}$ and Unfriend by $\{p\}$ or $\{q\}$. By Guarded Obliviousness (Section 2), the social graph is oblivious. Interactivity. For disjoint nonempty $P, P' \subset \Pi$, a correct run of $\mathcal{F}(P \cup P')$ in which some $p \in P$ and $q \in P'$ will Befriend and the transaction is taken is no interleaving of correct runs of $\mathcal{F}(P)$ and $\mathcal{F}(P')$, since the Befriend step changes the local states of both p and q.

Lemma 2 (Friend-of-Friend Exactness). *In any run of the secure social graph in which no state loss occurs, for every* $p \in P, q \in F_p$*:* $N_p(q) = F_q$.

Proof. By induction on run length. Initially every friend set is empty, so the claim holds vacuously. We check each transaction that writes a friend set or a friend-of-friend map.

Befriend(p, q). It sets $N'_p(q) = F'_q$ and $N'_q(p) = F'_p$ for the new edge, and for every prior friend $r \in F_p$ sets $N'_r(p) = F'_p$ and likewise for F_q, so every record of the two changed sets F'_p, F'_q is updated to match. Records of unchanged friend sets are untouched and remain exact by hypothesis.

Unfriend(p, q). Symmetric: p and q are removed from each other's maps, and every remaining friend's record of the two changed sets is updated to F'_p, F'_q.

Replace(p, p'). p' inherits N_p, exact by hypothesis; each friend r replaces its record of p by a record of p' at the same value $N_r(p) = F_p = F_{p'}$; and every s recording a changed F_r has $N_s(r)$ refreshed to F'_r. All changed sets are thus matched.

No state loss occurs by hypothesis, and Recover is enabled only after a state loss; so these are the only transactions to consider, and exactness is preserved.

Theorem 2 (The Secure Social Graph is Grassroots). *The secure social graph is grassroots.*

Proof. Obliviousness. Befriend, Unfriend, and Replace have nonempty guards. StateLoss and Recover are unguarded but never cross-group-enabled in an interleaving of disjoint runs: in a run of $\mathcal{F}(P)$ every friend set lies within P, so StateLoss's participant $\{p\} \subseteq P$ and Recover's participants $\{p, q\}$ with q a friend (custodian) of p lie in P. Hence every cross-group transaction has a nonempty guard, and by Guarded Obliviousness (Section 2) the secure social graph is oblivious. Interactivity. Inherited from the social graph (Theorem 1): Befriend changes both participants' states.

Theorem 3 (Implementation). *With the projection σ of Definition 6, the secure social graph is an implementation of the social graph in the sense of Definition 11, correct and complete in the sense of Definition 12.*

Proof. σ is an implementation. At the initial secure configuration every $F_p = \emptyset$, so $\sigma(c)_p = \emptyset$ for every p; this is the initial social-graph configuration, as required.

Each secure transaction maps to a social-graph transition or a stutter. Befriend(p, q) adds the mutual pair to F_p, F_q; under σ this adds q to $\sigma(c)_p$ and p to $\sigma(c)_q$ and changes no other projected set, the social-graph Befriend(p, q). Unfriend(p, q) removes the pair from both F_p and F_q, so q leaves $\sigma(c)_p$ and p leaves $\sigma(c)_q$, the social-graph Unfriend(p, q). Recover(p, q), on a run with no state-loss fault, sets $F'_p = N_q(p) = F_p$ by Friend-of-Friend Exactness (Lemma 2), so $\sigma(c)_p$ is unchanged—a stutter. Replace(p, p'), after relabelling p' to p, leaves every projected friend set fixed: $F'_{p'} = F_p$ and each friend r has p' in place of p, which under the relabelling is p again—a stutter.

Correct. Let r' be a correct run of the secure social graph using only its transactions (Definition 3). Its non-stutter image $\sigma(r')$ consists of social-graph Befriend and Unfriend transitions in order, as Recover and Replace project to stutters. Consecutive distinct configurations of $\sigma(r')$ are therefore social-graph transitions, so $\sigma(r')$ is a social-graph run. For liveness: a social-graph class enabled throughout a suffix of $\sigma(r')$ is a Befriend or Unfriend class whose precondition is a condition on friend sets alone; the corresponding secure class is enabled throughout the corresponding suffix of r', since the projected precondition holds exactly when the secure one does and the guards coincide. As r' is live, that class is taken in r', hence in $\sigma(r')$. So $\sigma(r')$ is live, hence correct.

Complete. Let $\rho = c_0 \to c_1 \to \cdots$ be a correct run of the social graph. We construct a secure run $\rho' = c'_0 \to c'_1 \to \cdots$ with $\sigma(c'_i) = c_i$ and $F_p(c'_i) = F_p(c_i)$ for every i and p, by induction on i. For $i = 0$, c'_0 is the initial secure configuration, and $\sigma(c'_0) = c_0$ as shown above. For the step $c_i \to c_{i+1}$: if it is a change-volition transaction of some p, let c'_{i+1} apply the same change to p's volitional state, which σ preserves. If it is Befriend(p, q), then $q \notin F_p$ and p, q will it at c_i, hence at c'_i, where friend sets and volitional states coincide; so the secure Befriend(p, q) is enabled at c'_i, and we take it. Its friend-set effect is that of the social-graph Befriend, so $F_r(c'_{i+1}) = F_r(c_{i+1})$ for every r, and its friend-of-friend updates are determined. Unfriend is the same. As Befriend and Unfriend update both endpoints, friend sets along ρ' are mutual, so $\sigma(c'_{i+1})_r = F_r(c'_{i+1}) = F_r(c_{i+1})$ for every r, that is, $\sigma(c'_{i+1}) = c_{i+1}$. No step of ρ' is a stutter under σ, each projecting to the corresponding step of ρ; hence $\sigma(\rho') = \rho$.

It remains to show that ρ' is correct. It is safe by construction. For liveness, a Befriend or Unfriend class enabled throughout a suffix of ρ' is enabled throughout the corresponding suffix of ρ, as preconditions, volitional states, and guards coincide; ρ being live, it is taken in ρ, and by construction in ρ'. Replace is never enabled, since the volitional states of ρ' are those of ρ, which has no Replace class to will. Recover(p, q) in a configuration with no state loss has $N_q(p) = F_p$ and $N_p(r) = F_r$ for every $r \in N_q(p)$ by Friend-of-Friend Exactness (Lemma 2), so its effect is the identity; a volitional machine transaction requires

$c \neq c'$ (Definition 10), so it induces no transition of ρ' and imposes no liveness obligation. Hence ρ' is live, so correct, and $\rho = \sigma(\rho')$ as Definition 12 requires.

Theorem 4 (Fault-Resilience). *With the projection σ of Definition 6 and F the state-loss faults (Definition 4), the secure social graph is an F-resilient implementation of the social graph (Definition 13), restricted to friendship-preserving runs (Definition 5).*

Proof. Let $r' \subseteq T' \cup F$ be a live friendship-preserving run, F the state-loss faults (Definition 4). The secure social graph transactions map to social-graph transitions or stutters as in the proof of Theorem 3. A state-loss fault at p sets $F_p := \emptyset$ but changes no other agent's friend set, so for each friend r of p the membership $p \in F_r$ is untouched and $p \in \sigma(c)_r$ still holds; hence $\sigma(c)_p = \{r : p \in F_r\}$ is unchanged and every $\sigma(c)_r$ is unchanged—a stutter. Recover(p, q) on a friendship-preserving run sets $F'_p = N_q(p) = F^-_p$, the friend set held just before the loss; since a state-lost agent takes no step until recovery and Befriend and Unfriend update both endpoints symmetrically, $F^-_p = \{r : p \in F_r\}$ and $\sigma(c)_p$ is unchanged—a stutter. Thus the non-stutter image $\sigma(r')$ consists of social-graph Befriend and Unfriend transitions in order, so it is a social-graph run, and the liveness argument of Theorem 3 applies verbatim, the faults and the Recover transitions being stutters. Hence $\sigma(r')$ is a correct run of the social graph.

Lemma 5 (Known-Peers Containment). *In any run of $\mathcal{F}(P)$, for every configuration c along the run and every $p \in P$: $known_p \subseteq P$.*

Proof. By induction on run length. Initially $known_p = \emptyset \subseteq P$. Only discover modifies *known*, and a discover transaction with participants $\{p, q\}$ lies in $R(P)$ only if $\{p, q\} \subseteq P$, so it adds only members of P.

Lemma 6 (Outbox Containment). *In any run of $\mathcal{F}(P)$, for every configuration c along the run, every $p \in P$, and every* $\mathsf{message}(s, r, \cdot) \in o_p$*: $r \in P$.*

Proof. Only platform transactions write to o_p, and by outbox well-formedness any message added to o_p has recipient in $known_p$. By Lemma 5, $known_p \subseteq P$.

Theorem 5 (CVA Grassroots). *Every CVA protocol is grassroots.*

Proof. Obliviousness. Discover is guarded by $\{p\}$, so by Guarded Obliviousness [22] no cross-group discover is enabled in any interleaving of independent runs. Platform transactions and Advance-date are unary, so they pose no cross-group case. The remaining case is a cross-group communicate with $p \in P$, $q \in P'$: its precondition requires $\mathsf{message}(p, q, \cdot) \in o_p$, but by Lemma 6 applied to a P-run every message in o_p has recipient in P, so the precondition fails.

Interactivity. For any disjoint nonempty $P, P' \subset \Pi$, pick $p \in P$ and $q \in P'$. The transaction $discover(p, q)$ is in $R(P \cup P')$ and, when p wills it, is enabled at $c0(P \cup P')$. A correct run of $\mathcal{F}(P \cup P')$ whose first step carries out $discover(p, q)$ exists, and is no interleaving of correct runs of $\mathcal{F}(P)$ and $\mathcal{F}(P')$ since its first step's participants span both groups.

Obliviousness and interactivity together yield grassroots [22].

Lemma 7 (Knowledge Monotonicity). *In any run of the social graph CVA implementation, once* $q \in \mathrm{dom}(\mathrm{FMap}_p)$ *or* $f \in \mathrm{dom}(\mathrm{FoFMap}_p[q])$ *becomes true, it remains true at all later states.*

Proof. By induction on the length of the run. The claim concerns the domains $\mathrm{dom}(\mathrm{FMap}_p)$ and $\mathrm{dom}(\mathrm{FoFMap}_p[q])$, so it suffices to show that no transaction removes a key from either. Every transaction that writes platform state — Accept offer, Resolve simultaneous offer, Integrate accept, End friendship, Integrate unfriend (Sections D.3, D.4), and Integrate stream update (Section D.5) — either leaves FMap_p and FoFMap_p unchanged, or assigns $\mathrm{FMap}_p[q] := x$ for some q, or assigns $\mathrm{FoFMap}_p[q][f] := x$ for some q, f. An assignment adds the key when absent and retains it when present; none deletes a key. Hence both domains are non-decreasing along the run.

Lemma 8 (Friendship Monotonicity). *In any run of the social graph CVA implementation,* $\mathrm{epoch}_p(q)$ *and* $\mathrm{epoch}_p(q, f)$ *are monotonically non-decreasing in the natural order on* $\mathbb{N}$.

Proof. By induction on the run, inspecting every transaction that writes an epoch. For the direct epoch $\mathrm{epoch}_p(q)$: Accept offer and Resolve simultaneous offer set $\mathrm{FMap}_q[p] := x$ under the precondition $\mathrm{epoch}_q(p) < x$; Integrate accept sets $\mathrm{FMap}_p[q] := x$ under $\mathrm{epoch}_p(q) < x$; End friendship sets $\mathrm{FMap}_p[q] := \mathrm{epoch}_p(q) + 1$; and Integrate unfriend sets $\mathrm{FMap}_q[p] := x$ under $\mathrm{epoch}_q(p) < x$. For the observed epoch $\mathrm{epoch}_p(q, f)$: Integrate stream update sets it under $\mathrm{epoch}_r(q, f) < x$. In every case the value written strictly exceeds the value stored, and no other transaction writes an epoch; hence both epochs are non-decreasing.

Lemma 9 (Message Bounds). *For every message that has been sent but not yet fully processed by its recipient:*
1. if $\mathsf{friend_request}(x)$ *is in transit from* B *to* A*, then* $\mathrm{epoch}_B(A) \geq x - 1$*;*
2. if $\mathsf{accept}(x)$ *is in transit from* B *to* A*, then* $\mathrm{epoch}_B(A) \geq x$*;*
3. if $\mathsf{unfriend}(x)$ *is in transit from* B *to* A*, then* $\mathrm{epoch}_B(A) \geq x$*;*
4. if $\mathsf{stream_update}(f, x)$ *is in transit from* B*, then* $\mathrm{epoch}_B(f) \geq x$.

Proof. Each bound holds at the instant the message is emitted and persists thereafter by Friendship Monotonicity (Lemma 8). (1) Offer friendship at B sends $\mathsf{friend_request}(x)$ with $x = \mathrm{epoch}_B(A) + 1$ and does not modify FMap_B, so $\mathrm{epoch}_B(A) = x - 1$ when sent. (2) Accept offer and Resolve simultaneous offer at B set $\mathrm{FMap}_B[A] := x$ before sending $\mathsf{accept}(x)$, so $\mathrm{epoch}_B(A) = x$ when sent. (3) End friendship at B sets $\mathrm{epoch}_B(A) := x$ before sending $\mathsf{unfriend}(x)$, so $\mathrm{epoch}_B(A) = x$ when sent. (4) A $\mathsf{stream_update}(f, x)$ is emitted either as a per-edge update or as a snapshot entry. A per-edge update is emitted by a transaction (Accept offer, Resolve simultaneous offer, Integrate accept, End friendship, or Integrate unfriend) that has, in the same step, just set $\mathrm{epoch}_B(f) := x$. A snapshot entry is emitted by an activating transaction with $x := \mathrm{epoch}_B(f)$ for an existing friend f. In both cases $\mathrm{epoch}_B(f) = x$ when sent.

Lemma 10 (Observer Bound). *For any observer* A *and any pair* (p, q)*, at all times* $\mathrm{epoch}_A(p, q) \leq \max(\mathrm{epoch}_p(q), \mathrm{epoch}_q(p))$.

Proof. By induction on the run. Initially $\text{epoch}_A(p, q) = 0$ and the bound holds. The only transaction that writes FoFMap_A is Integrate stream update, which sets $\text{epoch}_A(p, q) := x$ either on processing $\mathsf{stream_update}(q, x)$ from p, in which case $x \leq \text{epoch}_p(q)$ by Message Bounds (4), or, through its symmetric assignment, on processing $\mathsf{stream_update}(p, x)$ from q, in which case $x \leq \text{epoch}_q(p)$ by Message Bounds (4). Either way the new value is at most $\max(\text{epoch}_p(q), \text{epoch}_q(p))$. All other transactions leave $\text{epoch}_A(p, q)$ unchanged, and by Friendship Monotonicity the right-hand side never decreases, so the bound is preserved.

Invariant 23 (FoFMap Symmetry) *For every $p, q, f \in \Pi$ at every reachable state of the social graph CVA implementation,* $\text{epoch}_p(q, f) = \text{epoch}_p(f, q)$.

Proof. By induction on the run. In the initial state $\text{FoFMap}_p = \emptyset$, so $\text{epoch}_p(q, f) = \text{epoch}_p(f, q) = 0$. Only Integrate stream update (Section D.5) writes FoFMap, setting $\text{FoFMap}_r[q][f] := x$ and $\text{FoFMap}_r[f][q] := x$ in one step. It assigns the two mirror entries the same value simultaneously; entries it does not touch are unchanged. By the induction hypothesis the mirror entries were equal before the step, so they remain equal after it. Hence $\text{epoch}_p(q, f) = \text{epoch}_p(f, q)$ at every reachable state.

Lemma 11 (Monotone Integration). *Let m be a message from p to q concerning the friendship between p and r (possibly $r = q$) with epoch $m.e$, and let e be the epoch q holds for that friendship. If $m.e \leq e$, m is not consumed and does not change q's state.*

Proof. Here e is $\text{epoch}_q(p)$ when $r = q$ and $\text{epoch}_q(p, r)$ otherwise. The transactions at q that consume m are, for $r = q$, Accept offer and Resolve simultaneous offer on a $\mathsf{friend_request}$, Integrate accept on an accept, and Integrate unfriend on an $\mathsf{unfriend}$ (Sections D.3, D.4); and, for $r \neq q$, Integrate stream update on a $\mathsf{stream_update}$ (Section D.5). Each requires $e < m.e$. With $m.e \leq e$ none is enabled, and by Friendship Monotonicity (Lemma 8) e never decreases, so none becomes enabled later and m is never consumed. The only preconditions at q that mention a message other than the one consumed concern q's own outbox, so a message sitting in i_q enables and disables nothing: q takes the same transactions, with the same effects, as in the run without m.

Lemma 12 (Post-Accept State). *Suppose p sent $\mathsf{accept}(x)$ to q at time t_a (via Accept offer or Resolve simultaneous offer), and neither p nor q willed End friendship on the other during $(t_a, t]$, for some $t \geq t_a$. Then* $\text{epoch}_q(p) \in \{x - 1, x\}$ *at t.*

Proof. p sends $\mathsf{accept}(x)$ only from Accept offer or Resolve simultaneous offer, each fired on receiving $\mathsf{friend_request}(x)$ from q, which q sent from Offer friendship toward p. By Message Bounds (1) (Lemma 9), $\text{epoch}_q(p) \geq x - 1$ from that send onward. The only transaction that raises $\text{epoch}_q(p)$ to x is Integrate accept on the reply from p; any value above x requires End friendship or a fresh Offer, both excluded on $(t_a, t]$ by hypothesis. By Friendship Monotonicity (Lemma 8), $\text{epoch}_q(p) \in \{x - 1, x\}$ at t.

Lemma 13 (Post-Unfriend State). *Suppose* $\mathrm{epoch}_p(q) = \mathrm{epoch}_q(p) = x$ *with* x *odd at time* t_0*, at least one of* p, q *willed End friendship on the other at some* $t_1 \geq t_0$*, and neither willed Offer friendship on the other during* $[t_0, t]$*. Then* $\mathrm{epoch}_p(q), \mathrm{epoch}_q(p) \in \{x, x+1\}$ *for all* $t \geq t_1$*.*

Proof. Without loss of generality p willed End friendship(q) at t_1, setting $\mathrm{epoch}_p(q) := x+1$ and sending $\mathsf{unfriend}(x+1)$. By Friendship Monotonicity $\mathrm{epoch}_p(q) \geq x+1$ thereafter, and advancing past $x+1$ needs a fresh Offer (excluded), so $\mathrm{epoch}_p(q) = x+1$. For q: starting from x at t_0, the only writes available without a fresh Offer are q willing End friendship toward p or q executing Integrate unfriend on $\mathsf{unfriend}(x+1)$, each setting $\mathrm{epoch}_q(p) := x+1$. Hence $\mathrm{epoch}_q(p) \in \{x, x+1\}$.

Lemma 14 (Observed-Epoch Provenance). *If* $\mathrm{epoch}_p(q, f) = x \geq 1$ *at time* t_1*, then there is a time* $t_2 \leq t_1$ *at which* $\mathrm{epoch}_q(f) = x$ *or* $\mathrm{epoch}_f(q) = x$*.*

Proof. By induction on the run. Initially every observed epoch is 0, so the claim is vacuous. The only transaction writing FoFMap_p is Integrate stream update, which sets $\mathrm{epoch}_p(q, f) := x$ (with its mirror) on processing $\mathsf{stream_update}(f, x)$ from q, or on processing $\mathsf{stream_update}(q, x)$ from f through the symmetric assignment. In the first case the message was emitted by q at a time $t_2 \leq t_1$ when $\mathrm{epoch}_q(f) = x$, in the second by f when $\mathrm{epoch}_f(q) = x$, in both cases by the emission rule underlying Message Bounds (4) (Lemma 9).

Theorem 11 (Friend List Soundness). *If* $q \in \widetilde{F}_p$ *at time* t_3*, then there are times* $t_1, t_2 \leq t_3$ *such that* p *wants to befriend* q *at* t_1 *and* q *wants to befriend* p *at* t_2*.*

Proof. That $q \in \widetilde{F}_p$ means $\mathrm{epoch}_p(q)$ is odd at t_3; let $s \leq t_3$ be the last time p wrote this odd value. The writing transaction is Accept offer(q), Resolve simultaneous offer(q), or Integrate accept on an accept from q.

Intent of p. In the first two cases p willed the transaction, an intent act toward q; set $t_1 := s$. In the third, Integrate accept completes an Offer friendship(q) that p willed at some $t_1 < s$; take that t_1. In all cases p willed an offer or acceptance toward q at $t_1 \leq t_3$. Since s is the last write of $\mathrm{epoch}_p(q)$ and it is odd, p willed no End friendship(q) in $(t_1, t_3]$ (such a transaction would advance the epoch to an even value). Hence p wants to befriend q at t_1.

Intent of q. The transaction at s was triggered by a message from q: a $\mathsf{friend_request}$ (triggering Accept offer or Resolve simultaneous offer), sent by the willed Offer friendship(p) of q; or an accept (triggering Integrate accept), sent by the willed Accept offer or Resolve simultaneous offer of q toward p. Either is a willed offer or acceptance by q toward p at some $t_2 \leq s \leq t_3$; at that instant the interval $(t_2, t_2]$ is empty, so q wants to befriend p at t_2.

Theorem 12 (Friend-of-Friend Soundness). *If* $\mathrm{epoch}_p(q, r)$ *is odd at time* t_3*, then there are times* $t_1, t_2 \leq t_3$ *such that* q *wants to befriend* r *at* t_1 *and* r *wants to befriend* q *at* t_2*.*

Proof. Let $x = \mathrm{epoch}_p(q, r)$ at t_3, odd. By Observed-Epoch Provenance (Lemma 14) there is $t' \le t_3$ with $\mathrm{epoch}_q(r) = x$ or $\mathrm{epoch}_r(q) = x$; without loss of generality $\mathrm{epoch}_q(r) = x$, odd, so $r \in \widetilde{F}_q$ at t'. Friend List Soundness (Theorem 11) applied to q at t' yields $t_1, t_2 \le t' \le t_3$ with q wanting to befriend r at t_1 and r wanting to befriend q at t_2.

Theorem 13 (Addressability). *If $q \in \widetilde{F}_p$ at time t, then $q \in known_p$ and $p \in known_q$ at t.*

Proof. Addressability in CVA is mutual known-set membership, which permits each side to send to the other (Section 5). Known sets never shrink: only Discover writes them, and only by union (Definition 15). It therefore suffices to exhibit, for each direction, a handshake step that placed the peer in the set.

The value $\mathrm{epoch}_p(q)$ became odd via Accept offer, Resolve simultaneous offer, or Integrate accept. In the first two, p sends $\mathsf{accept}(x)$ to q in the same step; in the third, Integrate accept completes an Offer friendship(q) in which p had sent friend_request to q. A platform transaction emits a message to q only when $q \in known_p$, hence $q \in known_p$. Symmetrically, q sent p a friend_request (if q offered) or an accept (if q accepted) during the handshake, hence $p \in known_q$. Both memberships persist by monotonicity.

Lemma 15 (Mutual Friendship Liveness). *If p wants to befriend q at every time $t' \ge t$ and q wants to befriend p at every time $t' \ge t$, then there is a time $t^* > t$ such that, from t^* onward,* $\mathrm{epoch}_p(q) = \mathrm{epoch}_q(p)$ *and this common value is odd.*

Proof. By hypothesis there are witnessing offer-or-accept acts by p toward q and by q toward p at times $\le t$; and, since wanting holds at every $t' \ge t$, neither agent wills End friendship on the other at any time $\ge t$ — an End friendship would set its own epoch even and, being its latest act toward the other, falsify wanting at the next state until a later offer, contradicting the hypothesis. Hence after t no unfriend passes between p and q, and no epoch between them is advanced by End friendship or Integrate unfriend.

Finitely many direct messages after t. The direct messages between p and q are friend_request, accept, and unfriend; the last is excluded after t. Offer friendship emits a friend_request only while the offerer is inactive, and Accept offer or Resolve simultaneous offer emits an accept only in response to a friend_request. In a correct run every such message is eventually delivered and integrated; a delivered friend_request from a wanting recipient yields an accept whose integration makes both sides active, after which Offer friendship emits nothing further. Thus only finitely many direct messages pass between p and q after t, and there is a time $t_0 > t$ by which all have been integrated and no further direct message between p and q is ever sent.

Both active at t_0. A handshake completes, so $\mathrm{epoch}_p(q)$ and $\mathrm{epoch}_q(p)$ are odd at t_0; with no even-writing transition available after t, Friendship Monotonicity (Lemma 8) keeps them odd thereafter.

Agreement via Message Bounds. Let $x = \max(\text{epoch}_p(q), \text{epoch}_q(p))$ at t_0; without loss of generality $\text{epoch}_p(q) = x$. The last transaction setting $\text{epoch}_p(q) := x$ is Integrate accept on an $\mathsf{accept}(x)$ from q, giving $\text{epoch}_q(p) \geq x$ by Message Bounds (2) (Lemma 9); or Accept offer / Resolve simultaneous offer, which sends $\mathsf{accept}(x)$ to q in the same step, a message integrated by t_0, so again $\text{epoch}_q(p) \geq x$. As x is the maximum, $\text{epoch}_q(p) = x = \text{epoch}_p(q)$.

Set $t^* := t_0$. After t^* no direct message between p and q is sent or pending and no volition between them remains, so neither epoch changes; hence $\text{epoch}_p(q) = \text{epoch}_q(p) = x$, odd, from t^* onward.

Theorem 14 (Friendship Establishment). *If p wants to befriend q at every time $t' \geq t$ and q wants to befriend p at every time $t' \geq t$, then there is a time $t^* > t$ such that, from t^* onward, $q \in \widetilde{F}_p$ and $p \in \widetilde{F}_q$.*

Proof. By Mutual Friendship Liveness (Lemma 15) there is $t^* > t$ from which onward $\text{epoch}_p(q) = \text{epoch}_q(p)$ is a common odd value. By the definition of $\widetilde{F}$ as the odd-epoch entries, $q \in \widetilde{F}_p$ and $p \in \widetilde{F}_q$ from t^* onward.

Theorem 15 (Friend-of-Friend Visibility). *If p wants to befriend q at every time $t' \geq t_1$, q wants to befriend p at every time $t' \geq t_1$, q wants to befriend r at every time $t' \geq t_2$, and r wants to befriend q at every time $t' \geq t_2$, then there is a time $t^* > \max(t_1, t_2)$ such that, from t^* onward, $\text{epoch}_p(q, r)$ is odd, so (q, r) appears in the friend-of-friend view at p.*

Proof. By Mutual Friendship Liveness (Lemma 15) applied to (q, r), there is a time from which $\text{epoch}_q(r) = \text{epoch}_r(q) = e$, a common odd value, stably. By Friendship Establishment (Theorem 14) applied to (p, q), there is a time from which $q \in \widetilde{F}_p$ and $p \in \widetilde{F}_q$ stably; in particular $p \in \text{dom}(\text{FMap}_q) = \text{Rec}_q$.

We show p eventually processes a $\mathsf{stream_update}(r, e)$ from q. Consider the transaction at which q set $\text{epoch}_q(r) := e$. If $p \in \text{Rec}_q$ at that step, q emits the per-edge $\mathsf{stream_update}(r, e)$ to p. Otherwise p joins Rec_q only afterwards, and the activating transaction of the p–q edge sends p the on-join snapshot, which includes $\mathsf{stream_update}(r, \text{epoch}_q(r))$ for the existing friend r; since $\text{epoch}_q(r) = e$ is stable by then, the value conveyed is e. Either way, by eventual delivery p processes a $\mathsf{stream_update}(r, e)$ from q with $q \in \text{dom}(\text{FMap}_p)$, and Integrate stream update sets $\text{epoch}_p(q, r) := e$.

Let $t^* > \max(t_1, t_2)$ be a time past all the above. From t^* onward, Friendship Monotonicity (Lemma 8) gives $\text{epoch}_p(q, r) \geq e$, and Observer Bound (Lemma 10) gives $\text{epoch}_p(q, r) \leq \max(\text{epoch}_q(r), \text{epoch}_r(q)) = e$. Hence $\text{epoch}_p(q, r) = e$, odd, from t^* onward.

Theorem 16 (Unfriend Propagation). *If from some time t onward p does not want to befriend q, or from t onward q does not want to befriend p, then there is a time $t^* \geq t$ such that, from t^* onward:*

1. *$q \notin \widetilde{F}_p$ and $p \notin \widetilde{F}_q$; and*
2. *for every r that is a sustained mutual friend of p or of q, $\text{epoch}_r(p, q)$ is even, so (p, q) does not appear in the friend-of-friend view at r.*

Proof. Without loss of generality p does not want to befriend q at any time $\geq t$.

(1). Suppose $\text{epoch}_p(q)$ were odd at some $t'' \geq t$, that is $q \in \widetilde{F}_p$. By Friend List Soundness (Theorem 11), p wants to befriend q at some $t_1 \leq t''$, with no End friendship(q) willed by p in $(t_1, t'']$. If $t_1 \geq t$ this contradicts the hypothesis directly; if $t_1 < t$ then, since $t \in (t_1, t'']$, p wants to befriend q at t (witness t_1), again a contradiction. Hence $\text{epoch}_p(q)$ is even for all $t'' \geq t$. Applied to $\text{epoch}_q(p)$, the same argument (an odd value yields, by Friend List Soundness at q, a time $\leq t''$ at which p wants to befriend q, with the same contradiction) shows $\text{epoch}_q(p)$ is even for all $t'' \geq t$. Thus $q \notin \widetilde{F}_p$ and $p \notin \widetilde{F}_q$ from t onward.

(2). If $\text{epoch}_r(p, q)$ is never odd from t onward, the claim holds. Otherwise, by Observed-Epoch Provenance (Lemma 14) some direct side of the p–q edge was once odd, so p and q were mutual friends at a common odd epoch which, by part (1) and the hypothesis, was then ended and never re-established; by Post-Unfriend State (Lemma 13) and eventual delivery of the unfriend, $\text{epoch}_p(q) = \text{epoch}_q(p) = e'$ for a common even value e' from some time on. Let r be a sustained mutual friend of p (the case of q is symmetric, using the stream of q). By Friendship Establishment (Theorem 14) applied to (p, r), eventually $r \in \text{Rec}_p$ stably. The transaction that set $\text{epoch}_p(q) := e'$ emits $\mathsf{stream_update}(q, e')$ to Rec_p; if $r \in \text{Rec}_p$ then, r receives it; otherwise r joins afterwards and the on-join snapshot from p carries $\mathsf{stream_update}(q, \text{epoch}_p(q)) = \mathsf{stream_update}(q, e')$. Either way, by eventual delivery r processes $\mathsf{stream_update}(q, e')$ from p with $p \in \text{dom}(\text{FMap}_r)$, and Integrate stream update sets $\text{epoch}_r(p, q) := e'$. By Friendship Monotonicity (Lemma 8), $\text{epoch}_r(p, q) \geq e'$ thereafter; by Observer Bound (Lemma 10), $\text{epoch}_r(p, q) \leq \max(\text{epoch}_p(q), \text{epoch}_q(p)) = e'$. Hence $\text{epoch}_r(p, q) = e'$, even, so (p, q) does not appear in the friend-of-friend view at r.

Take t^* to be a time past all the events above.

Lemma 16 (Half Friendship Mutuality at CVA). *Consider a run of the secure social graph CVA implementation that uses only the Befriend, Unfriend, and Replace transactions (no Periodic Re-broadcast and no Restore) in which, whenever an agent is replaced, its cascade reaches every friend connected to its vouched seed through the disseminated graph (the hypothesis of Theorem 18). If* $\text{epoch}_p(q) = x$ *with x odd at time t, then at t at least one of:*

1. $\text{epoch}_q(p) \in \{x, x + 1\}$*;*
2. *a* friend_request*,* accept*,* unfriend*, or* rebind *message of epoch $\geq x$ between p and q is in transit;*
3. *q has been replaced—q ran Replace to a new identity q'—and p has not yet integrated the rename, so a* $\mathsf{new_identity}(q, q', \cdot)$ *addressed to p is in transit or in i_p.*

Proof. By Friendship Monotonicity (Lemma 8), $\text{epoch}_p(q)$ is non-decreasing; let T be the last transition up to t that set $\text{epoch}_p(q) := x$, so $\text{epoch}_p(q) = x$ throughout $[T, t]$.

Replaced q. Suppose q ran Replace, to q', at some time $\leq t$. By this hypothesis q's cascade reaches p, so a $\mathsf{new_identity}(q, q', \cdot)$ is addressed to p. Had p executed

Integrate new identity on it, $\mathrm{FMap}_p[q]$ would have been removed (renamed to q'), contradicting $\mathrm{epoch}_p(q) = x$ at t; so p has not, and the message is in transit to p or lies in i_p—case (3). Henceforth assume q is not replaced by t; then $\mathrm{epoch}_q(p)$ is defined and non-decreasing.

The transitions writing an odd value to $\mathrm{epoch}_p(q)$ in this fragment are Accept offer / Resolve simultaneous offer, Integrate accept, Integrate new identity (renaming some o to q at p), and Integrate rebind. We case on T.

Accept offer / Resolve simultaneous offer. T fires on $\mathsf{friend_request}(x, \cdot)$ from q and adds $\mathsf{accept}(x, \cdot)$ to o_p toward q in the same step. If that accept is unprocessed it is in transit with epoch x—case (2). Otherwise q processed it by Integrate accept, setting $\mathrm{epoch}_q(p) := x$. As $\mathrm{epoch}_p(q) = x$ is odd at t, p integrated no $\mathsf{unfriend}$ from q after T; so either no End friendship between them occurred and $\mathrm{epoch}_q(p) = x$ (case 1), or one did and Post-Unfriend State (Lemma 13) gives $\mathrm{epoch}_q(p) \in \{x, x+1\}$ (case 1) unless the $\mathsf{unfriend}(x+1)$ is still in transit to p (case 2).

Integrate accept. T fires on $\mathsf{accept}(x, \cdot)$ from q, which q sent only after setting $\mathrm{epoch}_q(p) := x$ (Message Bounds (2), Lemma 9); so $\mathrm{epoch}_q(p) \geq x$, and the End-friendship analysis above gives case (1) or case (2).

Integrate new identity (rename $o \to q$ at p). p renamed a replaced friend o to its new identity q, setting $\mathrm{epoch}_p(q) := \mathrm{epoch}_p(o) = x$ and adding $\mathsf{rebind}(x, \cdot, \cdot, \cdot)$ to o_p toward q in the same step. If unprocessed, that rebind is in transit with epoch x—case (2); otherwise q (the recovered identity) processed it by Integrate rebind, setting $\mathrm{epoch}_q(p) := x$—case (1).

Integrate rebind. Here p is a recovered identity; T fires on $\mathsf{rebind}(x, \cdot, \cdot, \cdot)$ from q, which q sent (in its Integrate new identity) only after setting $\mathrm{epoch}_q(p) := x$. Hence $\mathrm{epoch}_q(p) \geq x$, giving case (1) absent further activity or case (2) with a later message in transit.

In every case one of (1)–(3) holds.

Theorem 6 (Secure Quiescent Correspondence). *Each completed transaction protocol of the secure social graph CVA implementation realises, under the mapping $F_p \mapsto \widetilde{F}_p$, the corresponding transaction of the abstract secure social graph (Section 4.2); the correspondence is at the granularity of completed protocols at quiescence, not a step-by-step bisimulation:*

1. *A completed Befriend handshake in CVA realises the abstract Befriend(p, q).*
2. *A completed Unfriend in CVA realises the abstract Unfriend(p, q).*
3. *A completed Replace cascade in CVA (Vouch, Announce new identity, Integrate new identity, Integrate rebind) realises the abstract Replace(p, p') on every friendship it reaches; on a friendship-preserving run (Definition 5) it reaches every recoverable friend. The one unrecoverable friendship—recorded, at the fault, only by the two friends, the identity of one then lost—is not recovered; the specification does not require its recovery (Definition 5).*
4. *A completed Restore in CVA, after a state loss at p, realises the abstract Recover(p, q): it sets $\widetilde{F}_p$ to the friends that still record p, which at quiescence is the full set of friendships recorded at p before the state loss.*

Proof. The configuration is read against the abstract friend set $\widetilde{F}$ (Definition 26), each live agent's friendships as recorded in its own state, with each replaced identity relabelled to the original it replaced.

(1), (2). A completed Befriend handshake (Offer friendship, Accept offer or Resolve simultaneous offer, Integrate accept, with the emitted stream updates delivered) leaves $\mathrm{epoch}_p(q) = \mathrm{epoch}_q(p)$ at a common odd value: at quiescence, with neither party replaced, Half Friendship Mutuality (Lemma 16, case 1 with no message in transit) gives $\mathrm{epoch}_q(p) \in \{\mathrm{epoch}_p(q), \mathrm{epoch}_p(q) + 1\}$, and the completed handshake with no intervening unfriend forces equality; each records the other active—$q \in \widetilde{F}_p$ and $p \in \widetilde{F}_q$, the effect of abstract Befriend(p, q) on each side's friend set. A completed Unfriend (End friendship, Integrate unfriend) advances both epochs to the next even value, removing q from $\widetilde{F}_p$ and p from $\widetilde{F}_q$—abstract Unfriend(p, q). The identity-record fields, set once and preserved, are inert to the mapping.

(3). By Replace Convergence (Theorem 18), on a run quiescent at p before Replace the cascade renames every $r \in F_p$ from p to p', yields $\widetilde{F}_{p'} = F_p$, and leaves no FMap_r retaining p. This is exactly abstract Replace(p, p'): $F'_{p'} := F_p$, $F'_p := \emptyset$, and each $r \in F_p$ substitutes p' for p. More generally, on any friendship-preserving run (Definition 5) the cascade reaches every recoverable friend, and abstract Replace holds on each such edge. The one unrecoverable friendship—recorded, at the fault, only by the two friends, so that no stream_update carried it to another agent—is not recovered: the fresh identity p' holds no record of w, so $w \notin \widetilde{F}_{p'}$, and w retains a stale record of the retired key. The specification does not require its recovery (Definition 5; the Remark in Section D.17).

(4). After a state loss at p, Restore (Section D.15) is carried entirely by Integrate checkpoint: each friend r that still records p keeps $p \in \mathrm{Rec}_r$ and re-broadcasts checkpoint(IR_r, L_r) with the pair $(p, \mathrm{epoch}_r(p)) \in L_r$; on the first such checkpoint, p installs the skeleton $\mathrm{FMap}_p[r]$ from the carried IR_r and the direct-heal clause sets $\mathrm{epoch}_p(r) := \mathrm{epoch}_r(p)$. A carried epoch is odd only when r records the r–p edge active, which by Message Bounds (Lemma 9) holds only after an offer or acceptance by p at that epoch; so each recovered active r has $p \in F_r$ at the recovering state, and the edge is mutual once p re-records it. At quiescence every such r has re-broadcast to p (Section D.14), so $\widetilde{F}_p$ becomes exactly the set of friendships recorded at p before the state loss, the value the abstract Recover(p, q) assigns from any one custodian q that holds the friend-of-friend record for p (Section 4.2). Passive recovery is complete and needs no custodian authorisation, the asymmetry with Replace noted in Section D.15.

Theorem 18 (Replace Convergence). *Suppose the run is quiescent at p before Replace (Definition 27) and every $c \in K_p$ vouches (so $V = K_p$). Then the cascade reaches every friend of p: each $w \in F_p$ receives* new_identity*, renames $p \to p'$, and rebinds. After the cascade terminates, $\widetilde{F}_{p'} = F_p$ and for every $r \in F_p$ the friendship between r and p has been renamed to one between r and p', realising the abstract Replace exactly.*

Proof. Fix $w \in F_p$, so $\mathrm{epoch}_p(w)$ is odd.

Coverage. When p last set $\text{epoch}_p(w)$ it emitted $\mathsf{stream_update}(w, \text{epoch}_p(w))$ to every $r \in \text{Rec}_p = \text{dom}(\text{FMap}_p)$. By quiescence at p before Replace, every custodian $c \in K_p$ has integrated it, so $w \in \text{dom}(\text{FoFMap}_c[p])$ and $(w, \cdot) \in L_c$. Hence w lies in the known-friend set N at the first firing of Announce new identity. As p' discovers each named friend (Discover is always enabled at p'; Section 5), eventually $w \in \mathit{known}_{p'}$, and Announce sends $\mathsf{message}(p', w, \mathsf{new_identity}(p, p', K_p, IR_{p'}))$.

Rename and rebind. By Half Friendship Mutuality at quiescence (Lemma 16, case 1, with no message in transit) $\text{epoch}_w(p) = \text{epoch}_p(w)$, so $p \in \text{dom}(\text{FMap}_w)$ with $IR_w[p] = (K_p, \sigma_p)$. On delivery, w executes Integrate new identity: the vouching set $V = K_p$ satisfies $V \subseteq K_p$ and $|V| = |K_p| \geq \lceil \sigma_p |K_p| \rceil$, so the guard passes; w renames $p \to p'$ in FMap_w and FoFMap_w, carrying the epoch $e_w := \text{epoch}_w(p)$, and sends $\mathsf{rebind}(e_w, IR_w, L_w, L_w^p)$ to p'. By reliable delivery p' processes it through Integrate rebind, setting $\text{FMap}_{p'}[w] := (e_w, IR_w)$, a full entry; since $e_w = \text{epoch}_p(w)$ is odd, $w \in \widetilde{F}_{p'}$.

Outcome. As $w \in F_p$ was arbitrary, every friend of p is renamed and rebinds, so $\widetilde{F}_{p'} = F_p$; and each such r executed Integrate new identity, after which $p \notin \text{dom}(\text{FMap}_r)$, so no FMap_r retains p. This is abstract $\text{Replace}(p, p')$ exactly.

Theorem 17 (CVA Fault-Resilience). *Let F'' be the CVA state-loss faults: the resets of known_p, o_p, i_p, FMap_p, and FoFMap_p to empty (Section D.15), disjoint from the CVA transactions, and let $\sigma'' := (F_p \mapsto \widetilde{F}_p)$. Restricted to friendship-preserving runs, the secure social graph CVA implementation is, at quiescence, an F''-resilient implementation of the abstract secure social graph (Definition 13): a state-loss fault and the Restore that follows it return $\widetilde{F}_p$ to its pre-fault value, so at quiescence they are a stutter and σ'' maps every live run over $T'' \cup F''$ to a correct run of the abstract secure social graph.*

Proof. Befriend, Unfriend, and Replace protocols realise the corresponding abstract transitions at quiescence by the Correspondence (Theorem 6). A state-loss fault empties FMap_p, so momentarily $\widetilde{F}_p = \emptyset$; on a friendship-preserving run Restore is complete (Section D.15, Lemma 17), so at the next quiescence $\widetilde{F}_p$ is again the set of friendships p recorded before the fault. The fault and its Restore thus net to no change in $\widetilde{F}_p$, with the intermediate states non-quiescent, so observed at quiescence the round-trip is a stutter. Hence the quiescent image of a live friendship-preserving run over $T'' \cup F''$ uses only abstract Befriend, Unfriend, and Replace, and is a correct run of the abstract secure social graph; that is, σ'' is F''-resilient.

Lemma 18 (Recovery Requires a Record). *Let the friendship between p and w be unrecoverable at time t (Definition 5). Then in every continuation in which neither person wills a fresh Offer friendship or acceptance toward the other, the two never again record each other.*

Proof. Every transaction that installs a name in FMap_x or FoFMap_x other than Accept offer, Resolve simultaneous offer and Integrate accept—which the excluded volition rules out—reads that name from an incoming message, and every

message carries only names drawn from its sender's own maps. By unrecoverability no agent other than p and w records the edge at t, and the fault erases it at the endpoint that lost its state or its identity. Hence no message from t onward carries w to p's side, or p's successor to w: under state loss neither endpoint holds the other, and under identity loss w is named in no L_c and in no L_p, so Announce new identity never addresses w and no rebind reveals w to p'. By induction on the run neither ever records the other.

Lemma 17 (Repair after Faults). *Consider a friendship-preserving run with finitely many faults, the last at time t_F. Then there is a time $t^* \geq t_F$ from which onward no staleness caused by the faults remains:* $\mathrm{epoch}_p(q)$ *agrees with* $\mathrm{epoch}_q(p)$ *except while a volitional transaction between p and q is in progress, and* $\mathrm{epoch}_p(q, r)$ *equals* $\mathrm{epoch}_q(r)$ *for every observer p. From t^* the run is indistinguishable from one of the same volitions in which no message was lost, so Friendship Establishment (Theorem 14), Friend-of-Friend Visibility (Theorem 15) and Unfriend Propagation (Theorem 16) hold with t^* in place of the initial time.*

Proof. A fault erases only the faulting agent's own state and the messages in its boxes, so the only staleness it introduces is at an agent that missed a message the faulting agent had sent or was to receive. Faults are finite in number and none occurs after t_F, so finitely many such divergences arise, all by t_F.

Direct. Suppose $\mathrm{epoch}_p(q) > \mathrm{epoch}_q(p)$, the effect of a lost accept or unfriend; a lost friend_request writes no entry and leaves no divergence. Re-broadcast is unguarded and enabled whenever $\mathrm{Rec}_p \neq \emptyset$, so p re-broadcasts infinitely often, and by eventual delivery q integrates a checkpoint carrying $(q, \mathrm{epoch}_p(q))$ in L; the direct-heal clause finds it above $\mathrm{epoch}_q(p)$ and adopts it (Definition 24).

Observed. Suppose $\mathrm{epoch}_r(p, q) < \mathrm{epoch}_p(q)$, the effect of a lost stream_update. Every periodic checkpoint of p carries $(q, \mathrm{epoch}_p(q))$ in its snapshot, which the observer-heal clause absorbs; by the Observer Bound (Lemma 10) it rises no higher.

A state loss is the case in which the faulting agent lost its whole state; on a friendship-preserving run its friends still record it, and their checkpoints rebuild it by the same two clauses (Section D.15). Each divergence is therefore repaired within finitely many re-broadcast rounds; let t^* be the time by which all have been. No divergence arises after t_F, so from t^* every direct and observed epoch holds the value a loss-free run of the same volitions would hold, and the proofs of the three cited theorems apply verbatim with t^* in place of the initial time.